

Learn PocketFlow and PocketFlow Creator

PocketFlow Creator Community

June 2026

- [Learn PocketFlow and PocketFlow Creator](#)
- [Introduction](#)
 - [What You'll Learn](#)
 - [How to Use This Guide](#)
- [Part 0: Understanding the Foundations](#)
 - [Chapter 1: What is PocketFlow?](#)
- [About PocketFlow](#)
 - [The philosophy](#)
 - [How a node works](#)
 - [Wiring a flow](#)
 - [Node base classes](#)
 - [Cookbook patterns](#)
 - [Why PocketFlow Creator was born](#)
 - [Chapter 2: What is PocketFlow Creator?](#)
- [About PocketFlow Creator](#)
 - [Design Principles](#)
 - [Application Architecture](#)
 - [Project File Layout](#)
 - [The RAD Coding Model](#)
 - [Validation Error Codes](#)
 - [Internationalization](#)
 - [Contributing](#)
- [Part 1: Getting Started](#)
 - [Chapter 3: Installation and Setup](#)
- [Getting Started](#)
 - [Installation](#)
 - [First Launch](#)
 - [The Six Panels](#)
 - [Quick-start Workflow](#)
 - [Keyboard Shortcuts](#)
 - [Next Steps](#)
 - [Chapter 4: Your First Flow](#)
- [Your First Flow](#)
 - [Step 1 — Create a New Project](#)
 - [Step 2 — Create a New Flow](#)

- [Step 3 — Add a Start Node](#)
- [Step 4 — Add an LLM Node](#)
- [Step 5 — Connect the Nodes](#)
- [Step 6 — Add a Stop Node](#)
- [Step 7 — Write the Prompt](#)
- [Step 8 — Validate](#)
- [Step 9 — Run](#)
- [Step 10 — Export](#)
- [What the Shared Store, Reads, and Writes Mean](#)
- [What You Built](#)
- [Next Steps](#)
- [Part 2: Tutorials and Learning](#)
- [Part 1 — Getting Started with PocketFlow Creator](#)
 - [Tutorial 0: IDE Tour](#)
 - [Tutorial 1: Hello LLM](#)
 - [Tutorial 2: Your First Flow — Hello World](#)
 - [Tutorial 3: Using the Properties Inspector](#)
 - [Tutorial 4: The Code Editor — RAD Node Coding](#)
 - [Tutorial 5: Creating a Custom Node Type](#)
 - [Tutorial 6: Project Templates](#)
- [Part 2 — PocketFlow Patterns in Creator](#)
 - [Tutorial 7: Hello World — Single Node Q&A](#)
 - [Tutorial 8: Chat with History](#)
 - [Tutorial 9: Structured Output — Resume Data Extraction](#)
 - [Tutorial 10: Multi-Stage Workflow — Article Writer](#)
 - [Tutorial 11: Conditional Routing — Chat Guardrail](#)
 - [Tutorial 12: Agent with Tools](#)
 - [Tutorial 13: Retrieval-Augmented Generation \(RAG\)](#)
 - [Tutorial 14: Map-Reduce / Batch Processing](#)
 - [Tutorial 15: Human-in-the-Loop](#)
 - [Tutorial 16: LLM-as-Judge / Evaluator Loop](#)
 - [Tutorial 17: Multi-Agent System](#)
 - [Tutorial 23: Streaming Output](#)
 - [Tutorial 24: Memory — Short-Term and Long-Term Context](#)
 - [Tutorial 26: Async Processing with AsyncNode](#)
 - [Tutorial 27: Async Batch Processing with AsyncBatchNode](#)
 - [Tutorial 28: Concurrent Async Batch with AsyncParallelBatchNode](#)
 - [Tutorial 29: Using the Agent Node](#)
 - [Tutorial 30: Using the RAG Node](#)
 - [Tutorial 31: Using the Judge Node](#)
 - [Tutorial 32: Human Input + Classifier — Interactive Sentiment Triage](#)
- [Part 3 — Advanced Creator Features](#)
 - [Tutorial 18: Validation and Error Badges](#)
 - [Tutorial 19: Debug Mode and Breakpoints](#)

- [Tutorial 20: Subflow Composition](#)
- [Tutorial 21: Exporting and Running a Standalone Project](#)
- [Tutorial 22: Shared Store Designer](#)
- [Tutorial 25: Packaging for Distribution](#)
- [Part 4 — Creator System Exercises](#)
 - [Exercise A: Build a Complete News Summariser](#)
 - [Exercise B: Coding Agent with Memory](#)
 - [Exercise C: Multi-Provider LLM Router](#)
 - [Exercise D: Full IDE Workout](#)
- [Part 3: Advanced Topics](#)
 - [Hardware I/O Integration](#)
- [Tutorial: Hardware I/O Nodes — Serial, Audio, Video, and Webcam](#)
 - [Hardware I/O Node Categories](#)
 - [Example 1: Read from USB Serial \(Arduino\)](#)
 - [Example 2: Record Audio from Microphone](#)
 - [Example 3: Capture Webcam and Send to LLM](#)
 - [Example 4: Video Recording with Duration Limit](#)
 - [Example 5: Serial Communication Loop \(Arduino ↔ Creator\)](#)
 - [Installation of Optional Dependencies](#)
 - [Properties Reference](#)
 - [Tips and Best Practices](#)
 - [Troubleshooting](#)
 - [Next Steps](#)
 - [Standalone Python Scripts](#)
- [Tutorial: Exporting Standalone Python Scripts](#)
 - [Why Standalone Scripts?](#)
 - [Step 1: Create a Simple Flow](#)
 - [Step 2: Export as Standalone Script](#)
 - [Step 3: What's Inside the Standalone Script](#)
 - [Step 4: Run the Standalone Script](#)
 - [Step 4b: Interactive Nodes \(Human Input / Review\)](#)
 - [Step 5: Error Handling for Missing Dependencies](#)
 - [Supported Features in Standalone Scripts](#)
 - [Limitations](#)
 - [Next Steps](#)
- [Part 4: Reference](#)
 - [All Node Types](#)
- [Node Types Reference — Complete Catalog](#)
 - [Quick Navigation](#)
 - [Flow Control](#)
 - [AI / LLM](#)
 - [AI / Reasoning](#)
 - [Web / Search](#)
 - [Data / Vector](#)

- [Database / SQL](#)
- [Voice / Audio](#)
- [Hardware I/O](#)
- [Document / Vision](#)
- [Code / Execution](#)
- [Data Processing](#)
- [Calendar](#)
- [MCP / Agent Protocol](#)
- [Observability / Utility](#)
- [Data Structures / Memory](#)
- [Security](#)
- [Human-in-the-Loop](#)
- [Batch / Async](#)
- [I/O](#)
- [System / Shell](#)
- [Networking](#)
- [Text / Data Processing](#)
- [Resilience](#)
- [Messaging](#)
- [Context Compact Node](#)
- [Conversation History Node](#)
- [Summary](#)
- [UI Reference](#)
- [Graph Canvas](#)
 - [Basic Operations](#)
 - [Navigation](#)
 - [Auto Arrange](#)
 - [Undo / Redo](#)
 - [Node Anatomy](#)
 - [Breakpoints](#)
 - [Code Sync](#)
- [Component Palette](#)
 - [Section 1 — Built-in Nodes](#)
 - [Section 2 — Scientific & Engineering Add-on Nodes](#)
 - [Section 3 — Custom Nodes](#)
 - [Using the Palette](#)
 - [Node Lifecycle](#)
- [Object Inspector](#)
 - [Node Properties](#)
 - [Understanding Actions, Reads, and Writes](#)
 - [LLM Node Prompt Properties](#)
 - [How to Edit a Field](#)
 - [Edge Properties](#)
 - [Validation Feedback](#)

- [Code Editor](#)
 - [The Code File](#)
 - [Opening the Editor](#)
 - [Marker Format](#)
 - [Syntax Highlighting](#)
 - [Saving](#)
 - [YAML and Markdown Editors](#)
- [Project Explorer](#)
 - [Tree Structure](#)
 - [Opening Files](#)
 - [After Opening a Graph](#)
 - [Project Metadata](#)
- [Shared Store Designer](#)
 - [What Is the Shared Store?](#)
 - [Opening the Designer](#)
 - [Columns](#)
 - [Adding, Editing, Removing Rows](#)
 - [Validate](#)
 - [How It Affects Runs](#)
 - [Saving](#)
- [Provider Manager](#)
 - [Overview](#)
 - [Creating a New Profile](#)
 - [Provider Types](#)
 - [Testing Connections](#)
 - [API Key Security](#)
 - [Using Providers in Workflows](#)
 - [Troubleshooting](#)
 - [More Information](#)
 - [Custom Nodes](#)
- [Creating Custom Nodes](#)
 - [Approach 1 — Using the GUI Wizard](#)
 - [Approach 2 — Writing a Node Package](#)
 - [Approach 2b — Multi-File Node Packages](#)
 - [Installing a Package via the GUI](#)
 - [The Node Type Library Dialog](#)
 - [Load Order and the `type\_id` Namespace](#)
 - [Sharing Your Node Package](#)
 - [Developing a Package in an External Editor](#)
 - [Quick Reference](#)
- [Appendix: Quick Reference](#)
- [PocketFlow Node Reference](#)
 - [The Node Data Contract: Actions, Reads, and Writes](#)
 - [Flow Control](#)

- [LLM / AI](#)
- [Data / I/O](#)
- [Processing](#)
- [Human in the Loop](#)
- [AI / Reasoning](#)
- [Web / Search](#)
- [Data / Vector](#)
- [Database / SQL](#)
- [Voice / Audio](#)
- [Document / Vision](#)
- [Code / Execution](#)
- [Data Processing](#)
- [Calendar](#)
- [MCP / Agent Protocol](#)
- [Observability / Utility](#)
- [Data Structures / Memory](#)
- [Reference](#)
- [System / Shell / Hardware](#)
- [Networking / Sockets](#)
- [AI / LLM Utilities](#)
- [Text / Data Processing](#)
- [Resilience / Flow Utilities](#)
- [Messaging / Notifications](#)
- [Security / Configuration](#)
- [Addon Nodes — Geospatial](#)
- [Addon Nodes — Hydrology / Water](#)
- [Addon Nodes — Weather / Atmosphere](#)
- [Addon Nodes — Building Energy](#)
- [Addon Nodes — Aerospace](#)
- [Addon Nodes — Wind Energy](#)
- [Addon Nodes — Scientific Computing](#)
- [Addon Nodes — Data Catalog](#)

Learn PocketFlow and PocketFlow Creator

A Comprehensive Guide from Beginner to Expert

Introduction

Welcome to the complete learning guide for PocketFlow and PocketFlow Creator! This ebook will take you from complete beginner to proficient user, covering everything from basic concepts to advanced workflows, standalone script generation, and integration with real-world systems.

What You'll Learn

- **PocketFlow Fundamentals:** Understand the core concepts of flow-based programming
- **PocketFlow Creator IDE:** Master the visual interface and all its features
- **Building Flows:** Create simple flows, then progress to complex multi-node workflows
- **83+ Node Types:** Learn how to use data processing, AI/LLM, hardware I/O, and integration nodes
- **Advanced Patterns:** Implement error handling, conditional logic, looping, and state management
- **AI Integration:** Connect your flows to Ollama, OpenAI, Anthropic, and other LLM providers
- **Hardware Integration:** Control serial devices, capture audio/video, and interact with physical systems
- **Standalone Scripts:** Export flows as self-contained Python scripts for CI/CD pipelines
- **Custom Nodes:** Create and share your own specialized node types

How to Use This Guide

This ebook is structured to build knowledge progressively:

1. **Start with Part 1** if you're new to PocketFlow
 2. **Progress through Parts 2-4** as your skills grow
 3. **Reference the Node Types Catalog** when building specific features
 4. **Explore Hardware I/O and Standalone Scripts** for advanced use cases
 5. **Use the UI Reference** for specific tool documentation
-

Part 0: Understanding the Foundations

Chapter 1: What is PocketFlow?

About PocketFlow

PocketFlow is a minimalist LLM framework — the entire runtime is **100 lines of Python** — that expresses LLM workflows as directed graphs of nodes connected by labelled action edges.

Repository

<https://github.com/The-Pocket/PocketFlow>

Author

Zachary Huang ([@ZacharyHuang](#))

Organisation

[The Pocket](#)

Licence

MIT

The philosophy

Most LLM frameworks grow into sprawling abstractions that hide what is actually happening. PocketFlow takes the opposite approach: the framework is small enough to read in a single sitting, yet powerful enough to express every major LLM pattern — chat loops, agents, retrieval-augmented generation, batch processing, human-in-the-loop, and multi-agent systems.

The core insight is that every LLM application is a directed graph:

- **Nodes** do the work — they call LLMs, tools, databases, or run arbitrary Python.
 - **Edges** are labelled with action strings that `post()` returns, making control flow explicit and testable.
 - **The shared store** is a plain dict — the only communication channel between nodes, no hidden state.
-

How a node works

```
class MyNode(Node):  
    def prep(self, shared):  
        # Read from the shared store; prepare inputs for exec.  
        return shared.get("question", "")
```



```

def exec(self, prep_res):
    # Do the work: LLM call, tool call, file I/O, anything.
    return llm.complete(prep_res)

def post(self, shared, prep_res, exec_res):
    # Write results back; return an action string to route
    the flow.

    shared["answer"] = exec_res
    return "default"

```

Wiring a flow

```

flow = fetch_node - "ok" >> summarise_node
flow = fetch_node - "error" >> retry_node
flow = summarise_node >> done_node    # shorthand for "default"

```

`post()` returns "ok" or "error" and the framework follows the matching edge. There is no magic — the routing table is just a dict of action strings to next nodes.

Node base classes

Class	Purpose
Node	Standard single-execution node
BatchNode	Runs <code>exec()</code> once per item returned by <code>prep()</code>
AsyncNode	Async node (<code>async def exec</code>)
AsyncBatchNode	Async batch processing

Cookbook patterns

The PocketFlow repository ships 40+ cookbook examples covering:

Level	Topics
Beginner	Hello World, Chat, Structured Output, Workflow, Routing
Intermediate	Agent with tools, RAG, Map-Reduce, HITL, LLM-as-Judge, Multi-Agent
Advanced	Streaming, Memory, Subflow, Async, Web search, Code execution

Why PocketFlow Creator was born

PocketFlow's simplicity is a strength, but designing a multi-node flow by writing Python directly means holding the entire graph in your head — which nodes exist, how they connect, what actions each one declares, and how shared store keys flow between them.

PocketFlow Creator was built to give PocketFlow a visual design surface without taking away any of its transparency. The goal is the same RAD experience that made Delphi and Visual Basic productive for a generation of developers — drag a node onto a canvas, wire its action ports, inspect its properties, and see the generated Python immediately — while keeping the output as plain, readable PocketFlow code that runs without the designer installed.

The designer is a *generator*, not a runtime wrapper. Every exported project is self-contained Python that you own completely.

- [Getting Started](#)
- [Your First Flow](#)
- [About PocketFlow Creator](#)
- [Help Home](#)

Chapter 2: What is PocketFlow Creator?

About PocketFlow Creator

PocketFlow Creator is a RAD (Rapid Application Development) visual designer for building PocketFlow LLM workflows. It follows the Delphi/VB style: every visible object has properties, every transition is an action, every generated behavior can be inspected as Python, and every custom behavior belongs in user-owned files that are never overwritten.

Version: 0.2.0

Framework target: PocketFlow (any version)

Language: Python 3.10+

GUI framework: PySide6 (Qt 6)

Design Principles

Principle	Meaning
Project generator, not a runtime	

Principle	Meaning
	Generated projects run without Creator installed
Plain-text project files	.pfcproj.yaml and .pfcgraph.yaml are human-readable and version-control friendly
Never overwrite custom/	Hand-edited node implementations survive every re-export
Validate before generate	The graph must pass validation before code is generated or a flow is run
One node type model	Built-in and custom nodes use the same NodeTypeDefinition — no special cases
Offline-first	No calls to external services during design time; LLM calls only happen at run time

Application Architecture

GUI (app/) MainWindow · GraphCanvas · Editors · HelpBrowser NodeTypeWizard · SharedStoreDesigner
Services (controller layer) ProjectLoader/Saver · GraphLoader/Saver · Exporter CodeManager · FlowRunner
Domain (model / validation / generation) GraphModel · NodeModel · EdgeModel NodeTypeDefinition · ProjectModel GraphValidator · PythonGenerator
Runtime (providers) LLMProvider (protocol) · OllamaProvider · MockProvider

The GUI never imports directly from `validation/` or `generation/` — it goes through the controller layer. The domain packages are UI-agnostic pure Python and can be used from scripts, tests, or CI pipelines.

Project File Layout

```
MyProject/
├── MyProject.pfcproj.yaml      ← project metadata and settings
├── graphs/
│   └── main.pfcgraph.yaml     ← graph: nodes, edges, properties
├── prompts/
│   └── ask_llm.md             ← LLM prompt files (Markdown)
├── node_types/
│   └── my_custom_node.yaml    ← custom node type definitions
├── tools/
│   └── search.py              ← tool implementations
├── schemas/
│   └── output_schema.json     ← JSON Schema for structured
output nodes
```

After export, the standalone project appears in `exports/MyProject/`:

```
exports/MyProject/
├── generated/                 ← regenerated on every export
│   ├── nodes.py               ← node class stubs
│   └── flow.py                ← PocketFlow wiring
├── custom/                    ← NEVER overwritten – your
implementations live here
│   └── my_node_impl.py
├── tests/
│   └── test_flow.py
└── main.py
```

The RAD Coding Model

PocketFlow Creator follows the Delphi RAD model for node code:

1. **Drop a node on the canvas** → Creator adds a class stub to `code/<graph>.py`
2. **Double-click the node** → Python editor opens scrolled to that class
3. **Edit the class** → changes persist in `code/<graph>.py`
4. **Delete the node from canvas** → the class block is removed from the code file

5. **Export** → Creator merges the code file into custom/ (never overwriting existing files)

The marker format:

```
# --- NODE_START: node_abc ---  
class MyNode(Node):  
    def prep(self, shared): ...  
    def exec(self, prep_res): ...  
    def post(self, shared, prep_res, exec_res): return "default"  
# --- NODE_END: node_abc ---
```

Validation Error Codes

Code	Meaning
------	---------

PFCE1001	No start node selected
----------	------------------------

PFCE1002	Duplicate node ID
----------	-------------------

PFCE1003	Start node ID does not exist
----------	------------------------------

PFCE2001	Edge source node does not exist
----------	---------------------------------

PFCE2002	Edge destination node does not exist
----------	--------------------------------------

PFCE2003	Edge has no action label
----------	--------------------------

PFCE2101	Edge action not declared by source node
----------	---

PFCE2102	Subflow node missing or unresolved subflow_ref
----------	--

Internationalization

The UI supports multiple languages. Change the language in **Tools > Options > Language**. A restart is required for the change to take effect.

Currently shipped: **English** (en), **Spanish** (es), **French** (fr), **German** (de), **Chinese** (zh), **Japanese** (ja).

Contributing

See docs/12_developer_guide.md in the source tree for development setup, architecture notes, and contribution guidelines.

Part 1: Getting Started

Chapter 3: Installation and Setup

Getting Started

Installation

Prerequisites

- Python 3.10 or later
- pip

Install from source

```
git clone https://github.com/your-org/pocketflow-creator.git
cd pocketflow-creator
./scripts/setup-prj.sh           # creates .venv and installs all
dependencies
```

Launch the application

```
./scripts/run_app.sh           # Linux / macOS
scripts\run_app.bat           # Windows
```

Or, if installed into an active virtualenv:

```
pocketflow-creator
```

First Launch

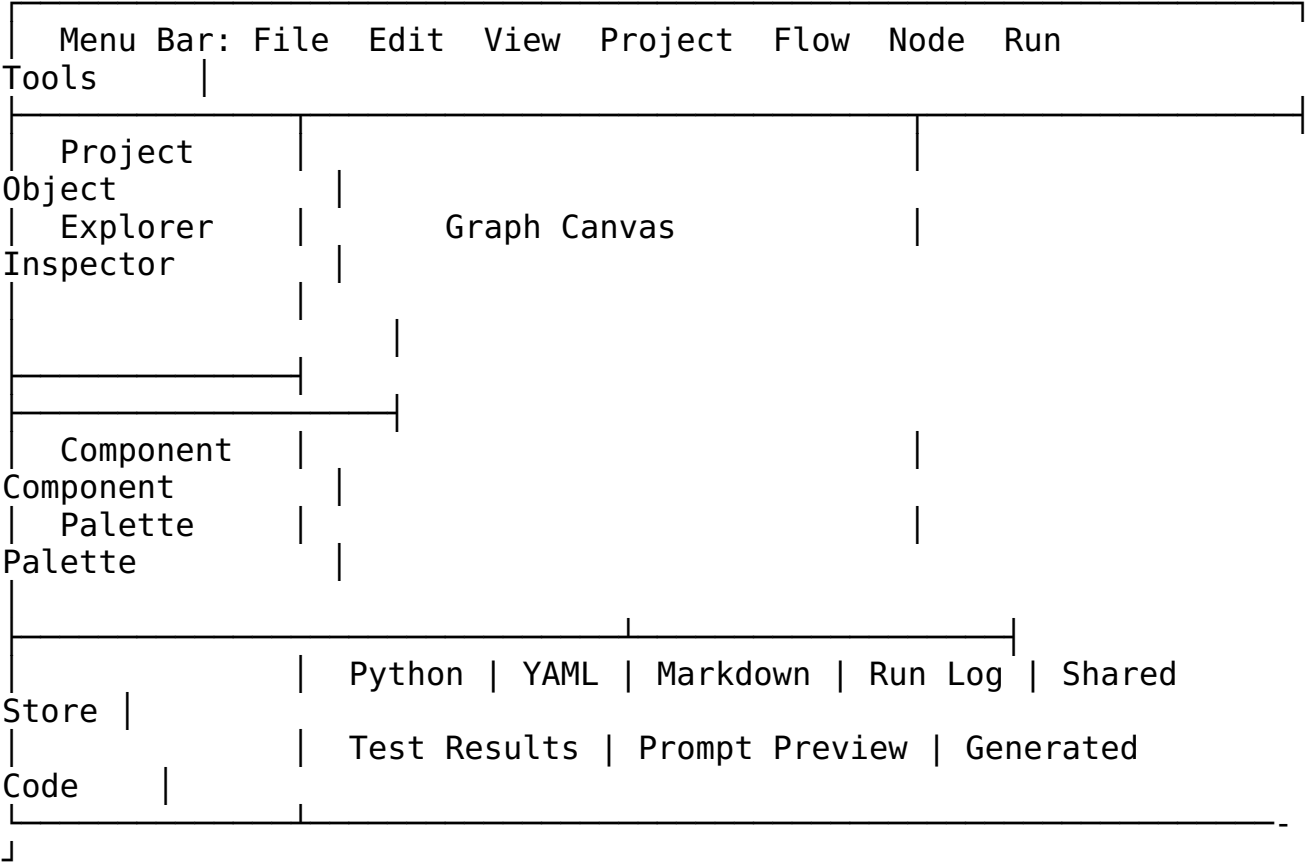
PocketFlow Creator main window
PocketFlow Creator main window

On first launch the application opens with no project loaded. The canvas is empty and most menu items are greyed out until a project is opened or created.

To create your first project: File > New Project...

To open an existing project: File > Open Project... and select a `.pfcproj.yaml` file.

The Six Panels



Panel	What it does
Project Explorer	Tree view of graphs, prompts, node types, and tools in the open project
Component Palette	Drag-and-drop source for built-in node types
Graph Canvas	Visual editor — place nodes, draw edges, arrange the flow
Object Inspector	Edit properties of the selected node or edge
Bottom tabs	Code editor (Python/YAML/Markdown), Run Log, Shared Store view, Test Results, Prompt Preview, Generated Code
Status bar	Current operation, validation errors, and YAML parse feedback

Quick-start Workflow

1. **File > New Project...** — name the project and choose a folder
2. Drag a **Start Node** from the Component Palette onto the canvas
3. Drag an **LLM Node** and connect its input port to the Start Node's default action port
4. Drag a **Stop Node** and connect the LLM Node's default action port to it
5. Select the LLM Node and fill in **Title**, **Prompt Type** (string for inline text or path for a file), and **Prompt File** in the Object Inspector
6. **Project > Validate Project** — fix any red badges
7. **Run > Run Active Flow** — watch the Run Log tab

For a detailed walkthrough see [Your First Flow](#).

Three fields to know in the Object Inspector:

- **Actions** — comma-separated strings that `post()` can return; each becomes a labelled output port. Every outgoing edge label must match a declared action.
- **Reads** — shared-store keys that `prep()` pulls in. Documents what this node depends on (not enforced at runtime, but used by the Data Flow Report).
- **Writes** — shared-store keys that `post()` pushes out. Documents what this node produces for downstream nodes.

These three fields are the node's data contract. Fill them in before writing code and the validator and Data Flow Report will catch mismatches for you.

Keyboard Shortcuts

Action	Shortcut
New project	Ctrl+N
Open project	Ctrl+O
Save	Ctrl+S
Undo / Redo	Ctrl+Z / Ctrl+Y
Run	F5
Debug	Shift+F5
Toggle breakpoint	F9
Auto Arrange...	Ctrl+Shift+L
Zoom in	Ctrl++
Zoom out	Ctrl+-

Action	Shortcut
Zoom to fit	Ctrl+0
Zoom to selected node	Ctrl+Shift+Z
Delete selected node/edge	Delete
Help	F1

Next Steps

- [Your First Flow](#) — step-by-step Hello World
 - [About PocketFlow](#) — the framework this tool targets
 - [Tutorials Part 1](#) — IDE deep-dive
-

Chapter 4: Your First Flow

Your First Flow

This guide walks you through building a complete Hello World flow — a single LLM node that answers a question — from an empty project to a running, exportable PocketFlow application.

Time: ~15 minutes

Prerequisites: Application installed and launched. See [Getting Started](#).

Step 1 — Create a New Project

1. Choose **File > New Project...**
2. Enter project name: HelloWorld
3. Choose a folder for the project files
4. Click **OK**

The Project Explorer now shows:

```
HelloWorld/  
  graphs/  
  prompts/  
  node_types/  
  tools/
```

Step 2 — Create a New Flow

1. Choose **Flow > New Flow...**
2. Name it `main`
3. Click **OK**

A blank canvas appears and `graphs/main.pfcgraph.yaml` appears in the Project Explorer.

Step 3 — Add a Start Node

1. In the **Component Palette** (left panel), find **Start Node**
2. Drag it onto the canvas

The Start Node appears as a rounded rectangle with a green port on its right edge. It is automatically set as the flow's start node.

Step 4 — Add an LLM Node

1. In the Component Palette, find **LLM Node**
 2. Drag it to the right of the Start Node
 3. In the **Object Inspector** (right panel), set:
 - **Title:** `Ask LLM`
 - **Prompt Type:** `path` (*select from the drop-down*)
 - **Prompt File:** `prompts/hello.md` (*relative path to the prompt file*)
-

Step 5 — Connect the Nodes

1. Hover over the Start Node until its default action port (right edge) turns blue
2. Click and drag from that port to the LLM Node's input port (left edge)
3. Release — an edge labelled **default** appears

What does “default” mean?

Every node's `post()` method returns a **string** called an **action**. PocketFlow uses that string to look up the matching outgoing edge and decide which node to run next. The edge you just drew is labelled "default" because the Start Node declares `default` as its only action — it always returns "default" from `post()`.

The **Actions** field in the Inspector is where you declare which strings `post()` can return. A linear flow only needs default. A branching flow (e.g. approve/reject) needs multiple actions — one outgoing edge per possible return value.

Step 6 — Add a Stop Node

1. Drag a **Stop Node** from the palette to the right of the LLM Node
2. Connect the LLM Node's default action port to the Stop Node's input port

Your canvas should now look like:

```
[Start] --default--> [Ask LLM] --default--> [Stop]
```

Step 7 — Write the Prompt

Since you set `prompt_type = path`, the node reads the prompt from a file at runtime.

1. In the Project Explorer, double-click `prompts/hello.md`
2. The **Markdown** editor tab opens
3. Type:

```
You are a helpful assistant.
```

```
Answer this question in one sentence:
```

```
What is PocketFlow?
```

1. Press **Ctrl+S** to save

Alternative: Set `prompt_type = string` and type the prompt text directly into the **Prompt File** field in the Inspector. This skips the file entirely — useful for short, one-off prompts that do not need version control.

Step 8 — Validate

1. Choose **Project > Validate Project** (or **Ctrl+Shift+V**)
2. The status bar should show: Validation passed — 0 errors

If you see red badges on nodes, click the node to read the error in the Object Inspector, fix the property, and validate again.

Step 9 — Run

1. Choose **Run > Run Active Flow** (or F5)
2. Switch to the **Run Log** tab at the bottom
3. You will see one entry per node executed, with the shared store state before and after

If you have Ollama running locally the LLM Node will call it. Otherwise configure a provider in **Tools > Provider Manager** or accept the Mock Provider output for testing.

Debug run in progress — Run Log showing step-by-step node execution and shared store output

Debug run in progress — Run Log showing step-by-step node execution and shared store output

Step 10 — Export

1. Choose **File > Export PocketFlow Project...**
2. PocketFlow Creator writes a standalone Python package to `exports/HelloWorld/`
3. Run it independently:

```
cd exports/HelloWorld
pip install -e .
python main.py
```

What the Shared Store, Reads, and Writes Mean

When you watched the Run Log in Step 9 you saw “shared store state before and after” for each node. That store is the **only channel** through which nodes pass data to each other — it is just a Python dict that every node can read from and write to.

- A node’s `prep()` method **reads** from the store (pulling in what it needs).
- A node’s `post()` method **writes** back into the store (leaving results for downstream nodes).

In the Object Inspector the **Reads** and **Writes** fields let you document which keys a node uses. They are not enforced at runtime, but they are the foundation of the **Data Flow Report** (Project > Data Flow Report), which shows the entire key lifecycle across every node in the graph.

The three Inspector fields to always fill in for each node:

Field	What it declares
Actions	Which strings <code>post ()</code> can return (controls edge routing)
Reads	Which shared-store keys <code>prep ()</code> pulls in (documents dependencies)
Writes	Which shared-store keys <code>post ()</code> pushes out (documents outputs)

What You Built

File	Purpose
<code>graphs/main.pfcgraph.yaml</code>	Graph structure — nodes, edges, properties
<code>prompts/hello.md</code>	LLM prompt file — used when <code>prompt_type = path</code> (never overwritten on re-export)
<code>exports/HelloWorld/generated/nodes.py</code>	Generated node class stubs
<code>exports/HelloWorld/generated/flow.py</code>	Generated PocketFlow wiring code
<code>exports/HelloWorld/custom/</code>	Your hand-edited node implementations

Next Steps

- [Tutorial 3: Using the Properties Inspector](#)
 - [Tutorial 4: The Code Editor — RAD Node Coding](#)
 - [Tutorial 7: Hello World — Single Node Q&A](#)
-

Part 2: Tutorials and Learning

Part 1 — Getting Started with PocketFlow Creator

Work through these tutorials in order. Each one builds on the previous, so the concepts introduced early become the vocabulary for everything that follows.

Tutorial 0: IDE Tour

What you'll learn: The purpose of each panel in PocketFlow Creator and how they work together as a unified design environment.

Prerequisites: App launched (python -m pocketflow_creator or pocketflow-creator).

Why a Visual Designer for LLM Workflows?

Writing LLM applications as plain Python is possible, but it forces you to hold the entire flow in your head: which nodes exist, how they connect, which actions each one can return, and which shared-store keys flow between them. Small flows are manageable; multi-node pipelines with loops, branches, and sub-graphs become hard to reason about quickly.

PocketFlow Creator externalises that mental model onto a canvas. You see the graph, you see the data contracts between nodes, and you can validate the whole structure before writing a single line of code. The generated Python is plain, readable PocketFlow code that runs without the designer installed — the canvas is a design tool, not a runtime wrapper.

The Six Panels

Menu Bar: File Edit View Project Flow Node Run					
Tools					
Project Explorer		Graph Canvas			
Inspector (left dock)		(drag nodes here, wire edges between ports)			
Component Palette (left dock)		(right dock)			
Component Palette sources)		(drag			
(left dock)					

Store Code		Python	Markdown	YAML	Run Log	Shared
		Test Results		Prompt Preview		Generated

Project Explorer (left dock, top) Shows every file in your project as a tree: graphs, prompt files, node type definitions, and tool scripts. Double-clicking a graph opens it on the canvas. Double-clicking a prompt file opens it in the Markdown editor. Think of this as the file browser for your LLM application.

Component Palette (left dock, bottom) The source of new nodes. Every built-in node type lives here, along with any snippets you have configured. Drag a node type from the palette onto the canvas to create an instance of it. The palette also shows custom node types once you have defined them — they appear alongside the built-ins.

Graph Canvas (centre) The main design surface. This is where you place nodes, draw edges between them, and see the shape of your flow at a glance. Nodes are the work units; edges are the labelled transitions between them. The canvas is interactive: you can drag nodes to reposition them, click edges to inspect or relabel them, and zoom with Ctrl+Scroll.

Each node shows its data contract directly on its tile:

- **Left port (input)** — labelled with the shared-store key the node reads (its `input_key` property). Nodes without an explicit input key show `in`.
- **Right ports (outputs)** — one circle per action, each labelled with the action name (e.g. `default`, `pass`, `fail`). Multi-action nodes grow taller to accommodate all their ports; each port is wired to a separate outgoing edge.

Reading these labels lets you trace the data flow across the graph without opening the Inspector or the code editor.

Object Inspector (right dock, top) Shows and edits the properties of whatever is selected on the canvas. Selecting a node shows its ID, type, title, actions, and any custom properties. Selecting an edge shows its action label. Changes made here are reflected on the canvas immediately — the inspector and the canvas are always in sync.

Component Palette (right dock, bottom) A secondary palette for quick access when the left dock is collapsed.

Bottom Tabs The tabbed panel at the bottom switches between several views:

Tab	Purpose
Python	Live code editor for the current graph's node implementations
Markdown	Editor + live preview for prompt files
YAML	Editor for metadata and schema files, with parse-error feedback
Run Log	Step-by-step output from the last run
Shared Store	Live JSON view of the shared store during and after a run
Prompt Preview	Shows the rendered prompt for the selected LLM node
Generated Code	Read-only view of the exported <code>flow.py</code>
Test Results	Output from <code>pytest</code> when you run the test suite
Problems	Validation errors from the most recent validate operation
Data Flow	Plain-text report showing which shared-store keys each node reads and writes, execution order, and any routing warnings — generated via Project > Data Flow Report

How the Panels Work Together

The typical design loop in Creator is:

1. Drag nodes from the **Palette** onto the **Canvas**
2. Wire edges between nodes (output port → input port)
3. Click a node to inspect and edit its properties in the **Inspector**
4. Double-click a node to jump to its implementation in the **Python** tab
5. Press F5 to run, then read the **Run Log** and **Shared Store** tabs
6. Fix issues flagged in the **Problems** tab

Each panel has a focused job, and they all stay in sync. There is no save-then-refresh cycle — what you see is always current.

Quick Reference: Keyboard Shortcuts

Action	Shortcut
New project	Ctrl+N
Open project	Ctrl+O

Action	Shortcut
Save	Ctrl+S
Save all	Ctrl+Shift+S
Export project	Ctrl+E
Validate	Ctrl+Shift+V
Run active flow	F5
Debug active flow	Shift+F5
Toggle breakpoint	F9
Undo	Ctrl+Z
Redo	Ctrl+Y
Zoom in	Ctrl++
Zoom out	Ctrl+-
Zoom to fit	Ctrl+0
Zoom to selected node	Ctrl+Shift+Z
Auto Arrange...	Ctrl+Shift+L
Delete selected node	Delete

Tutorial 1: Hello LLM

What you'll learn: Make your first real LLM call from a PocketFlow node, understand the three-method node lifecycle (prep / exec / post), produce structured JSON output, and configure four different LLM providers.

Prerequisites: None. You do not need an open project to understand this tutorial.

Understanding the Node Lifecycle

Every PocketFlow node has exactly three methods, and they have distinct responsibilities. Understanding why they are separate is more important than memorising the signatures.

prep(self, shared) — *Read from the world, prepare inputs.* This method has access to shared, the dictionary that flows through the entire graph. Its job is to extract whatever the node needs and return it as prep_res. Keeping reads in prep keeps exec pure: exec receives only what it needs and nothing more.

exec(self, prep_res) — *Do the work.* This is where the LLM call, tool call, file read, or calculation happens. exec receives only what prep handed it — it never touches shared directly. This separation is intentional: it makes exec easy to test in isolation and easy to swap out without touching the data-flow logic.

post(self, shared, prep_res, exec_res) — *Write results back, choose the next step.* This method receives both what prep prepared and what exec produced. It writes results into shared and returns a string — the **action** — that tells the graph which edge to follow next. Returning "default" follows the default edge.

This three-phase design means every node has a clear contract: read → work → write/route.

Part A — Plain Text Response

The simplest possible flow sends a prompt to an LLM and prints the response.

Graph

```
[Start] --default--> [HelloLlm] --default--> [Stop]
```

Node code

```
class HelloLlm(Node):
    def prep(self, shared: dict) -> str:
        # We read nothing from shared here – the prompt is
        hardcoded.
        # In a real flow, you might read shared["user_input"]
        instead.
        return 'Reply with exactly this text and nothing else: LLM says: Hello User'

    def exec(self, prep_res: str) -> str:
        # call_llm is the only thing that changes when you switch
        providers.
        # The rest of the node is provider-agnostic.
        return call_llm(prep_res)

    def post(self, shared: dict, prep_res: str, exec_res: str) -> str:
        # Store the result so downstream nodes can use it.
        shared["message"] = exec_res
        print(exec_res) # → LLM says: Hello User
        return "default" # follow the "default" edge to the
        next node
```

Why this matters: Even this trivial example demonstrates the full lifecycle. prep prepares the prompt, exec calls the LLM, post stores the result and routes the flow. In a larger application you would replace the hardcoded prompt with

something read from shared, and the print with a downstream node that formats and delivers the output.

Expected output:

LLM says: Hello User

Part B — Structured JSON Response

Plain text output is fine for printing, but most real applications need typed, structured data that downstream nodes can work with programmatically. The solution is to instruct the LLM to respond with JSON and parse it in exec.

Why parse in exec rather than post? Because exec is the work step — if the parse fails (malformed JSON from a non-deterministic model), you catch the error at the work boundary, not after you have already tried to write to shared. You can also add retry logic around exec without touching prep or post.

```
import json

class HelloLlm(Node):
    def prep(self, shared: dict) -> str:
        # The prompt explicitly demands JSON and specifies the exact structure.
        # Being precise here greatly reduces malformed responses.
        return (
            'Respond with ONLY valid JSON – no markdown, no extra text.\n'
            'Use exactly this structure: {"message": "LLM says: Hello User"}'
        )

    def exec(self, prep_res: str) -> dict:
        raw = call_llm(prep_res)

        # Some models wrap responses in markdown code fences even when asked not to.
        # Strip them defensively rather than assuming the model will always comply.
        raw = raw.strip().removeprefix("```json").removeprefix("```").removesuffix("```")
        return json.loads(raw) # raises ValueError on bad JSON – handle or retry here

    def post(self, shared: dict, prep_res: str, exec_res: dict) -> str:
        # exec_res is now a dict, not a string.
```

```
shared["result"] = exec_res
print(exec_res["message"])    # → LLM says: Hello User
return "default"
```

Expected output:

LLM says: Hello User

The value at `shared["result"]` is a proper Python dict:

```
{"message": "LLM says: Hello User"}
```

Downstream nodes can access `shared["result"]["message"]` directly, pass it to a formatter, validate it against a schema, or route on its fields.

Part C — Configuring an LLM Provider

The `call_llm` function is deliberately left as a placeholder in PocketFlow examples. Swap in whichever implementation matches your provider. All four options below produce the same interface — a function that takes a string and returns a string — so your node code never changes when you switch providers.

Option 1 — Ollama (local, no API key)

[Ollama](#) runs open-source models on your own machine. It is the best starting point: free, private, and works entirely offline once the model is downloaded.

When to use: Development and prototyping, privacy-sensitive applications, no-cost experimentation with open-source models.

Install and pull a model:

```
# Install Ollama from https://ollama.com, then:
ollama pull llama3.2          # ~2 GB – fast, good for most tasks
ollama pull mistral           # alternative if you prefer Mistral
```

Provider code:

```
import json
import urllib.request

def call_llm(prompt: str, model: str = "llama3.2") -> str:
    """Send a prompt to Ollama running on localhost and return
    the response."""
    url = "http://localhost:11434/api/generate"
    payload = json.dumps({
        "model": model,
```

```

        "prompt": prompt,
        "stream": False,           # get the full response at once,
not a stream
    }).encode()
    req = urllib.request.Request(
        url, data=payload,
        headers={"Content-Type": "application/json"},
    )
    with urllib.request.urlopen(req, timeout=60) as resp:
        return json.loads(resp.read())["response"].strip()

```

In PocketFlow Creator: Tools > Provider Manager → set Provider to Ollama, Base URL to <http://localhost:11434>, Model to llama3.2.

Option 2 — OpenAI API

OpenAI's hosted models are widely used and have excellent instruction-following. gpt-4o-mini is fast and inexpensive; gpt-4o is more capable for complex reasoning.

When to use: Production applications where response quality and reliability matter; when you need function calling or vision capabilities.

```

pip install openai

from openai import OpenAI

_client = OpenAI(api_key="sk-...")  # or export OPENAI_API_KEY
in your shell

def call_llm(prompt: str, model: str = "gpt-4o-mini") -> str:
    response = _client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content.strip()

```

In PocketFlow Creator: Tools > Provider Manager → Provider: OpenAI, API Key: your key, Model: gpt-4o-mini.

Option 3 — Anthropic Claude API

Claude models are particularly strong at following precise instructions and producing well-structured output — both important properties when asking for JSON.

When to use: Tasks requiring nuanced reasoning, long context windows, or especially reliable instruction-following.

```
pip install anthropic

import anthropic

_client = anthropic.Anthropic(api_key="sk-ant-...") # or
ANTHROPIC_API_KEY env var

def call_llm(prompt: str, model: str = "claude-
haiku-4-5-20251001") -> str:
    message = _client.messages.create(
        model=model,
        max_tokens=1024,
        messages=[{"role": "user", "content": prompt}],
    )
    return message.content[0].text.strip()
```

Model guide: claude-haiku-4-5-20251001 is fast and cheap; claude-sonnet-4-6 balances quality and speed; claude-opus-4-7 is most capable.

In PocketFlow Creator: Tools > Provider Manager → Provider: Claude, API Key: your Anthropic key, Model: claude-haiku-4-5-20251001.

Option 4 — Google Gemini API

Gemini has a generous free tier on gemini-1.5-flash, making it a good option for experimentation without a credit card.

When to use: Free-tier prototyping, multimodal tasks (when combined with image input), Google Cloud integration.

```
pip install google-generativeai

import google.generativeai as genai

genai.configure(api_key="AIza...") # or GOOGLE_API_KEY env var
_model = genai.GenerativeModel("gemini-1.5-flash")

def call_llm(prompt: str) -> str:
    response = _model.generate_content(prompt)
    return response.text.strip()
```

In PocketFlow Creator: Tools > Provider Manager → Provider: Gemini, API Key: your Google AI Studio key, Model: gemini-1.5-flash.

Summary

Step	Key concept
Part A	prep / exec / post lifecycle; returning an action string
Part B	Parse structured output in exec; downstream nodes get typed data
Part C	<code>call_llm</code> is the only thing that changes between providers

The most important insight from this tutorial: **your node logic is decoupled from the provider**. A flow written today against Ollama runs tomorrow against Claude by changing one function. PocketFlow's architecture enforces this separation by design.

Tutorial 2: Your First Flow — Hello World

What you'll learn: Create a project, place nodes on the canvas, wire them with action edges, set properties in the inspector, and run the flow.

Prerequisites: Tutorial 0 — you know what each panel does.

What Is a Flow, Exactly?

A PocketFlow flow is a **directed graph**: a set of nodes connected by directed edges. Each node does a unit of work and then returns an action string (like "default" or "error"). The edge labelled with that action string determines which node runs next.

This is fundamentally different from a plain Python function chain. There is no hardcoded call sequence in the framework code — only a routing table that says “if node A returns 'ok', go to node B; if it returns 'error', go to node C.” This makes flows easy to visualise, test, and modify without touching unrelated code.

Every flow needs at least three things:

- A **Start Node** — the unique entry point the runner looks for
- At least one **processing node** — where the real work happens
- A **Stop Node** — a terminal point (you can have several, one per exit path)

Steps

1. Create a project

File > New Project... opens the project creation dialog. Give it the name `hello_world` and choose a folder on disk.

Why a project? A project is a folder that holds everything together: graph files, prompt files, node type definitions, generated code, and test files. Creator uses the project structure to know where to look for each kind of file. Without a project, the code editor and the export pipeline have nowhere to write.

2. Open the graph

In Project Explorer, double-click `graphs/main.pfcgraph.yaml`. The canvas appears empty.

Why YAML? Graph files are plain YAML — human-readable, diffable in git, and editable in a text editor if needed. You are never locked into the visual format.

3. Add a Start Node

Drag **Start Node** from the Component Palette onto the canvas. Click it to select it, then in the Object Inspector change **Title** to **Start**.

Why rename? The title is what you see on the canvas. Meaningful titles make large flows readable at a glance. The ID (auto-generated, shown in the inspector but not editable) is what the code uses internally.

4. Add an LLM Prompt Node

Drag **LLM Prompt Node** onto the canvas, to the right of Start. Set **Title** to **Ask LLM**.

In the Inspector you will see two prompt-related properties:

- **prompt_type** — a drop-down with two choices:
 - `string` (*default*) — type the prompt directly into the `prompt_file` field
 - `path` — enter a relative file path; the file is read at run time
- **prompt_file** — the prompt text or file path, depending on `prompt_type`

For a quick first run, leave `prompt_type` as `string` and type your prompt directly into `prompt_file` (e.g. Say hello to the world in one sentence.). Switch to `path` when you want to manage longer prompts as reusable Markdown files.

5. Add a Stop Node

Drag **Stop Node** to the right of Ask LLM. Set **Title** to **End**.

Why a Stop Node? The runner stops when it reaches a node with no outgoing edges, or explicitly when it reaches a Stop Node. Having an explicit Stop Node makes the exit point visible in the graph and prevents accidental infinite loops.

6. Wire the edges

Click the **output port** (right side circle) of Start and drag to the **input port** (left side circle) of Ask LLM. A dialog or inspector field will ask for the action label — type `default`. Repeat: wire Ask LLM → End with action `default`.

Why label edges? PocketFlow routes by comparing the string returned by `post()` to the edge labels. A node that returns `"default"` will follow any edge labelled `"default"`. If a node has two edges — `"ok"` and `"error"` — it can route to different destinations based on what happened in `exec`. Labelling everything `"default"` is fine for linear flows; the labels matter when you start branching.

7. Create the prompt file

Click the **Markdown** tab at the bottom. Type:

Say hello to the world in one sentence.

Save with `Ctrl+S` to `prompts/hello.md`.

Why Markdown for prompts? Prompts are text that LLMs read. Markdown lets you structure longer prompts with headers and bullet points without any special syntax. The live preview on the right side of the Markdown tab lets you see the rendered version as you type.

8. Validate the graph

Project > Validate Project (`Ctrl+Shift+V`). Check the **Problems** tab.

Why validate before running? The validator catches structural errors — missing start node, dangling edges, undeclared actions — that would cause the runner to fail at runtime. It is faster to catch these in the designer than to read a Python traceback.

9. Run with the mock provider

Run > Run Active Flow (`F5`). Watch the **Run Log** tab.

What is the mock provider? When no real LLM provider is configured, Creator uses a mock that echoes the prompt back. This lets you test the flow structure and routing logic without an API key or internet connection.

Expected result: Three nodes on canvas, no red error badges, Run Log shows three steps — Start, Ask LLM, End — each with a status of completed.

Tutorial 3: Using the Properties Inspector

What you'll learn: Every field in the Object Inspector, how live sync works, and why getting properties right before writing code saves time.

Prerequisites: Tutorial 2 — a project with nodes on the canvas.

The Inspector as Your Node's Contract

The Inspector is not just a label editor. Every field you fill in here becomes part of the node's public contract: what actions it can return, which shared-store keys it reads and writes, and what its configuration parameters are. When you define these clearly in the Inspector before writing code, you have a spec to code against. Other nodes in the graph can see what you declared, and the validator can check that your edges are consistent with your actions.

The Three Most Important Fields: Actions, Reads, Writes

These three fields are worth understanding deeply because they appear on every node and control both how the flow routes and how data moves between nodes.

Actions — controlling which path the flow takes

After a node finishes its work, its `post()` method returns a **string**. PocketFlow looks up the outgoing edge whose label matches that string and follows it to the next node. The edge label is the **action**.

```
[NodeA]  -- "approved" -->  [HandleApproved]
         -- "rejected" -->  [HandleRejected]
```

`NodeA.post()` returns "approved" → graph follows the "approved" edge

The **Actions** field in the Inspector is the comma-separated list of all strings `post()` might return. Declaring them before drawing edges creates a labelled output port on the canvas for each one — one port per possible route.

The validator enforces that every outgoing edge label matches a declared action. This means the graph structure *cannot* become inconsistent with the code routing without the validator telling you immediately.

Rule of thumb: Declare actions first, then draw edges from the named ports.

Reads — documenting what the node pulls from the shared store

The **shared store** is the dict (shared) that every node can see. It is the only channel through which nodes pass data to each other. A node's `prep()` method reads the inputs it needs:

```
def prep(self, shared: dict) -> str:
    # Pull the user's question from the store before calling the
    LLM.
    return shared["user_input"]
```

The **Reads** field documents which keys `prep()` consumes — for example, `user_input`.

Important: Reads is documentation, not enforcement. You could read `shared["anything"]` in `prep()` without declaring it. But if you *do* declare it, the Data Flow Report can verify that some upstream node actually writes that key. An undeclared read is a silent assumption; a declared read is a checkable contract.

Writes — documenting what the node pushes into the shared store

After doing its work, `post()` stores results for downstream nodes:

```
def post(self, shared: dict, prep_res, exec_res: str) -> str:
    shared["llm_response"] = exec_res    # downstream nodes can
    now read "llm_response"
    return "default"
```

The **Writes** field documents which keys `post()` produces — for example, `llm_response`. Combined with Reads declarations across all nodes, the Data Flow Report can show the complete lifecycle of every key: which node writes it first, which nodes read it, and whether any node reads a key that nobody writes.

How they map to the three-method lifecycle

Inspector field	Node method	What happens
Reads store depends on)	<code>prep(shared)</code>	Pull inputs from the shared (documents what this node
access pure computation)	<code>exec(prepare_res)</code>	Do the work – no shared store (no Inspector field; this is

Writes store produces)	post(shared)	Push outputs into the shared (documents what this node
Actions follow	return value of post()	Choose which outgoing edge to (validated against edge labels)

This three-phase design means every node has a single, clear responsibility at each step: *read* → *work* → *write/route*. The Inspector fields make that responsibility explicit and visible on the canvas.

Filling them in: a practical rule

Before writing any code for a new node, fill in: 1. **Actions** — what `post()` will return (determines how many output ports appear) 2. **Reads** — what shared-store keys `prep()` will need 3. **Writes** — what shared-store keys `post()` will produce

You now have a spec. The validator enforces Actions. The Data Flow Report checks Reads and Writes. You can hand this spec to a teammate or come back to it six months later and understand the node without reading any Python.

Node Properties

Click any node on the canvas to see its properties:

Field	Editable	Purpose
ID	No	Unique internal identifier — used in <code>NODE_START</code> markers and graph YAML
Type	No	The palette type this instance was created from
Title	Yes	The name shown on the canvas tile
Position X/Y	No	Canvas coordinates — drag the node to reposition
Actions	Yes	Comma-separated list of action strings this node's <code>post()</code> can return
Reads	Yes	Shared-store keys this node reads in <code>prep()</code> (documented contract)
Writes	Yes	

Field	Editable	Purpose
		Shared-store keys this node writes in <code>post()</code> (documented contract)

Live Sync in Practice

1. Select the **Ask LLM** node you created in Tutorial 2
2. Change **Actions** from `default` to `success, failure`
3. Watch the canvas — the edge you wired with "default" will now show a validation error badge because "default" is no longer a declared action on this node

This is live sync: the inspector change immediately propagates to the validator. You do not have to save and reload — the designer maintains a consistent in-memory model and the canvas reflects it continuously.

1. Change Actions back to `default` to clear the error
2. Change **Reads** to `user_input` and **Writes** to `llm_response`

These documentation fields appear in exported code comments and in the shared-store schema helper, but do not affect runtime behaviour.

Edge Properties

Click an edge (the connecting line) rather than a node. The Inspector shows:

- **Action** — the label on this edge. This must match one of the source node's declared Actions, otherwise the validator flags error PFCE2101. Clicking the action field lets you rename the edge in place.

Node Port Labels

The canvas reflects the Inspector values without you needing to read any code:

- **Input label** (lower-left of the node tile) — displays the value of the node's `input_key` property, or `in` if none is set. This tells you at a glance which shared-store key the node expects to find when it runs.
- **Output labels** (right side, one per action) — each action port is labelled with its action string. Multi-action nodes grow taller so every port has its own clearly labelled row. Edges drawn from a specific port carry that action automatically.

If you change `input_key` in the Inspector, the canvas label updates immediately.

The Data Flow Report

Once your graph has more than a few nodes, understanding which key is written where and read by whom becomes non-trivial. The **Data Flow Report** answers that question in one click:

Project > Data Flow Report — or look at the **Data Flow** tab in the bottom panel.

The report contains three sections:

NODE DATA FLOW — a table listing every node in BFS execution order, with columns for the shared-store keys it reads and the keys it writes. Multi-action nodes also show their branch destinations.

SHARED STORE KEY LIFECYCLE — for every key that appears in the graph, one row showing which node writes it and which nodes read it. A key marked (external) in the “Written by” column must be supplied by the caller before the flow starts.

DATA FLOW NOTES — routing and key warnings: - A key that is written but never read (likely dead output) - A key that is read but never written (must come from outside the flow) - A key written by more than one node (later write silently overwrites the earlier one) - An edge whose action doesn’t match any action the source node declares

Run the report whenever you add new nodes, change `input_key/output_key` properties, or add a new routing branch. It is the fastest way to catch data-contract mismatches before running the flow.

When to Use the Inspector vs. the Code Editor

Use the Inspector for everything structural: titles, actions, documented reads/writes, and any properties defined by the node type (like `prompt_type`, `prompt_file`, or `top_k`). Use the code editor for the actual implementation of `prep`, `exec`, and `post`. The split keeps the graph self-describing — you can understand a flow’s structure without reading any code.

Tutorial 4: The Code Editor — RAD Node Coding

What you’ll learn: How double-clicking a node opens and scaffolds its Python class, what the `NODE_START` and `NODE_END` markers do, and how changes in the code file sync back to the canvas.

Prerequisites: Tutorial 2 — an open project with nodes on the canvas.

The Code File and the Graph File are Two Views of the Same Thing

Every graph (`graphs/main.pfcgraph.yaml`) has a companion code file (`code/main.py`). The graph file describes the structure — which nodes exist, how they are connected. The code file contains the implementation — what each node actually does. Creator keeps these two files in sync automatically:

- When you **add a node** to the canvas (drag from palette or click a toolbar button), Creator adds a class stub to the code file immediately
- When you **double-click a node**, Creator opens the code file and scrolls to that node's class
- When you **delete a node** from the canvas, Creator removes its class from the code file
- When you **delete a node's class** from the code file and save, Creator removes the node from the canvas

This bidirectional sync means you never have orphaned code and you never have nodes with missing implementations.

The NODE_START / NODE_END Markers

Creator wraps each node class in a pair of marker comments:

```
# --- NODE_START: node_abc12345 ---
class AskLLM(Node):
    """llm_prompt_node: Ask LLM"""

    def prep(self, shared: dict) -> object:
        return None

    def exec(self, prep_res: object) -> object:
        return None

    def post(self, shared: dict, prep_res: object, exec_res:
object) -> str:
        return "default"

# --- NODE_END: node_abc12345 ---
```

The markers use the node's unique ID (not the title) so they survive title changes. **Do not remove the markers.** They are how Creator locates your class when syncing. Everything between them is yours to edit freely — the markers are just bookends.

Steps

1. Open a node's code

Double-click the **Ask LLM** node on the canvas. The Python tab opens and scrolls to the AskLlm class. If the code file does not exist yet, Creator creates it with the file header and a stub for this node.

Why double-click rather than navigating to the file manually? Because the code file may contain many node classes. Double-clicking is a direct jump — you land exactly at the class you want to edit, regardless of how many other classes are in the file.

2. Read the stub

The generated stub shows all three methods with placeholder return values. The docstring tells you the type_id and title. The type signature is intentionally loose (object) so you can annotate it with real types as you implement each method.

3. Implement the node

Replace the stub bodies with real logic:

```
# --- NODE_START: node_abc12345 ---
class AskLlm(Node):
    """llm_prompt_node: Ask LLM"""

    def prep(self, shared: dict) -> str:
        # Resolve the prompt based on prompt_type set in the
Inspector.
        # "string" → use the text directly; "path" → read from a
file.

        prompt_type = shared.get("prompt_type", "string")
        prompt_value = shared.get("prompt_file", "")
        if prompt_type == "path" and prompt_value:
            with open(prompt_value, encoding="utf-8") as f:
                return f.read()
        return prompt_value or "What is PocketFlow?"

    def exec(self, prep_res: str) -> str:
        # call_llm is your provider helper from Tutorial 1.
        return call_llm(prep_res)

    def post(self, shared: dict, prep_res: str, exec_res: str) -
> str:
        # Store the response for downstream nodes.
        shared["llm_response"] = exec_res
        return "default"

# --- NODE_END: node_abc12345 ---
```

4. Save

Ctrl+S writes the file to disk. The canvas does not need to be refreshed — the graph model and the code file are separate concerns that stay in sync through the bidirectional markers, not through re-parsing.

5. Delete a node from the canvas

Click a node and press Delete. The node disappears from the canvas and its class (the entire `NODE_START...NODE_END` block) is removed from the code file automatically. This prevents the code file from accumulating dead classes as your graph evolves.

6. Delete a class from the code file

Open the Python tab, manually delete the `NODE_START...NODE_END` block for a node, and save (Ctrl+S). Creator detects that the marker is gone and removes the corresponding node from the canvas. This is the reverse direction of the sync.

When to Write Code Here vs. Exporting

The embedded editor is for active development. It gives you immediate feedback via the Run Log and Shared Store tabs. When you are done and want to ship, use File > Export to get a clean standalone Python package that runs without Creator.

Tutorial 5: Creating a Custom Node Type

What you'll learn: When to create a custom node type (versus using a Basic Node), how the Node Type Wizard works, and how custom types appear in the palette.

Prerequisites: Tutorial 2.

When to Create a Custom Node Type

A **Basic Node** is the right choice for one-off logic that is specific to a single flow. But when you find yourself building the same kind of node repeatedly — with the same structure, the same properties, and the same action pattern — that is a signal to create a custom node type.

Custom node types give you: - A named entry in the Component Palette so you can drag and drop them like built-ins - Typed properties with defaults that appear in the Inspector for every instance - A YAML definition that can be shared across projects and checked into version control - A Python skeleton generated automatically, ready for your implementation

The rule of thumb: if you would build this node more than twice, define it as a type.

Steps

1. Open the Node Type Wizard

Node > New Custom Node Type... opens the wizard.

2. Fill in the identity fields

- **Node Type ID:** `sentiment_node` — this is the internal identifier, used in graph files and code markers. Use lowercase with underscores. Once set, changing this breaks existing graphs that use the type.
- **Display Name:** `Sentiment Analyser` — what appears in the palette and on the canvas tile
- **Category:** `Analysis` — groups related types in the palette

Why separate ID and display name? The ID is a stable technical identifier; the display name can change freely without breaking anything.

3. Set the base class

Leave **Base Class** as `Node` for a standard single-execution node. Choose `BatchNode` if this type will process a list of items, or `AsyncNode` if it performs async I/O. The base class is what PocketFlow uses to call the right execution strategy.

4. Declare actions

In the **Actions** field, enter: `positive, negative, neutral`

These are the strings your `post()` method can return. The wizard records them in the YAML definition so that when someone adds an instance of this type to a canvas, the validator knows which actions are valid. Every outgoing edge must match one of these.

5. Add a property

Click **Add Property**: - Name: `threshold` - Type: `number` - Default: `0.5`

Properties defined here appear in the Object Inspector for every instance of this type. A developer can configure the threshold per-instance without touching the code — the inspector writes the value into the graph YAML, and your prep can read it from `shared` or from the node's `properties` dict.

6. Click Create

The wizard writes two files: - `node_types/sentiment_node.yaml` — the full type definition, including properties schema and action declarations - `custom/sentiment_node.py` — a Python skeleton with the correct base class

7. Implement the skeleton

Open `custom/sentiment_node.py` from Project Explorer and fill in the logic:

```
class SentimentNode(Node):
    def prep(self, shared: dict) -> str:
        # Read the text to classify from the shared store.
        return shared.get("text", "")

    def exec(self, prep_res: str) -> str:
        # Simple keyword-based classifier for demonstration.
        # Replace with a real model call in production.
        text = prep_res.lower()
        if any(w in text for w in ["great", "love", "excellent",
"happy"]):
            return "positive"
        if any(w in text for w in ["bad", "hate", "terrible",
"awful"]):
            return "negative"
        return "neutral"

    def post(self, shared: dict, prep_res: str, exec_res: str) -
> str:
        shared["sentiment"] = exec_res
        # Return the sentiment label as the action – the graph
routes on it.
        return exec_res
```

8. Use the new type

Sentiment Analyser now appears in the Component Palette under “Analysis”. Drag it onto any canvas and wire three outgoing edges: `positive`, `negative`, `neutral`. The validator will flag an error if any of the three declared actions is unwired, reminding you to handle every exit path.

Tutorial 6: Project Templates

What you’ll learn: How to start a new project from a built-in template, what files a template creates, and when templates save meaningful time.

Prerequisites: None.

Why Templates Matter

Starting from a blank canvas is fine for learning, but real workflows have a predictable structure: a graph file, one or more prompt files, a code file, and a project manifest. Templates pre-build that structure so you can jump straight to the interesting part — designing the flow — without repeating boilerplate setup.

Templates also enforce consistency. On a team, everyone starting from the same template means every project has the same folder layout, the same naming conventions, and the same initial graph structure. That makes onboarding and code review much easier.

Steps

1. Open the template picker

File > New From Template... shows the available templates. Select **Simple LLM Flow**.

2. Configure and create

Choose a folder and enter a project name. Click **Create**. Creator generates:

```
my_project/
├── project.pfcproj.yaml      ← project manifest (name,
    provider, model settings)
├── graphs/
│   └── main.pfcgraph.yaml   ← Start → LLM Prompt → Stop, pre-
wired
├── prompts/
│   └── main.md              ← placeholder prompt file
├── code/
│   └── (generated on first node open)
└── node_types/
    └── (empty – add custom types here)
```

3. Explore the pre-wired graph

Open `graphs/main.pfcgraph.yaml`. You see three nodes already placed and connected. Use View > Zoom to Fit (Ctrl+0) to centre the view. The template has done the structural work; your job is to replace the placeholder prompt and implement the node bodies.

4. Edit the prompt

Double-click `prompts/main.md` in Project Explorer. The Markdown editor opens. Replace the placeholder with an instruction relevant to what you are building. Save.

5. Run

F5 — the mock provider echoes the prompt. Switch to a real provider in Tools > Provider Manager and run again to see a genuine LLM response.

What to Do When No Template Fits

If none of the templates matches your use case, start from the **Blank Project** template (File > New Project...). Build the first version manually, then use File > Export to get the raw file structure. You can use that exported structure as the basis for your own template for future projects.

[→ Continue to Part 2 — PocketFlow Patterns](#)

Part 2 — PocketFlow Patterns in Creator

Every AI workflow, no matter how complex, is built from a small set of recurring patterns. This section maps each pattern to a Creator project you can build and run. Understanding these patterns — not just their code — is what lets you design systems confidently.

Work through these tutorials in any order after completing Part 1. Each tutorial begins with a conceptual explanation of the pattern and why it exists, then walks through every step with a reason for it.

[← Tutorials Index](#)

Tutorial 7: Hello World — Single Node Q&A

What you'll learn: The simplest possible flow: one question, one answer, and why even this trivial case benefits from the prep/exec/post structure.

Why this pattern exists

Before building multi-node pipelines, it helps to see the full lifecycle in its simplest form. A single-node Q&A flow has no branching, no looping, and no shared state complexity — it's the clearest possible illustration of how PocketFlow moves data through a node.

The `prep` → `exec` → `post` split may seem like overhead when all three fit in ten lines. It pays off the moment you want to retry `exec` without re-reading the database, or mock `exec` in a test without touching real storage. The discipline starts here.

The pattern

```
[Start] --default--> [AskLLM] --default--> [Stop]
```

A single active node reads from shared state, calls the LLM, and writes the answer back. Start and Stop are structural markers, not code.

Step-by-step

Step 1: Create the project.

Open Creator and choose File > New Project. Name it `tut_hello_world`. Creator creates the folder structure (`graphs/`, `code/`, `prompts/`) and opens an empty canvas. Starting with a fresh project — not a template — forces you to make every decision consciously.

Step 2: Add Start and Stop nodes.

Drag **Start Node** onto the canvas. Drag **Stop Node** onto the canvas. These nodes hold no code; they are entry and exit markers that the validator uses to confirm your flow has a clear beginning and end. PocketFlow requires exactly one node declared as the start; the Start Node sets this automatically.

Step 3: Add an LLM Prompt Node.

Drag **LLM Prompt Node** onto the canvas. In the Inspector, set its Title to `Ask LLM`. The LLM Prompt Node is a Basic Node with a predefined code template — it already contains `prep`, `exec`, and `post` stubs wired for calling an LLM. You will fill in the logic.

Set the prompt properties in the Inspector:

- **prompt_type** — choose `string` to type the prompt directly, or `path` to load it from a Markdown file at run time
- **prompt_file** — the prompt text (if `string`) or relative file path (if `path`)

For this tutorial, leave `prompt_type` as `string` and enter your question directly in `prompt_file` — no file needed.

Step 4: Wire the edges.

Draw an edge from **Start** → **Ask LLM**, typing `default` as the action. Draw another from **Ask LLM** → **Stop**, also `default`. Every edge in PocketFlow is a named action — even when there is only one path, the action must be explicit. This explicitness becomes critical in routing flows where multiple outgoing edges exist.

Step 5: Arrange and Zoom to Fit.

Drag nodes into a left-to-right arrangement that matches the flow, then use **View > Zoom to Fit** (Ctrl+0) to centre the canvas. For automatic layout, use **View > Auto Arrange...** (Ctrl+Shift+L) — choose an algorithm (Layered BFS, Grid, or Force-directed), a connector style, and spacing, then click OK. The arrangement is undoable with Ctrl+Z.

Step 6: Write the node code.

Double-click **Ask LLM** to open it in the code editor. You will see `# ---` `NODE_START` and `# ---` `NODE_END` markers. Everything you write goes between those markers. Fill in:

```
class AskLlm(Node):
    def prep(self, shared):
        # Read the question from shared state, or use a default.
        # prep's job is to gather inputs. Keep it free of side
effects.
        return shared.get("question", "What is PocketFlow?")

    def exec(self, prep_res):
        # exec receives exactly what prep returned.
        # It does the work – here, the LLM call.
        # exec is retried on failure, so it must be idempotent.
        return call_llm(prep_res)

    def post(self, shared, prep_res, exec_res):
        # post writes results back to shared state and chooses
the next action.
        # It runs exactly once, even if exec was retried.
        shared["answer"] = exec_res
        print("Answer:", exec_res)
        return "default"
```

Notice the role separation: `prep` reads, `exec` works, `post` writes. The LLM call lives in `exec` because that is the unit the framework knows how to retry. If the API times out, PocketFlow retries `exec` without re-running `prep` or re-writing `post`.

Step 7: Run the flow.

Run > Run Active Flow. The Run Log tab shows each node as it executes. The Shared Store tab shows the `answer` key appear after **Ask LLM** completes. If you

see an error, read the full traceback in the Run Log — it includes the node name and method where the failure occurred.

Expected result: The Shared Store shows answer: "PocketFlow is a minimal LLM orchestration framework..." (or whatever your LLM returns).

Tutorial 8: Chat with History

What you'll learn: How to maintain conversational context across multiple turns by accumulating a history list in the shared store, and how to model a loop in a Creator graph.

Why this pattern exists

A single-turn Q&A forgets everything between calls. A chatbot must remember what was said. PocketFlow's shared store is the natural place to keep history — it persists across every node execution for the life of the flow. The trick is designing the graph to loop: after printing a reply, the flow returns to the input node and waits for the next message.

Modeling a loop in a directed graph requires at least one node with two outgoing actions: one to continue the loop and one to exit it. The `GetInput` node plays this role.

The pattern

```
[Start] → [GetInput] —continue—→ [CallLLM] → [PrintReply] →  
[GetInput]  
[GetInput] —exit————→ [Stop]
```

Step-by-step

Step 1: Create the project.

New project: `tut_chat`. This flow will use five nodes — slightly more complex than Tutorial 7.

Step 2: Add the nodes.

Add in order: Start Node, two Basic Nodes (titles: `Get Input`, `Print Reply`), one LLM Prompt Node (title: `Call LLM`), Stop Node. The order you add them does not matter; wiring defines the execution order.

Step 3: Configure actions on Get Input.

Select **Get Input** in the Inspector. Set Actions to `continue`, `exit`. A node's declared Actions list is what the validator checks against outgoing edges — if you wire an edge with action `exit` but the node doesn't declare it, the validator flags an error. Declare every action the node's post can return.

Step 4: Wire the graph.

- Start → Get Input: `default`
- Get Input → Call LLM: `continue`
- Get Input → Stop: `exit`
- Call LLM → Print Reply: `default`
- Print Reply → Get Input: `default` ← this is the loop-back edge

The loop-back is what makes this a conversation rather than a one-shot flow. Without it, the flow would end after one turn.

Step 5: Write Get Input's code.

```
class GetInput(Node):
    def prep(self, shared):
        # Nothing to read from shared – this node reads from the user.

        return None

    def exec(self, prep_res):
        # input() blocks until the user types. In a UI, this would be replaced
        # with an async await on a message queue.
        return input("You: ").strip()

    def post(self, shared, prep_res, exec_res):
        if exec_res.lower() in ("quit", "exit", "bye"):
            return "exit"
        # setdefault initialises the key on first turn without overwriting it.
        shared.setdefault("history", [])
        shared["history"].append({"role": "user", "content": exec_res})

        return "continue"
```

`setdefault` is the right tool here: it initialises the list on the first turn without clearing it on subsequent turns. Using `shared["history"] = []` would reset the history every turn — a subtle bug that wouldn't show up until the second message.

Step 6: Write Call LLM's code.

```

class CallLlm(Node):
    def prep(self, shared):
        # Pass the full history so the LLM has context for each
reply.
        return shared.get("history", [])

    def exec(self, prep_res):
        return call_llm_with_history(prep_res)

    def post(self, shared, prep_res, exec_res):
# Append the assistant's reply so future turns include it as
context.
        shared["history"].append({"role": "assistant",
"content": exec_res})
        shared["last_reply"] = exec_res
        return "default"

```

Both user and assistant messages go into history. That full list is what you pass to the LLM on the next turn. Without the assistant messages, the LLM cannot refer to what it said.

Step 7: Write Print Reply's code.

```

class PrintReply(Node):
    def prep(self, shared):
        return shared.get("last_reply", "")

    def exec(self, prep_res):
        print(f"\nAssistant: {prep_res}\n")
        return None

    def post(self, shared, prep_res, exec_res):
        return "default"

```

Step 8: Debug the loop.

Run > Debug Active Flow. Set a breakpoint on **Call LLM** (F9). Each time the flow pauses there, check the Shared Store tab — watch history grow with each turn. This is the most reliable way to verify your history accumulation logic is correct.

Expected result: A working multi-turn conversation where the LLM remembers prior messages until you type “quit”, “exit”, or “bye”.

Tutorial 9: Structured Output — Resume Data Extraction

What you'll learn: How to instruct an LLM to return JSON, how to parse and validate that JSON in exec, and when structured output is the right pattern to use.

Why this pattern exists

LLMs naturally produce prose. When downstream code needs to read specific fields — a name, a list of skills, a number — prose is the wrong format. Structured output means prompting the LLM to return JSON, then parsing it in exec before handing the result to post.

The discipline here is important: parsing happens in exec, not post. That way, if the LLM returns malformed JSON, exec raises an exception and the framework can retry the call rather than writing bad data to the shared store.

The pattern

```
[Start] → [LoadResume] → [ExtractFields] → [ValidateOutput] → [Stop]
```

Step-by-step

Step 1: Create the project.

New project: tut_structured_output.

Step 2: Add and wire the nodes.

Add Start, three Basic Nodes (titles: Load Resume, Extract Fields, Validate Output), Stop. Wire sequentially with default actions.

Step 3: Set Inspector metadata for Extract Fields.

Select **Extract Fields**. In Inspector set Reads: resume_text and Writes: extracted_data. This metadata does not affect execution but documents the data contract — future readers of the graph can immediately see what this node consumes and produces. Think of it as the function signature for the node.

Step 4: Write Load Resume's code.

```
class LoadResume(Node):  
    def prep(self, shared):  
        return None
```

```

    def exec(self, prep_res):
        # In production, read from a file or API. Here we use a static string.
        return """Jane Smith, jane@example.com
10 years of experience in software engineering.
Skills: Python, SQL, Machine Learning, Docker,
Kubernetes"""

    def post(self, shared, prep_res, exec_res):
        shared["resume_text"] = exec_res
        return "default"

```

Step 5: Write Extract Fields's code.

```

import json

class ExtractFields(Node):
    def prep(self, shared):
        return shared["resume_text"]

    def exec(self, prep_res):
        prompt = f"""Extract from this resume:
- name (string)
- email (string)
- years_of_experience (integer)
- skills (list of strings)

Resume:
{prep_res}

Return ONLY valid JSON. No explanation, no markdown fences.
Example:
{{"name": "...", "email": "...", "years_of_experience": 5,
"skills": ["Python"]}}"""

        raw = call_llm(prompt)
        # Strip markdown fences if the LLM adds them despite the instruction.
        if raw.strip().startswith("```"):
            raw =
raw.strip().removeprefix("```json").removeprefix("```").removesuffix("```")
            # json.loads raises ValueError on malformed JSON – exec retries on exception.
            return json.loads(raw)

    def post(self, shared, prep_res, exec_res):
        shared["extracted_data"] = exec_res
        return "default"

```

The fence-stripping code handles a common LLM behaviour: wrapping JSON in markdown code blocks despite being told not to. Do this defensively in `exec` rather than hoping the LLM follows instructions perfectly every time.

Step 6: Write Validate Output's code.

```
class ValidateOutput(Node):
    REQUIRED = {"name", "email", "years_of_experience", "skills"}

    def prep(self, shared):
        return shared.get("extracted_data", {})

    def exec(self, prep_res):
        missing = self.REQUIRED - set(prep_res.keys())
        if missing:
            raise ValueError(f"Missing fields: {missing}")
        return prep_res

    def post(self, shared, prep_res, exec_res):
        print("Extracted:", exec_res)
        return "default"
```

Validation in a separate node (not inside Extract Fields) makes the pipeline easy to extend: swap the extractor, keep the validator, and the contract is enforced regardless.

Step 7: Use the Shared Store Designer.

Tools > Shared Store Designer. Add rows: `resume_text` (string), `extracted_data` (object). This documents the schema so team members reading the project understand what the store holds.

Expected result: The Shared Store shows `extracted_data` as a populated JSON object with the four required fields.

Tutorial 10: Multi-Stage Workflow — Article Writer

What you'll learn: How to chain multiple LLM calls through distinct stages — outline, draft, polish — and why separating stages produces better output than a single giant prompt.

Why this pattern exists

A single prompt asking for “a complete article” gives the LLM no room to reason step by step. Breaking the task into stages lets the LLM focus on one goal at a time: first produce structure, then expand it, then refine it. Each stage can have its own

prompt, its own retry budget, and its own output stored separately in the shared store for inspection.

This pattern also makes debugging practical. If the final article is poor, you can inspect the outline and draft separately to identify where quality degraded.

The pattern

```
[Start] → [GenerateOutline] → [WriteDraft] → [ApplyStyle] → [Stop]
```

Step-by-step

Step 1: Create the project.

New project: `tut_workflow`.

Step 2: Add nodes and wire them.

Add Start, three LLM Prompt Nodes (titles: Generate Outline, Write Draft, Apply Style), Stop. Wire sequentially with default actions.

Step 3: Create prompt files.

In Project Explorer, right-click the `prompts/` folder and create three files: `outline.md`, `draft.md`, `style.md`. Storing prompts as files (not inline strings) means you can edit them without touching node code — and they show up in version control diffs as readable text changes, not code changes.

`prompts/outline.md`:

```
Create a 5-point outline for an article about: {topic}
Return only the outline as a numbered list. No introduction, no
conclusion header yet.
```

`prompts/draft.md`:

```
Expand this outline into a complete draft article:
```

```
{outline}
```

```
Write in a clear, engaging style. Include an introduction and
conclusion.
```

`prompts/style.md`:

```
Polish this article for publication. Make it concise, vivid, and
professional.
Remove redundancy. Keep all factual content.
```

```
{draft}
```

Step 4: Write Generate Outline's code.

```
class GenerateOutline(Node):
    def prep(self, shared):
        topic = shared.get("topic", "the future of AI")
        template = open("prompts/outline.md").read()
        return template.format(topic=topic)

    def exec(self, prep_res):
        return call_llm(prep_res)

    def post(self, shared, prep_res, exec_res):
        shared["outline"] = exec_res
        return "default"
```

Step 5: Write Write Draft's code.

```
class WriteDraft(Node):
    def prep(self, shared):
        template = open("prompts/draft.md").read()
        return template.format(outline=shared["outline"])

    def exec(self, prep_res):
        return call_llm(prep_res)

    def post(self, shared, prep_res, exec_res):
        shared["draft"] = exec_res
        return "default"
```

Step 6: Write Apply Style's code.

```
class ApplyStyle(Node):
    def prep(self, shared):
        template = open("prompts/style.md").read()
        return template.format(draft=shared["draft"])

    def exec(self, prep_res):
        return call_llm(prep_res)

    def post(self, shared, prep_res, exec_res):
        shared["final_article"] = exec_res
        print(exec_res)
        return "default"
```

Step 7: Set the topic and run.

In the Shared Store Designer, set `topic` with a default value (e.g., "the future of AI"). Run > Run Active Flow. After completion, inspect `outline`, `draft`, and `final_article` separately in the Shared Store tab to see how each stage built on the previous.

Expected result: Three intermediate keys in the shared store plus a polished final article that is noticeably better than what a single-prompt approach produces.

Tutorial 11: Conditional Routing — Chat Guardrail

What you'll learn: How to use a Router Node to branch the flow based on an LLM classification, and when guardrails belong in the graph structure versus in application code.

Why this pattern exists

Guardrails — topic filters, safety checks, input validators — are naturally expressed as branches in the flow graph, not as `if` statements buried inside a node. Putting the branch in the graph makes it visible, testable, and replaceable without touching unrelated code.

A Router Node is a node whose `post` method returns one of several named actions. Each action corresponds to an outgoing edge leading to a different path. The graph structure itself documents the branching logic.

The pattern

```
[Start] → [ClassifyInput] —on_topic→ [AnswerQuestion] → [Stop]
                        ↳off_topic→ [Redirect]      →
[Stop]
```

Step-by-step

Step 1: Create the project.

New project: `tut_guardrail`.

Step 2: Add the Router Node.

Drag **Router Node** onto the canvas. Set its Title to `Classify Input`. In Inspector, set Actions to `on_topic`, `off_topic`. The Router Node is a visual hint that this node controls branching — functionally it is a Basic Node, but its icon communicates intent.

Step 3: Add answer and redirect nodes.

Add two Basic Nodes: Answer Question and Redirect. Add two Stop Nodes (one per path). Wire: Start → Classify Input (default), Classify Input → Answer Question (on_topic), Classify Input → Redirect (off_topic), Answer Question → Stop (default), Redirect → Stop (default).

Step 4: Write Classify Input's code.

```
class ClassifyInput(Node):
    def prep(self, shared):
        return shared.get("user_question", "")

    def exec(self, prep_res):
        if not prep_res.strip():
            return "off_topic"
        prompt = (
            "\n"
            "You are a guardrail for a travel assistance chatbot.
            "Is the following question about travel,
            destinations, flights, hotels, or itineraries?\n"
            "Answer YES or NO only.\n\n"
            f"Question: {prep_res}"
        )
        answer = call_llm(prompt).strip().upper()
        return "on_topic" if answer.startswith("YES") else
"off_topic"

    def post(self, shared, prep_res, exec_res):
        shared["route"] = exec_res
        return exec_res # return the action string directly
```

exec returns the action string, and post returns it unchanged. This is a valid PocketFlow pattern: when exec directly computes the routing decision, post simply propagates it. The shared["route"] = exec_res line stores the decision for debugging.

Step 5: Write Answer Question and Redirect.

```
class AnswerQuestion(Node):
    def prep(self, shared):
        return shared.get("user_question", "")

    def exec(self, prep_res):
        return call_llm(f"Answer this travel question helpfully:
{prep_res}")

    def post(self, shared, prep_res, exec_res):
        shared["answer"] = exec_res
        print("Answer:", exec_res)
```

```

        return "default"

class Redirect(Node):
    def prep(self, shared):
        return None

    def exec(self, prep_res):
        return None

    def post(self, shared, prep_res, exec_res):
        print("I can only help with travel questions. Please ask
about destinations, flights, or hotels.")
        return "default"

```

Step 6: Test both paths.

Set `user_question` in the Shared Store Designer. Run once with a travel question ("Best beaches in Thailand") and once with an off-topic question ("Explain quantum physics"). Verify the Shared Store shows route: `on_topic` and route: `off_topic` respectively. Check the Run Log to confirm the correct path executed.

Expected result: Travel questions produce an answer; off-topic questions produce a redirect message. The graph visually documents this branching logic.

Tutorial 12: Agent with Tools

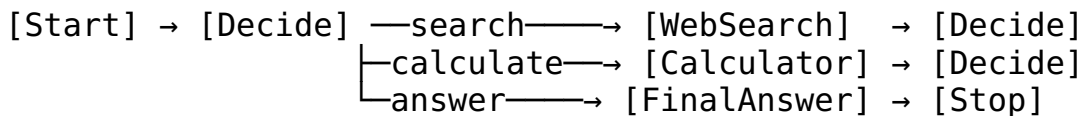
What you'll learn: How an agent decides which tool to call, executes it, and loops back to decide again — and why this decision loop is the right model for open-ended tasks.

Why this pattern exists

A workflow runs a fixed sequence of steps. An agent runs a variable sequence determined at runtime by the LLM's reasoning. The agent pattern — decide, act, observe, decide again — is how you build systems that can handle tasks where the number of steps is unknown in advance.

The key design choice: the Router Node decides, separate tool nodes act. Keeping decision and action apart makes each testable in isolation and makes it easy to add new tools without touching the decision logic.

The pattern



The loop-back edges (WebSearch → Decide, Calculator → Decide) are what make this an agent rather than a workflow. The agent decides, acts, then decides again with new information.

Step-by-step

Step 1: Create the project.

New project: tut_agent.

Step 2: Add the Router Node.

Add Router Node, title Decide. Set Actions: search, calculate, answer. This node is the heart of the agent — every iteration of the loop passes through it.

Step 3: Add tool nodes and final answer node.

Add Basic Nodes: Web Search, Calculator, Final Answer. Add Stop Node.

Step 4: Wire the graph.

- Start → Decide: default
- Decide → Web Search: search
- Decide → Calculator: calculate
- Decide → Final Answer: answer
- Web Search → Decide: default ← loop-back
- Calculator → Decide: default ← loop-back
- Final Answer → Stop: default

Step 5: Write Decide's code.

```
class Decide(Node):
    TOOLS = ["search", "calculate", "answer"]

    def prep(self, shared):
        return shared.get("question", ""),
shared.get("tool_history", [])

    def exec(self, prep_res):
        question, history = prep_res
        history_text = "\n".join(f"- {h}" for h in history[-6:])
        prompt = (
```

```

        f"Question: {question}\n"
        f"Steps taken so far:\n{history_text or 'none'}\n\n"
        "What should I do next? Choose EXACTLY one: search,
calculate, or answer.\n"
        "Respond with just the single word."
    )
    action = call_llm(prompt).strip().lower()
    return action if action in self.TOOLS else "answer"

    def post(self, shared, prep_res, exec_res):
        shared.setdefault("tool_history", []).append(f"decided:
{exec_res}")
        return exec_res

```

The [-6:] slice limits history to the last six steps. Without this, the prompt grows indefinitely and eventually exceeds the LLM's context window.

Step 6: Write the tool nodes.

```

class WebSearch(Node):
    def prep(self, shared):
        return shared.get("question", "")

    def exec(self, prep_res):
        # In production, call a real search API here.
        result = f"[Search result for: {prep_res}] The answer is
approximately 42."
        return result

    def post(self, shared, prep_res, exec_res):
        shared.setdefault("tool_history", []).append(f"search
result: {exec_res[:100]}")
        shared["last_search"] = exec_res
        return "default"

class Calculator(Node):
    def prep(self, shared):
        return shared.get("question", "")

    def exec(self, prep_res):
        # In production, parse and evaluate a mathematical
expression.
        return f"[Calculator result for: {prep_res}] = 1764"

    def post(self, shared, prep_res, exec_res):
        shared.setdefault("tool_history",
 []).append(f"calculated: {exec_res}")
        shared["last_calculation"] = exec_res

```

```

        return "default"

class FinalAnswer(Node):
    def prep(self, shared):
        history = shared.get("tool_history", [])
        question = shared.get("question", "")
        return question, history

    def exec(self, prep_res):
        question, history = prep_res
        context = "\n".join(history)
        return call_llm(
            f"Question: {question}\nResearch done:\n{context}"
            "\n\nProvide a final answer:"
        )

    def post(self, shared, prep_res, exec_res):
        shared["answer"] = exec_res
        print("Final answer:", exec_res)
        return "default"

```

Step 7: Set a question and run in debug mode.

In the Shared Store Designer set question to "What is 42 squared?". Run in Debug Mode (Shift+F5) with a breakpoint on Decide. Watch each iteration: the agent decides to calculate, acts, then decides to answer. The tool history grows with each step.

Expected result: The agent loop terminates when Decide returns answer. The Shared Store shows the full tool_history and a final answer.

Tutorial 13: Retrieval-Augmented Generation (RAG)

What you'll learn: The complete RAG pipeline — chunking, embedding, indexing, retrieval, and generation — and when RAG is the right approach versus fine-tuning or longer prompts.

Why this pattern exists

LLMs have a knowledge cutoff and a limited context window. When a question requires information the LLM wasn't trained on (your company's documents, recent events, private data), you can't simply ask the LLM — you must retrieve the relevant passages first and include them in the prompt.

RAG splits this into four stages: 1. **Index time:** Chunk documents, embed each chunk, store vectors. 2. **Query time:** Embed the question, find the closest chunks, generate an answer.

PocketFlow represents these as separate nodes, which means you can optimise each independently — swap the embedding model, change the chunk size, use a different retrieval strategy — without redesigning the whole pipeline.

The pattern

```
[Start] → [LoadDocs] → [EmbedChunks] → [StoreIndex] →  
[GetQuestion]  
      → [EmbedQuery] → [RetrieveChunks] → [GenerateAnswer] →  
[Stop]
```

Step-by-step

Step 1: Create the project.

New project: tut_rag.

Step 2: Add and wire nodes in two phases.

Phase 1 (indexing): Start → Load Docs → Embed Chunks → Store Index → Get Question. Phase 2 (query): Get Question → Embed Query → Retrieve Chunks → Generate Answer → Stop.

In a real system these phases might run independently. For this tutorial they run in sequence so you can inspect intermediate state in one flow.

Step 3: Document the data contracts.

Select each node and set Reads/Writes in Inspector. This makes the shared store schema explicit before writing any code:

- Load Docs: Writes raw_docs
- Embed Chunks: Reads raw_docs, Writes chunks, embeddings
- Get Question: Writes question
- Embed Query: Reads question, Writes query_vector
- Retrieve Chunks: Reads query_vector, embeddings, chunks, Writes context
- Generate Answer: Reads context, question, Writes answer

Step 4: Write the index-phase nodes.

```
class LoadDocs(Node):  
    def prep(self, shared):  
        return None
```

```

def exec(self, prep_res):
    # In production, read from files, a database, or an API.
    return (
        "PocketFlow is a minimal 100-line LLM orchestration
framework. "
        "It uses a shared store to pass data between nodes. "
        "Nodes implement prep, exec, and post methods. "
        "The framework supports sync, async, batch, and
parallel processing. "
        "PocketFlow is designed for clarity and
composability."
    )

def post(self, shared, prep_res, exec_res):
    shared["raw_docs"] = exec_res
    return "default"

class EmbedChunks(Node):
    CHUNK_SIZE = 100 # characters per chunk

    def prep(self, shared):
        text = shared["raw_docs"]
        return [text[i:i+self.CHUNK_SIZE] for i in range(0,
len(text), self.CHUNK_SIZE)]

    def exec(self, prep_res):
# In production, call a real embedding model (e.g. OpenAI text-
embedding-3-small).
        # Here we use a deterministic mock: hash-based floats.
        import hashlib
        def mock_embed(text):
            h = hashlib.md5(text.encode()).digest()
            return [b / 255.0 for b in h]
        return [(chunk, mock_embed(chunk)) for chunk in prep_res]

    def post(self, shared, prep_res, exec_res):
        shared["chunks"] = [c for c, _ in exec_res]
        shared["embeddings"] = [e for _, e in exec_res]
        return "default"

```

Step 5: Write the query-phase nodes.

```

class GetQuestion(Node):
    def prep(self, shared):
        return None

```

```

def exec(self, prep_res):
    return input("Question: ").strip()

def post(self, shared, prep_res, exec_res):
    shared["question"] = exec_res
    return "default"

class EmbedQuery(Node):
    def prep(self, shared):
        return shared["question"]

    def exec(self, prep_res):
        import hashlib
        h = hashlib.md5(prepare_res.encode()).digest()
        return [b / 255.0 for b in h]

    def post(self, shared, prep_res, exec_res):
        shared["query_vector"] = exec_res
        return "default"

class RetrieveChunks(Node):
    TOP_K = 3

    def prep(self, shared):
        return shared["query_vector"], shared.get("embeddings",
[]), shared.get("chunks", [])

    def exec(self, prep_res):
        q_vec, emb_list, chunks = prep_res
        def dot(a, b):
            return sum(x * y for x, y in zip(a, b))
        scored = [(dot(q_vec, e), c) for e, c in zip(emb_list,
chunks)]

        top = sorted(scored, reverse=True)[:self.TOP_K]
        return [chunk for _, chunk in top]

    def post(self, shared, prep_res, exec_res):
        shared["context"] = exec_res
        return "default"

class GenerateAnswer(Node):
    def prep(self, shared):
        context = "\n".join(shared.get("context", []))
        question = shared.get("question", "")
        return f"Context:\n{context}\n\nQuestion: {question}

\n\nAnswer:"

    def exec(self, prep_res):

```



```

        return call_llm(prepare_res)

    def post(self, shared, prepare_res, exec_res):
        shared["answer"] = exec_res
        print("Answer:", exec_res)
        return "default"

```

Step 6: Run and inspect.

Run > Run Active Flow. Type a question about PocketFlow when prompted. Inspect the Shared Store: chunks, embeddings, query_vector, context (the retrieved passages), and answer are all visible. This transparency — seeing every intermediate value — is one of the key advantages of the node-per-stage approach.

Expected result: The LLM answers based on the retrieved context, not its training data.

Tutorial 14: Map-Reduce / Batch Processing

What you'll learn: How BatchNode automatically iterates over a list without a manual loop, and when to use batch processing versus building an explicit loop in the graph.

Why this pattern exists

When you need to apply the same operation to every item in a list — summarise each document, classify each image, rate each resume — you could build an explicit loop in the graph. But that adds four extra nodes and wiring for something the framework can handle automatically.

BatchNode is the answer: its prep returns a list, and the framework calls exec once per item. Your exec code is written for a single item, not a list. post receives all results as a list and writes them to the shared store in one place.

The pattern

```

[Start] → [LoadItems] → [ProcessEach (BatchNode)] → [Aggregate] → [Stop]

```

Step-by-step

Step 1: Create the project.

New project: tut_map_reduce.

Step 2: Add a Batch Node.

Drag **Batch Node** from the palette. Title it **Process Each**. In Inspector, note that Base Class shows **BatchNode**. The palette distinguishes **BatchNode** visually to signal that this node has special iteration semantics.

Step 3: Write Load Items's code.

```
class LoadItems(Node):
    def prep(self, shared):
        return None

    def exec(self, prep_res):
        return [
            "Alice Johnson – 5 years Python, 3 years ML. Led team of 4.",
            "Bob Smith – 2 years JavaScript, 1 year React. Junior developer.",
            "Carol White – 12 years Java, 8 years architecture. Principal engineer.",
        ]

    def post(self, shared, prep_res, exec_res):
        shared["items"] = exec_res
        return "default"
```

Step 4: Write Process Each's code.

```
class ProcessEach(BatchNode):
    def prep(self, shared):
        # Return the list. BatchNode calls exec once per element.
        return shared["items"]

    def exec(self, item):
        # item is a single resume string, not the full list.
        # Write exec as if you're handling one item – the framework handles iteration.
        return call_llm(f"Rate this resume 1-10 and explain why: \n{item}")

    def post(self, shared, prep_res, exec_res):
        # exec_res is a list: one result per item, in the same order as prep returned.
        shared["ratings"] = exec_res
        return "default"
```

The mental model: **prep** hands the framework a shopping list. The framework checks off each item by calling **exec**. **post** receives the completed list.

Step 5: Write Aggregate's code.

```
class Aggregate(Node):
    def prep(self, shared):
        items = shared.get("items", [])
        ratings = shared.get("ratings", [])
        return list(zip(items[:30], ratings)) # first 30 chars
of each resume

    def exec(self, prep_res):
        lines = [f"Candidate {i+1}: {rating}" for i, (_, rating)
in enumerate(prep_res)]
        return "\n".join(lines)

    def post(self, shared, prep_res, exec_res):
        shared["summary"] = exec_res
        print(exec_res)
        return "default"
```

Step 6: Run and inspect.

Run > Run Active Flow. The Shared Store shows ratings as a three-element list — one rating per resume. The batch ran three LLM calls automatically without any loop logic in the graph or the code.

When to use BatchNode vs. explicit loops: - Use BatchNode when every item gets the same treatment. - Use an explicit loop in the graph (a node that increments an index and loops back) when different items need different paths through the graph.

Expected result: ratings contains three independent LLM evaluations; summary formats them.

Tutorial 15: Human-in-the-Loop

What you'll learn: How to pause a flow for human review, accept approval or rejection, and loop back for revision — and why this is structurally different from a simple prompt.

Why this pattern exists

Fully automated LLM pipelines are efficient but fallible. When the cost of a mistake is high (publishing content, sending an email, making a purchase), a human review gate is worth the latency. Human-in-the-loop is not a workaround for a bad LLM — it is a deliberate architectural choice that keeps a human in the critical path.

In PocketFlow, the review node is a regular node that reads from standard input. In production this becomes an async node waiting on a message queue or a web form submission. The graph structure is identical either way — only the implementation of exec changes.

The pattern

```
[Start] → [DraftContent] → [Review] —approved→ [Publish] → [Stop]
                                     ↳rejected→ [Revise] → [Review]
```

The loop from Revise back to Review is what makes this iterative rather than binary.

Step-by-step

Step 1: Create the project.

New project: tut_hitl.

Step 2: Add nodes.

Add: Start, Basic Node (Draft Content), Router Node (Review, Actions: approved, rejected), Basic Node (Publish), Basic Node (Revise), Stop.

Step 3: Wire the approval loop.

- Start → Draft Content: default
- Draft Content → Review: default
- Review → Publish: approved
- Review → Revise: rejected
- Revise → Review: default ← the loop-back
- Publish → Stop: default

Step 4: Write Draft Content's code.

```
class DraftContent(Node):
    def prep(self, shared):
        topic = shared.get("topic", "benefits of morning exercise")
        feedback = shared.get("feedback", "")
        if feedback:
            return f"Topic: {topic}\nPrevious feedback to incorporate: {feedback}"
            return f"Topic: {topic}"

    def exec(self, prep_res):
```

```

        return call_llm(f"Write a short 3-paragraph blog post
about: {prep_res}")

```

```

def post(self, shared, prep_res, exec_res):
    shared["draft"] = exec_res
    return "default"

```

When feedback exists (from a prior rejection), it is incorporated into the new prompt. This is how the revision loop improves on each iteration.

Step 5: Write Review's code.

```

class ReviewDraft(Node):
    def prep(self, shared):
        return shared.get("draft", "")

    def exec(self, prep_res):
        print("\n" + "="*60)
        print("DRAFT FOR REVIEW:")
        print("="*60)
        print(prep_res)
        print("="*60)
        decision = input("\nApprove? (yes/no): ").strip().lower()
        if decision in ("yes", "y"):
            return "approved"
        feedback = input("What should be improved? ").strip()
        return ("rejected", feedback)

    def post(self, shared, prep_res, exec_res):
        if isinstance(exec_res, tuple):
            action, feedback = exec_res
            shared["feedback"] = feedback
            return action
        return exec_res

```

Step 6: Write Publish and Revise.

```

class Publish(Node):
    def prep(self, shared):
        return shared.get("draft", "")

    def exec(self, prep_res):
        with open("output/published_post.md", "w") as f:
            f.write(prep_res)
        return "Published."

    def post(self, shared, prep_res, exec_res):
        print(exec_res)
        return "default"

```

```

class Revise(Node):
    def prep(self, shared):
        return shared.get("draft", ""), shared.get("feedback",
        "")

    def exec(self, prep_res):
        draft, feedback = prep_res
        return call_llm(f"Revise this draft based on the
        feedback.\nFeedback: {feedback}\n\nDraft:\n{draft}")

    def post(self, shared, prep_res, exec_res):
        shared["draft"] = exec_res
        return "default"

```

Step 7: Run in debug mode.

Run > Debug Active Flow. Set a breakpoint on **Review** so you can inspect the Shared Store before each review decision. After rejecting, watch feedback appear in the store. After approving, confirm the file was written.

Expected result: The flow loops until you approve, then writes the approved draft to a file.

Tutorial 16: LLM-as-Judge / Evaluator Loop

What you'll learn: How to use a second LLM call to evaluate the output of a first, loop for refinement, and use a maximum-iteration guard to ensure the flow always terminates.

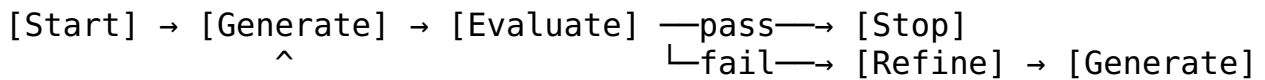
Why this pattern exists

An LLM's first attempt at a creative or analytical task is often good but not great. Rather than accepting first-pass output or manually reviewing it, you can automate the evaluation: one LLM generates, a second (the “judge”) evaluates and provides feedback, and the first refines based on that feedback. This is the evaluator loop.

The judge does not have to be a different model — it is a different call with a different prompt, focused on evaluation rather than generation. The separation of roles (generate vs. evaluate) improves quality because each call is optimised for its specific task.

The loop must always terminate. The maximum-iteration guard is not optional: without it, a stubborn judge and a poor generator could loop forever.

The pattern



Step-by-step

Step 1: Create the project.

New project: tut_judge.

Step 2: Add and wire nodes.

Add Start, Basic Node (Generate), Router Node (Evaluate, Actions: pass, fail), Basic Node (Refine), Stop.

Wire: Start $\rightarrow$ Generate $\rightarrow$ Evaluate, Evaluate $\rightarrow$ Stop (pass), Evaluate $\rightarrow$ Refine (fail), Refine $\rightarrow$ Generate (default).

Step 3: Write Generate's code.

```
class Generate(Node):
    def prep(self, shared):
        task = shared.get("task", "Write a haiku about software engineering.")

        feedback = shared.get("judge_feedback", "")
        if feedback:
            current = shared.get("output", "")
            return f"Task: {task}\n\nCurrent attempt:\n{current}\n\nFeedback: {feedback}\n\nRevise:"
        return task

    def exec(self, prep_res):
        return call_llm(prep_res)

    def post(self, shared, prep_res, exec_res):
        shared["output"] = exec_res
        shared["judge_feedback"] = "" # Clear feedback after incorporating it.
        return "default"
```

Step 4: Write Evaluate's code.

```
class Evaluate(Node):
    MAX_ITERATIONS = 3

    def prep(self, shared):
        return shared.get("output", ""),
shared.get("judge iteration", 0)
```

```

def exec(self, prep_res):
    output, iteration = prep_res
    # The guard: force pass after MAX_ITERATIONS to ensure
    termination.
    if iteration >= self.MAX_ITERATIONS:
        return "pass"
    criteria = shared.get("criteria", "The output should be
    creative, precise, and complete.")
    prompt = (
        f"Evaluate this output.\n"
        f"Criteria: {criteria}\n"
        f"Output:\n{output}\n\n"
        "Respond with PASS if the output meets the criteria,
    or FAIL followed by "
        "one sentence of specific feedback on what to
    improve."
    )
    verdict = call_llm(prompt).strip()
    if verdict.upper().startswith("PASS"):
        return "pass"
    return ("fail", verdict)

def post(self, shared, prep_res, exec_res):
    output, iteration = prep_res
    shared["judge_iteration"] = iteration + 1
    if isinstance(exec_res, tuple):
        action, feedback = exec_res
        shared["judge_feedback"] = feedback
        return action
    return exec_res

```

Step 5: Write Refine's code.

Refine is handled by Generate reading judge_feedback on its next call (see Step 3 above). Refine can be a simple passthrough that just signals the loop should continue:

```

class Refine(Node):
    def prep(self, shared):
        return None

    def exec(self, prep_res):
        return None

    def post(self, shared, prep_res, exec_res):
        return "default" # Routes back to Generate.

```


Step 6: Set criteria and run.

In the Shared Store Designer, set: - task: "Write a haiku about software engineering." - criteria: "Must be exactly 3 lines, 5-7-5 syllable structure, and relate to coding."

Run > Debug Active Flow. Set a breakpoint on **Evaluate**. Watch `judge_iteration` increment with each loop. After `MAX_ITERATIONS`, the judge forces pass regardless of quality.

Expected result: The flow runs 1–4 times (depending on quality) and produces a final output. `judge_iteration` in the Shared Store shows how many rounds it took.

Tutorial 17: Multi-Agent System

What you'll learn: How to model two independent agents with different roles in one flow, coordinate them through a shared store, and use a third agent as arbitrator.

Why this pattern exists

Some tasks benefit from adversarial or complementary perspectives: one agent argues for a position, another argues against, a third decides. This is the debate pattern, and it generalises to any multi-agent coordination problem — reviewer and author, planner and critic, proposer and validator.

In PocketFlow, multiple “agents” are just nodes with different system prompts. They share the same store, so each can read what the others produced. The coordinator (judge) node reads the full debate history and decides when to conclude.

The pattern

```
[Start] → [AgentA] → [AgentB] → [Judge] —continue→ [AgentA]
                                     ↳conclude→ [Summary]
→ [Stop]
```

Step-by-step

Step 1: Create the project.

New project: `tut_debate`.

Step 2: Add and wire nodes.

Add Start, Basic Node (Agent A), Basic Node (Agent B), Router Node (Judge, Actions: continue, conclude), Basic Node (Summary), Stop.

Wire: Start → Agent A → Agent B → Judge, Judge → Agent A (continue), Judge → Summary (conclude), Summary → Stop.

Step 3: Write Agent A's code (pro argument).

```
class AgentAPropose(Node):
    ROLE = "You argue IN FAVOR of the topic. Be persuasive and specific."

    def prep(self, shared):
        topic = shared.get("topic", "AI regulation is necessary")
        history = shared.get("debate_history", [])
        return topic, history

    def exec(self, prep_res):
        topic, history = prep_res
        context = "\n".join(history[-4:]) # Last 4 exchanges only.

        prompt = (
            f"Topic: {topic}\n"
            f"Role: {self.ROLE}\n"
            f"Debate so far:\n{context or 'None'}\n\n"
            "Make your argument in 2-3 sentences:"
        )
        return call_llm(prompt)

    def post(self, shared, prep_res, exec_res):
        shared.setdefault("debate_history", []).append(f"PRO: {exec_res}")
        return "default"
```

Step 4: Write Agent B's code (counter argument).

```
class AgentBCounter(Node):
    ROLE = "You argue AGAINST the topic. Be critical and precise."

    def prep(self, shared):
        topic = shared.get("topic", "AI regulation is necessary")
        history = shared.get("debate_history", [])
        return topic, history

    def exec(self, prep_res):
        topic, history = prep_res
        context = "\n".join(history[-4:])
        prompt = (
```

```

        f"Topic: {topic}\n"
        f"Role: {self.ROLE}\n"
        f"Debate so far:\n{context or 'None'}\n\n"
        "Make your counter-argument in 2-3 sentences:"
    )
    return call_llm(prompt)

def post(self, shared, prep_res, exec_res):
    shared["debate_history"].append(f"CON: {exec_res}")
    return "default"

```

Step 5: Write Judge's code.

```

class Judge(Node):
    MAX_ROUNDS = 4

    def prep(self, shared):
        history = shared.get("debate_history", [])
        round_num = shared.get("round", 0)
        return history, round_num

    def exec(self, prep_res):
        history, round_num = prep_res
        if round_num >= self.MAX_ROUNDS:
            return "conclude"
        recent = "\n".join(history[-4:])
        prompt = (
            f"You are an impartial debate judge.\n"
            f"Recent arguments:\n{recent}\n\n"
            "Is the debate sufficiently developed to conclude? "
            "Answer CONCLUDE if yes, CONTINUE if another round
would add value."
        )
        verdict = call_llm(prompt).strip().upper()
        return "conclude" if "CONCLUDE" in verdict else
"continue"

    def post(self, shared, prep_res, exec_res):
        shared["round"] = shared.get("round", 0) + 1
        return exec_res

```

Step 6: Write Summary's code.

```

class Summary(Node):
    def prep(self, shared):
        return shared.get("debate_history", []),
shared.get("topic", "")

    def exec(self, prep_res):

```

```

        history, topic = prep_res
        full_debate = "\n".join(history)
        return call_llm(
            f"Summarise this debate on '{topic}'. Identify the
strongest arguments "
            f"on both sides and state which side made the more
compelling case.\n\n"
            f"Debate:\n{full_debate}"
        )

    def post(self, shared, prep_res, exec_res):
        shared["conclusion"] = exec_res
        print("\n=== DEBATE CONCLUSION ===\n", exec_res)
        return "default"

```

Step 7: Run and watch the debate.

Set topic in the Shared Store Designer. Run > Run Active Flow. Watch `debate_history` grow in the Shared Store tab as each agent argues. The Run Log shows which agent ran each iteration.

Expected result: A multi-round debate terminates with a judge summary stored in `conclusion`. `round` in the Shared Store shows how many rounds the debate ran.

Tutorial 23: Streaming Output

What you'll learn: How to design a flow that makes incremental output visible as it is produced, and why streaming matters for user-facing applications.

Why this pattern exists

Standard `call_llm` returns the complete response when the model finishes generating. For a short response this is fine. For a long response — a detailed analysis, a full article — the user waits with no feedback and then sees everything at once. Streaming sends tokens as they are generated, making the system feel responsive.

In Creator, you model streaming by separating the generation step from the display step. The generator node produces output incrementally (or simulates it); the display node reads from the shared store and renders it.

The pattern

[Start] → [StreamLLM] → [PrintChunks] → [Stop]

Step-by-step

Step 1: Create the project.

New project: tut_streaming.

Step 2: Add and wire nodes.

Add Start, Basic Node (Stream LLM, Writes: chunks), Basic Node (Print Chunks), Stop. Wire sequentially with default actions.

Step 3: Write Stream LLM's code.

```
class StreamLlm(Node):
    def prep(self, shared):
        return shared.get("prompt", "Tell me a story in 5
sentences about a robot learning to paint.")

    def exec(self, prep_res):
        # With a real streaming API, you would yield tokens here.
        # With call_llm, we receive the full text and split it
into word chunks
        # to simulate streaming behavior for demonstration
purposes.
        full_text = call_llm(prepare_res)
        return full_text.split() # word-level chunks

    def post(self, shared, prep_res, exec_res):
        shared["chunks"] = exec_res
        return "default"
```

In production, replace `call_llm` with a streaming API call that yields tokens. The node structure — `prep`, `exec`, `post` — stays identical. Only `exec` changes.

Step 4: Write Print Chunks's code.

```
import time

class PrintChunks(Node):
    def prep(self, shared):
        return shared.get("chunks", [])

    def exec(self, prep_res):
        print()
        for chunk in prep_res:
            print(chunk, end=" ", flush=True)
            time.sleep(0.05) # simulate network latency between
tokens

        print()
```

```

    return len(prepare_res)

def post(self, shared, prepare_res, exec_res):
    shared["word_count"] = exec_res
    return "default"

```

Step 5: Run and observe.

Run > Run Active Flow. Watch words appear one at a time in the Run Log's output section. After the flow completes, the Shared Store shows chunks (the full list) and word_count.

Expected result: Words appear progressively, simulating the experience of a real streaming LLM response.

Tutorial 24: Memory — Short-Term and Long-Term Context

What you'll learn: How to maintain two memory layers — recent conversation history and a condensed long-term summary — and when to condense to avoid context window overflow.

Why this pattern exists

A chat history grows indefinitely. Eventually it exceeds the LLM's context window, causing errors or forcing the oldest messages to be truncated. The two-layer memory pattern solves this by periodically condensing old history into a summary, then resetting the short-term buffer. The LLM always has: (a) the full recent history, and (b) a compressed summary of everything older.

This is roughly how long-running AI assistants manage memory. The threshold for condensing (e.g., every 10 messages) and the quality of the summary determine how well the system remembers older context.

The pattern

```

[Start] → [GetInput] → [UpdateShortTerm] → [ShouldCondense?]
    —yes—→ [CondenseToLongTerm] → [CallLLM] → [GetInput]
    —no—→ [CallLLM] → [GetInput]
[GetInput] —exit—→ [Stop]

```

Step-by-step

Step 1: Create the project.

New project: tut_memory.

Step 2: Add and wire nodes.

Add: Start, Basic Node (Get Input, Actions: continue, exit), Basic Node (Update Short Term), Router Node (Should Condense?, Actions: yes, no), Basic Node (Condense To Long Term), LLM Prompt Node (Call LLM), Stop.

Wire: Start → Get Input, Get Input → Update Short Term (continue), Get Input → Stop (exit), Update Short Term → Should Condense?, Should Condense? → Condense To Long Term (yes), Should Condense? → Call LLM (no), Condense To Long Term → Call LLM (default), Call LLM → Get Input (default).

Step 3: Define the shared store schema.

Tools > Shared Store Designer. Add:

Key	Type	Default
short_term_history	array	
long_term_summary	string	
condensation_threshold	integer	10

Step 4: Write Get Input and Update Short Term.

```
class GetInput(Node):
    def prep(self, shared):
        return None

    def exec(self, prep_res):
        return input("You: ").strip()

    def post(self, shared, prep_res, exec_res):
        if exec_res.lower() in ("quit", "exit", "bye"):
            return "exit"
        shared.setdefault("short_term_history", []).append(
            {"role": "user", "content": exec_res}
        )
        return "continue"

class UpdateShortTerm(Node):
    def prep(self, shared):
        return None

    def exec(self, prep_res):
        return None
```

```

def post(self, shared, prep_res, exec_res):
    return "default"

```

Step 5: Write Should Condense? and Condense To Long Term.

```

class ShouldCondense(Node):
    def prep(self, shared):
        history = shared.get("short_term_history", [])
        threshold = shared.get("condensation_threshold", 10)
        return len(history), threshold

    def exec(self, prep_res):
        count, threshold = prep_res
        return "yes" if count >= threshold else "no"

    def post(self, shared, prep_res, exec_res):
        return exec_res

class CondenseToLongTerm(Node):
    def prep(self, shared):
        return (
            shared.get("short_term_history", []),
            shared.get("long_term_summary", "")
        )

    def exec(self, prep_res):
        history, old_summary = prep_res
        msgs = "\n".join(f"{m['role']}: {m['content']}" for m in
history)

        prompt = (
            f"Existing summary of older conversation:
\n{old_summary or 'None'}\n\n"
            f"New messages to incorporate:\n{msgs}\n\n"
            "Write an updated concise summary that captures all
important facts, "

            "decisions, and context from both the old summary and the new
messages:"
        )
        return call_llm(prompt)

    def post(self, shared, prep_res, exec_res):
        shared["long_term_summary"] = exec_res
        shared["short_term_history"] = []
# Reset the short-term buffer.
        return "default"

```

Step 6: Write Call LLM.


```

class CallLlm(Node):
    def prep(self, shared):
        summary = shared.get("long_term_summary", "")
        history = shared.get("short_term_history", [])
        messages = []
        if summary:
            messages.append({
                "role": "system",
                "content": f"Context from earlier in the
conversation: {summary}"
            })
        messages.extend(history)
        return messages

    def exec(self, prep_res):
        return call_llm_with_history(prep_res)

    def post(self, shared, prep_res, exec_res):
        shared.setdefault("short_term_history", []).append(
            {"role": "assistant", "content": exec_res}
        )
        print(f"\nAssistant: {exec_res}\n")
        return "default"

```

Step 7: Test condensation.

Run > Debug Active Flow. Set the `condensation_threshold` to 2 in the Shared Store Designer so condensation triggers after two exchanges. After two messages, watch `long_term_summary` populate and `short_term_history` reset to an empty list.

Expected result: After every `condensation_threshold` messages, the short-term history is compressed into `long_term_summary` and cleared. The conversation continues with the summary providing context for older exchanges.

Tutorial 26: Async Processing with AsyncNode

What you'll learn: How to write non-blocking I/O operations using `AsyncNode`, why async matters for I/O-bound tasks, and how `prep_async/exec_async/post_async` correspond to the synchronous lifecycle.

Why this pattern exists

A standard `Node.exec` blocks the thread while it waits for a network response, a database query, or a file system read. If you are running one node at a time, this is

fine. If you want to run multiple operations concurrently — or if you are integrating with an async web framework — blocking is a problem.

AsyncNode uses `async def` methods so the event loop can suspend execution during I/O waits and run other coroutines. The lifecycle is identical to a synchronous node: `prep_async` reads, `exec_async` works, `post_async` writes. The only difference is that each method is awaited.

The pattern

[Start] → [FetchData (AsyncNode)] → [Process] → [Stop]

Step-by-step

Step 1: Create the project.

New project: `tut_async_node`.

Step 2: Drag the Async Node from the palette.

Drag **Async Node** onto the canvas. Title it `Fetch Data`. In Inspector, confirm that Base Class shows `AsyncNode`. This tells the code generator and runner that this node uses the async lifecycle.

Step 3: Write Fetch Data's code.

```
import aiohttp

class FetchData(AsyncNode):
    async def prep_async(self, shared):
        return shared.get("url", "https://httpbin.org/get")

    async def exec_async(self, prep_res):
        async with aiohttp.ClientSession() as session:
            async with session.get(prep_res) as resp:
                resp.raise_for_status()
                return await resp.json()

    async def post_async(self, shared, prep_res, exec_res):
        shared["response"] = exec_res
        return "default"
```

The `async with` pattern ensures the HTTP session is closed even if an exception occurs. `raise_for_status()` turns HTTP error codes into exceptions, which triggers the framework's retry logic rather than silently writing an error response to the shared store.

Step 4: Wire Start → Fetch Data → Process → Stop.

Add a Basic Node (Process) that reads `shared["response"]` and formats the output.

Step 5: Install aiohttp if needed.

```
pip install aiohttp
```

Step 6: Run and inspect.

Run > Run Active Flow. The Shared Store shows `response` as a parsed JSON object from the HTTP response. Even though `exec_async` awaits a network call, the runner handles this transparently.

When to use AsyncNode: - HTTP calls, database queries, file I/O — any operation that waits on external systems. - When integrating with async web frameworks (FastAPI, aiohttp server). - Do not use it for CPU-bound work (e.g., matrix operations) — `async` does not add parallelism for CPU work, only for I/O waits.

Expected result: The Shared Store shows the parsed JSON response from `httpbin.org/get`.

Tutorial 27: Async Batch Processing with AsyncBatchNode

What you'll learn: How to process a list of items with async operations, one item at a time sequentially, and why sequential async is sometimes preferable to parallel async.

Why this pattern exists

`AsyncBatchNode` is to `AsyncNode` what `BatchNode` is to `Node`: it iterates over a list returned by `prep_async`, calling `exec_async` once per item. The iteration is sequential — each `exec_async` completes before the next begins.

This makes `AsyncBatchNode` the right choice when you need async I/O (to avoid blocking) but also need to respect rate limits or ordering guarantees. For true concurrency, use `AsyncParallelBatchNode` (Tutorial 28).

The pattern

```
[Start] → [FetchAll (AsyncBatchNode)] → [Aggregate] → [Stop]
```

Step-by-step

Step 1: Create the project.

New project: tut_async_batch.

Step 2: Drag Async Batch from the palette.

Title it Fetch All. Confirm Base Class: AsyncBatchNode in Inspector.

Step 3: Write Fetch All's code.

```
import aiohttp

class FetchAll(AsyncBatchNode):
    async def prep_async(self, shared):
        # Return the list. AsyncBatchNode calls exec_async once per URL.
        return shared.get("urls", [
            "https://httpbin.org/get",
            "https://httpbin.org/ip",
            "https://httpbin.org/headers",
        ])

    async def exec_async(self, url):
        # url is a single string, not the full list.
        async with aiohttp.ClientSession() as session:
            async with session.get(url,
                timeout=aiohttp.ClientTimeout(total=10)) as resp:
                return await resp.text()

    async def post_async(self, shared, prep_res, exec_res):
        # exec_res is a list: one result per URL.
        shared["pages"] = exec_res
        return "default"
```

Step 4: Write Aggregate's code.

```
class Aggregate(Node):
    def prep(self, shared):
        return shared.get("pages", [])

    def exec(self, prep_res):
        # Extract just the first 200 characters of each page.
        return [p[:200] for p in prep_res]

    def post(self, shared, prep_res, exec_res):
        shared["summaries"] = exec_res
        for i, s in enumerate(exec_res):
            print(f"Page {i+1}: {s[:80]}...")
        return "default"
```

Step 5: Run and compare timing.

Run > Run Active Flow. The three URLs are fetched one after the other. Check the Run Log's timing information — the total time is the sum of all three individual fetch times.

Now compare this to Tutorial 28 (parallel) where the total time is approximately the slowest single fetch.

Expected result: pages contains three raw HTTP response bodies; summaries contains their first 200 characters each.

Tutorial 28: Concurrent Async Batch with AsyncParallelBatchNode

What you'll learn: How to fire multiple async operations simultaneously using AsyncParallelBatchNode, and when concurrency is worth the added complexity.

Why this pattern exists

When you have ten URLs to fetch and each takes one second, sequential async takes ten seconds. Parallel async fires all ten simultaneously and finishes in approximately one second. The speedup is real and significant for I/O-bound workloads.

The tradeoff: parallel requests hit external services all at once. Most APIs have rate limits. The `max_concurrency` property caps how many requests run simultaneously, giving you control over the burst rate.

The pattern

[Start] → [ParallelFetch (AsyncParallelBatchNode)] → [Summarise] → [Stop]

Step-by-step

Step 1: Create the project.

New project: `tut_parallel_batch`.

Step 2: Drag Async Parallel Batch from the palette.

Title it `Parallel Fetch`. In Inspector, set `max_concurrency` to 5. This means no more than five requests run simultaneously even if the URL list has 100 entries.

Step 3: Write Parallel Fetch's code.

```

import aiohttp

class ParallelFetch(AsyncParallelBatchNode):
    async def prep_async(self, shared):
        return shared.get("urls", [
            "https://httpbin.org/get",
            "https://httpbin.org/ip",
            "https://httpbin.org/headers",
            "https://httpbin.org/user-agent",
            "https://httpbin.org/uuid",
        ])

    async def exec_async(self, url):
        async with aiohttp.ClientSession() as session:
            async with session.get(url,
                timeout=aiohttp.ClientTimeout(total=10)) as resp:
                return {
                    "url": url,
                    "status": resp.status,
                    "size": len(await resp.text()),
                }

    async def post_async(self, shared, prep_res, exec_res):
        # Results are in the same order as the input list, even
        though

        # execution happened concurrently.
        shared["results"] = exec_res
        return "default"

```

The result order guarantee is important: even though requests complete in arbitrary order, `post_async` receives results in the original URL list order. You can safely zip results with the input list.

Step 4: Write Summarise's code.

```

class Summarise(Node):
    def prep(self, shared):
        return shared.get("results", [])

    def exec(self, prep_res):
        lines = [f"{r['url']} → status {r['status']}, {r['size']} bytes" for r in prep_res]
        return "\n".join(lines)

    def post(self, shared, prep_res, exec_res):
        shared["report"] = exec_res
        print(exec_res)
        return "default"

```

Step 5: Run and observe the timing.

Run > Run Active Flow. All five requests fire concurrently. The Run Log shows them all completing before Summarise begins. The total time is close to the slowest individual request, not the sum of all requests.

When to use parallel vs. sequential async: - Use parallel when you have independent I/O operations and want minimum total time. - Use sequential (AsyncBatchNode) when you need to respect rate limits or when results from one call affect the next. - Use plain BatchNode when the operations are synchronous (no async I/O needed).

Expected result: results contains five entries with status codes and sizes. Total execution time is much less than five times the per-request time.

Tutorial 29: Using the Agent Node

What you'll learn: How the Agent Node encapsulates the tool-calling loop as a single configured component, and how it differs from building the agent loop manually in Tutorial 12.

Why this pattern exists

Tutorial 12 showed the agent pattern in full detail: a Router Node, three tool nodes, loop- back edges. That approach gives you complete control. The Agent Node is the same pattern wrapped into a reusable component with configurable properties.

Use the Agent Node when the tool list is known at configuration time and the decision logic is standard (ask the LLM, pick a tool, loop). Build the manual pattern when you need custom routing logic, non-standard tools, or visibility into each decision step.

The pattern

```
[Start] → [Agent (AgentNode)] —tool_call→ [ExecuteTool] →  
[Agent]                                     ↳answer→ [Stop]
```

Step-by-step

Step 1: Create the project.

New project: tut_agent_node.

Step 2: Drag Agent Node from the palette.

Title it Agent. In Inspector, set: - tools: ["search", "calculate"] - max_iterations: 10

These properties configure the node without requiring code changes.

Step 3: Write Agent's code.

```
TOOLS = {
    "search": lambda q: f"[Search result for '{q}': relevant information found]",
    "calculate": lambda expr: f"[Calculation result: {eval(expr)} if expr.replace('.', '').replace('-', '').isdigit() else 'N/A']]", # noqa: S307
}

class Agent(Node):
    def prep(self, shared):
        return shared.get("question", ""), shared.get("tool_history", [])

    def exec(self, prep_res):
        question, history = prep_res
        ctx = "\n".join(f"- {h}" for h in history[-6:])
        prompt = (
            f"Question: {question}\n"
            f"Available tools: {', '.join(TOOLS.keys())}\n"
            f"Steps taken:\n{ctx or 'none'}\n\n"
            "What should I do next? Reply with exactly one tool name, or 'answer' to give a final response."
        )
        return call_llm(prompt).strip().lower()

    def post(self, shared, prep_res, exec_res):
        question, history = prep_res
        iterations = shared.get("agent_iterations", 0) + 1
        shared["agent_iterations"] = iterations
        shared.setdefault("tool_history", []).append(f"step {iterations}: {exec_res}")

        if iterations >= 10 or exec_res not in TOOLS:
            return "answer"
        return "tool_call"
```

Step 4: Write Execute Tool's code.

```
class ExecuteTool(Node):
    def prep(self, shared):
```



```

history = shared.get("tool_history", [])
# The last entry is the most recent decision.
last = history[-1] if history else ""
# Parse "step N: tool_name" to find which tool to call.
tool_name = last.split(": ", 1)[-1] if ": " in last else ""

"search"

return tool_name, shared.get("question", "")

def exec(self, prep_res):
    tool_name, question = prep_res
    tool_fn = TOOLS.get(tool_name, TOOLS["search"])
    return tool_fn(question)

def post(self, shared, prep_res, exec_res):
    shared["tool_history"].append(f"result: {exec_res}")
    shared["last_tool_result"] = exec_res
    return "default"

```

Step 5: Wire and run in debug mode.

Wire: Start → Agent, Agent → Execute Tool (tool_call), Execute Tool → Agent (default), Agent → Stop (answer).

Run > Debug Active Flow. Set a breakpoint on **Agent** to watch each decision. After each tool call, check tool_history growing in the Shared Store. The loop terminates when the LLM decides answer or when max_iterations is reached.

Expected result: The agent loops through tool calls and terminates with a final answer in the Shared Store. agent_iterations shows how many rounds it took.

Tutorial 30: Using the RAG Node

What you'll learn: How the RAG Node separates the embedding step from the retrieval step in a clean, inspectable way, and when to use it versus the full RAG pipeline from Tutorial 13.

Why this pattern exists

Tutorial 13 showed the complete RAG pipeline: chunking, embedding, storing, querying, retrieving, generating. The RAG Node packages the embedding step into a single configured component, making the pipeline shorter to set up for standard use cases.

Use the full pipeline from Tutorial 13 when you need to customise chunking strategy, embedding models, or retrieval algorithms. Use the RAG Node when the defaults are sufficient and you want less boilerplate.

The pattern

[Start] → [EmbedQuery (RAG Node)] → [Retrieve] → [Generate] → [Stop]

Step-by-step

Step 1: Create the project.

New project: tut_rag_node.

Step 2: Drag RAG Node from the palette.

Title it Embed Query. In Inspector, set: - top_k: 5 — how many chunks to retrieve
- index_key: embeddings — the shared store key where embeddings are stored

Step 3: Write Embed Query's code.

```
class EmbedQuery(Node):
    def prep(self, shared):
        return shared.get("question", "")

    def exec(self, prep_res):
        # In production: return embedding_model.encode(prepare_res)
        import hashlib
        h = hashlib.md5(prepare_res.encode()).digest()
        return [b / 255.0 for b in h]

    def post(self, shared, prep_res, exec_res):
        shared["query_vector"] = exec_res
        return "default"
```

Step 4: Write Retrieve's code.

```
class Retrieve(Node):
    TOP_K = 5

    def prep(self, shared):
        return (
            shared["query_vector"],
            shared.get("embeddings", []),
            shared.get("chunks", []),
        )

    def exec(self, prep_res):
        q_vec, emb_list, chunks = prep_res
        def dot(a, b):
            return sum(x * y for x, y in zip(a, b))
        scored = sorted(
```

```

        [(dot(q_vec, e), c) for e, c in zip(emb_list,
chunks)],
        reverse=True,
    )
    return [c for _, c in scored[:self.TOP_K]]

def post(self, shared, prep_res, exec_res):
    shared["context"] = exec_res
    return "default"

```

Step 5: Write Generate's code.

```

class Generate(Node):
    def prep(self, shared):
        context = "\n\n".join(shared.get("context", []))
        question = shared.get("question", "")
        return (
            f"Use only the following context to answer the\n\n"
            f"Context:\n{context}\n\n"
            f"Question: {question}\n\n"
            f"Answer:"
        )

    def exec(self, prep_res):
        return call_llm(prep_res)

    def post(self, shared, prep_res, exec_res):
        shared["answer"] = exec_res
        print("Answer:", exec_res)
        return "default"

```

Step 6: Set up the index.

Before running, populate embeddings and chunks in the Shared Store Designer with pre-computed values, or run the indexing phase from Tutorial 13 first and copy the values across. In production, indexing runs once and the index is stored in a vector database.

Step 7: Run and inspect.

Set question in the Shared Store Designer. Run > Run Active Flow. Inspect query_vector, context (retrieved chunks), and answer in the Shared Store tab.

Expected result: The retrieved context chunks are semantically related to the question, and the answer is grounded in that context rather than the LLM's training data.

Tutorial 31: Using the Judge Node

What you'll learn: How the Judge Node evaluates LLM output against explicit criteria, how the max-iteration guard ensures termination, and how to configure the criteria for different quality standards.

Why this pattern exists

Tutorial 16 showed the evaluator loop built manually from scratch. The Judge Node packages that pattern into a single configured component. The criteria, the maximum iterations, and the pass/fail routing are all set in the Inspector rather than coded into exec.

This makes the judge easy to reuse: change the criteria for a new task without touching the node's code structure.

The pattern

```
[Start] → [Generate] → [Judge (Judge Node)] —pass—→ [Stop]
                                     ^               |fail—→ [Refine] →
[Generate]
```

Step-by-step

Step 1: Create the project.

New project: tut_judge_node.

Step 2: Drag Judge Node from the palette.

Title it Judge. In Inspector, set: - max_iterations: 3 — the flow will always pass after 3 evaluations regardless of quality - criteria: "Is the output clear, accurate, and complete? Does it fully answer the question?"

Step 3: Write Generate's code.

```
class Generate(Node):
    def prep(self, shared):
        task = shared.get("task", "Explain recursion in one
paragraph for a beginner.")
        feedback = shared.get("judge_feedback", "")
        if feedback:
            output = shared.get("output", "")
            return f"Revise this output based on the feedback.
\nFeedback: {feedback}\n\nOriginal:\n{output}"
        return task
```

```

def exec(self, prep_res):
    return call_llm(prepare_res)

def post(self, shared, prep_res, exec_res):
    shared["output"] = exec_res
    return "default"

```

Step 4: Write Judge's code.

```

class Judge(Node):
    MAX_ITERATIONS = 3

    def prep(self, shared):
        return shared.get("output", ""),
shared.get("judge_iteration", 0)

    def exec(self, prep_res):
        output, iteration = prep_res
        if iteration >= self.MAX_ITERATIONS:
            return "pass"

        criteria = shared.get(
            "criteria",
            "Is the output clear, accurate, and complete?"
        )
        prompt = (
            f"Evaluate this output against the criteria.\n"
            f"Criteria: {criteria}\n\n"
            f"Output:\n{output}\n\n"
            "Respond with PASS if the criteria are met, or FAIL"
followed by "
            "one sentence of specific, actionable feedback."
        )
        verdict = call_llm(prompt).strip()
        if verdict.upper().startswith("PASS"):
            return "pass"
        return ("fail", verdict)

    def post(self, shared, prep_res, exec_res):
        output, iteration = prep_res
        shared["judge_iteration"] = iteration + 1
        if isinstance(exec_res, tuple):
            action, feedback = exec_res
            shared["judge_feedback"] = feedback
            return action
        return exec_res

```

Step 5: Write Refine's code.

Refine is a passthrough — Generate reads `judge_feedback` directly on its next call:

```
class Refine(Node):
    def prep(self, shared):
        return None

    def exec(self, prep_res):
        return None

    def post(self, shared, prep_res, exec_res):
        return "default"
```

Step 6: Wire the refinement loop.

Start → Generate → Judge, Judge → Stop (pass), Judge → Refine (fail), Refine → Generate (default).

Step 7: Run in debug mode.

Set task and criteria in the Shared Store Designer. Run > Debug Active Flow with a breakpoint on **Judge**. After each evaluation, check `judge_feedback` in the Shared Store. After `max_iterations`, the Judge forces pass regardless of the verdict.

Step 8: Tune the criteria.

Change criteria in the Shared Store Designer and rerun without changing any code. The same Judge node evaluates against the new criteria. This is the advantage of making criteria a runtime property rather than hardcoding it in `exec`.

Expected result: The flow terminates in 1–4 iterations. `judge_iteration` shows the count. output contains the final (possibly refined) result. `judge_feedback` shows the last piece of feedback given (empty if the first attempt passed).

Tutorial 32: Human Input + Classifier — Interactive Sentiment Triage

What you'll learn: How to collect free-text input from the user at runtime using a Human Input Node, then route the flow to different branches based on sentiment classification with a Classifier Node.

Prerequisites: Tutorial 15 (Human-in-the-Loop) and Tutorial 11 (Conditional Routing).

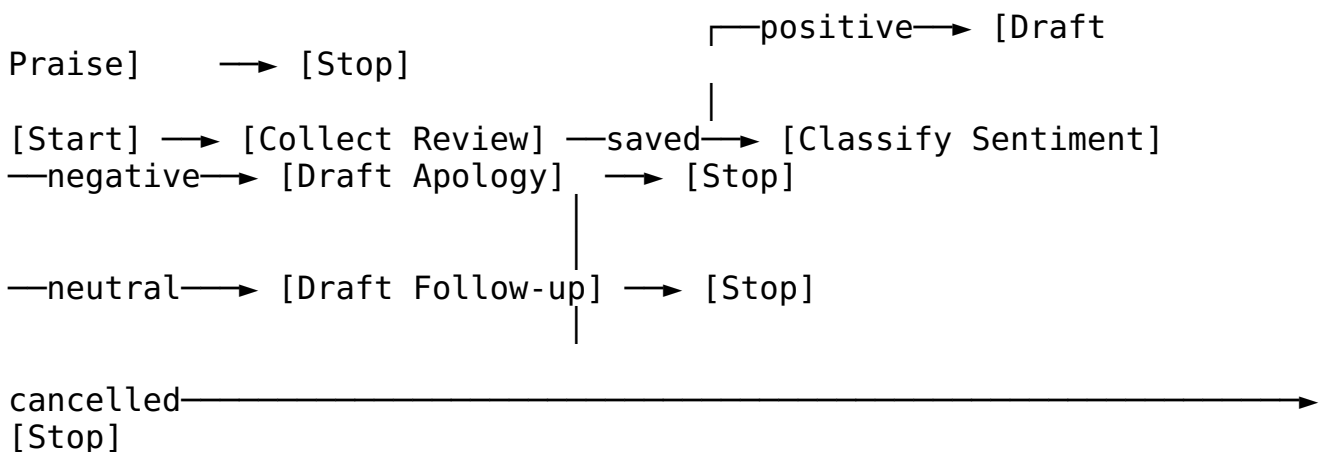
Why this combination exists

Most LLM workflows operate on data that was prepared before the run started — a file, a database query, a previous step's output. But many real applications need to ask the user something mid-flow: which document to process, what question to answer, or — as in this tutorial — what text to analyse.

The Human Input Node pauses the runner, shows a dialog to collect data, and merges the result into shared state before the next node executes. Paired with a Classifier Node, you get a pattern that appears constantly in customer support, content moderation, and triage systems: the user provides the input, the LLM labels it, and the flow branches to the handler that matches that label.

The flow you will build

A customer review sentiment triage tool. The user types a review into a dialog at runtime. The Classifier Node sends it to an LLM and asks it to label the sentiment as positive, negative, or neutral. Each branch drafts an appropriate follow-up response using a separate LLM Prompt Node.



The saved / cancelled split handles the case where the user closes the dialog without saving — the flow exits cleanly without calling the LLM.

Step-by-step

Step 1: Create the project.

File > New Project. Name it tut_sentiment_triage. Creator creates the standard folder structure and opens an empty canvas.

Step 2: Add the Start Node.

Drag **Start Node** from the palette onto the canvas. It becomes the flow's entry point automatically.

Step 3: Add a Human Input Node.

Drag **Human Input Node** onto the canvas to the right of Start. In the Inspector, set:

Property	Value
Title	Collect Review
input_type	form
fields	review:string::
output_key	input

The fields value `review:string::` defines a single text field: - `review` — the field label shown in the dialog and the key written to shared state - `string` — data type (plain text) - third segment empty — no default value - fourth segment empty — no dropdown choices

When the node runs, a dialog appears with a single text box labelled **review**. When the user clicks **Save**, the value is written to `shared[ 'review' ]`. When the user clicks **Cancel**, the node takes the cancelled action instead.

Field definition syntax: `label:type:default:choices`

Multiple fields are separated by semicolons. Examples: -
`name:string:Anonymous:` — text field, default “Anonymous” -
`rating:integer:5:` — integer spinner, default 5 -
`status:string::active,closed,pending` — dropdown, three choices

Step 4: Add a Classifier Node.

Drag **Classifier Node** onto the canvas to the right of Collect Review. In the Inspector, set:

Property	Value
Title	Classify Sentiment
input_key	review
categories	positive,negative,neutral

`input_key: review` tells the Classifier to read `shared[ 'review' ]` — the same key the Human Input Node wrote. `categories` lists the exact labels the LLM

should choose from. These labels must match the action names on the outgoing edges you will draw in Step 6.

Leave `prompt_file` empty. The Classifier auto-builds a prompt from categories and the text at `input_key`.

Step 5: Add three LLM Prompt Nodes and a Stop Node.

Drag three **LLM Prompt Node** items onto the canvas, one for each branch. Name them:

- Draft Praise (positive branch)
- Draft Apology (negative branch)
- Draft Follow-up (neutral branch)

Drag a **Stop Node** onto the canvas. A single Stop Node handles all three branches — you will connect all three LLM nodes to it.

For each LLM node, set `prompt_type` to `string` and fill `prompt_file` with a branch-specific prompt:

Draft Praise — `prompt_file`:

The customer left this positive review: {review}

Write a short, warm thank-you message (2–3 sentences) acknowledging their feedback.

Draft Apology — `prompt_file`:

The customer left this negative review: {review}

Write a sincere apology and offer to resolve the issue. Keep it under 4 sentences.

Draft Follow-up — `prompt_file`:

The customer left this neutral review: {review}

Write a friendly follow-up asking what we could do better. Keep it under 3 sentences.

The {review} placeholder is replaced at runtime with `shared['review']` — the text the user entered in the dialog.

For each LLM node, set `output_key` to `response` so all three branches write to the same key (the Stop node just ends the flow regardless).

Step 6: Wire the edges.

Connect nodes in this order, using the action names shown:

From	Action	To
Start	default	Collect Review
Collect Review	saved	Classify Sentiment
Collect Review	cancelled	Stop
Classify Sentiment	positive	Draft Praise
Classify Sentiment	negative	Draft Apology
Classify Sentiment	neutral	Draft Follow-up
Draft Praise	default	Stop
Draft Apology	default	Stop
Draft Follow-up	default	Stop

The Classifier Node's outgoing action names (positive, negative, neutral) must exactly match the category labels you set in the categories property. The Classifier asks the LLM to reply with one of those labels, then uses the reply to choose which edge to follow.

If the LLM returns a label that is not an exact match, the Classifier does fuzzy matching — it checks whether the response *contains* one of the action names. For example, "The sentiment is negative." matches the negative edge. If no match is found, it falls through to the first available action.

Step 7: Arrange and Validate.

Drag the nodes into a left-to-right arrangement that reflects the flow structure — Start on the left, Stop on the right, branches fanning out in between. Then Project > Validate Project (Ctrl+Shift+V). All nodes should show green — no missing connections or unreachable edges.

Step 8: Run the flow.

Run > Run Active Flow (F5).

A dialog titled **Collect Review** appears with a single field:

Collect Review
review: [

[Save] [Cancel]

Form vs List mode buttons: In **form mode** the dialog has **Save** and **Cancel** — click Save once all fields are filled. In **list mode** the dialog has **Add, Remove, Done**, and **Cancel** — use Add to collect as many items as needed, then click Done to finalise.

Type a review and click **Save**. The flow continues to the Classifier, which labels the sentiment, routes to the matching LLM node, and writes the drafted response to `shared['response']`.

Switch to the **Run Log** tab. You will see:

<code>[start]</code>	Start	→ default
<code>[collect_review]</code>	Collect Review	→ saved
<code>[classify]</code>	Classify Sentiment	→ negative
<code>[draft_apology]</code>	Draft Apology	→ default
<code>[stop]</code>	Stop	→ default

Switch to the **Shared Store** tab. You will see:

```
review: This product broke after two days. Very disappointed.
classify_label: negative
response: We're truly sorry to hear about your experience...
```

Click **Cancel** in the dialog instead of Save: the Run Log shows Collect Review → cancelled and the flow goes straight to Stop — no LLM call is made.

Step 9: Extend the flow.

Now that the skeleton works, try these variations:

Add a second field. Change fields in Collect Review to:

```
name:string:Anonymous;;review:string::
```

Two fields: name (with default “Anonymous”) and review. The Shared Store now contains `shared['name']` and `shared['review']`. Update the prompts to include `{name}` where appropriate.

Add a dropdown. Change fields to:

```
product:string::Widget Pro,Widget Lite,Widget Max;review:string::
```

The product field becomes a dropdown. Shared state gets `shared['product']` and `shared['review']`.

Add more categories. Add urgent to categories: positive, negative, neutral, urgent. Draw a new edge from **Classify Sentiment** with action urgent to a new **Draft Escalation** LLM node. No other node changes — just wire the new branch.

Key concepts from this tutorial

Concept	Where it appears
Human Input pauses the runner	Collect Review shows a dialog; the runner thread blocks until Save or Cancel
Field definitions	label: type: default: choices syntax; multiple fields separated by ;
saved / cancelled routing	Human Input takes saved when the user clicks Save, cancelled on Cancel
Prompt interpolation	{review} in any prompt is replaced with shared['review'] at runtime
Classifier label matching	The LLM is asked to choose one of the categories; the reply selects the outgoing edge
Fuzzy label matching	If the LLM elaborates rather than replying with just the label, the runner extracts the best match

[→ Continue to Part 3 — Advanced Features](#)

Part 3 — Advanced Creator Features

This section covers Creator features that go beyond building and running a single flow. You will learn how to catch problems before they happen (validation), investigate them when they do (debugging), compose flows from reusable sub-graphs (subflow composition), take your work outside Creator (export), document your data contracts formally (Shared Store Designer), and share your tools with others (packaging).

Each tutorial explains not just the steps but the reasoning behind each feature — why it exists, what problem it solves, and when you would reach for it.

[← Tutorials Index](#)

Tutorial 18: Validation and Error Badges

What you'll learn: How Creator's validator detects structural problems in your graph before you run, what each error code means, and how to read the visual error indicators.

Why validation exists

A flow that looks correct on the canvas can fail at runtime in ways that are hard to diagnose: a node with no outgoing edges leaves the runner nowhere to go; an action label that does not match a declared action causes a silent routing failure; a missing start node means the runner cannot determine where to begin. Catching these problems before running saves the time spent reading Python tracebacks to find graph-level errors.

Creator's validator analyses the graph structure — not the Python code — and reports problems as numbered error codes. Validation runs automatically before Run and before Export, so you cannot accidentally run a structurally broken flow.

Understanding error codes

Code	What is wrong	How to fix it
PFCE1001	No start node is set	Mark a node as the entry point: select it and choose Flow > Set Start Node
PFCE1002	Two nodes share the same ID	Delete and re-add the duplicate; Creator assigns a fresh ID
PFCE1003	The declared start node ID does not exist	Re-set the start node via Flow > Set Start Node
PFCE2001	An edge's source node has been deleted	Delete the dangling edge and re-wire
PFCE2002	An edge's destination node has been deleted	Delete the dangling edge and re-wire
PFCE2003	An edge has no action label	Click the edge label and type an action name
PFCE2101	An edge's action is not in the source node's declared Actions	Add the action to the node's Actions field in Inspector
PFCE2102	A Subflow Node has no subflow_ref set	Set subflow_ref in Inspector to a valid graph file path

Reading error badges

When validation finds a problem, Creator draws a red border around the affected node and marks the node's corner with a red badge. These badges persist until the error is resolved. The Problems panel (View > Problems) lists all errors with their codes, descriptions, and the node or edge where the error was found. Clicking a problem in the panel selects the affected element on the canvas.

Step-by-step

Step 1: Create a deliberately broken graph.

New project: `tut_validation`. Add a Basic Node onto the canvas. Do not add Start or Stop nodes. Do not wire any edges. This graph has PFCE1001 (no start node) and implicitly PFCE2001/PFCE2002 issues if any edges existed.

Step 2: Run the validator.

Project > Validate Project (Ctrl+Shift+V). The Problems panel opens listing at least PFCE1001. The Basic Node gains a red border badge because it is the only node and therefore implicated in the missing-start-node problem.

Step 3: Fix each error.

Select the Basic Node. Flow > Set Start Node. Run the validator again — PFCE1001 clears.

Add a Stop Node. Draw an edge from the Basic Node to the Stop Node. The edge has no label: PFCE2003 appears. Click the edge label (double-click the arrow between nodes) and type `default`. PFCE2003 clears.

Add a second Basic Node. Draw an edge from the first Basic Node to this second node with action `process`. The second node's declared Actions list does not include `process`: PFCE2101 appears. Select the first Basic Node. In Inspector, add `process` to the Actions field. PFCE2101 clears.

Step 4: Verify a clean graph.

Run Validate again. The Problems panel should be empty. All red badges should be gone. The flow is now structurally valid and can be run or exported.

Tip: Use validation as a first check whenever a flow fails unexpectedly. Graph-level errors in Creator are caught here; Python logic errors appear in the Run Log during execution.

Tutorial 19: Debug Mode and Breakpoints

What you'll learn: How to step through a running flow node by node, inspect the shared store at each pause, and use breakpoints to isolate where things go wrong.

Why debug mode exists

When a flow produces wrong output, the question is: which node produced it? You could add `print` statements everywhere and re-run, but that is slow and leaves the code cluttered. Debug Mode lets you pause execution at any node and inspect the full shared store state at that exact moment — before and after every key transformation.

Breakpoints in Creator work like breakpoints in a code debugger: execution pauses when the flow reaches the marked node, you inspect what you need, and you resume when ready. Unlike a code debugger, Creator's breakpoints pause between nodes (at the graph level) rather than between lines of code.

Step-by-step

Step 1: Open a project with a non-trivial flow.

Open the project from Tutorial 7 (Hello World), or any project with at least two active nodes. A flow with a single node is not useful for breakpoint demonstration because there is only one place to pause.

Step 2: Set a breakpoint.

Click the **Ask LLM** node to select it. Press F9, or use Node > Toggle Breakpoint. A red dot appears in the node's top-right corner. This dot is a persistent marker — it is saved with the project and visible every time you open the canvas.

Step 3: Start a debug session.

Run > Debug Active Flow (Shift+F5). The flow starts executing normally until it reaches Ask LLM. At that point, execution pauses and the status bar shows "Paused at: Ask LLM".

Step 4: Inspect state before the node runs.

With execution paused before Ask LLM, check the Shared Store tab. You see everything that nodes before Ask LLM have written. In this flow, Start wrote nothing — but in a multi-node flow you would see the outputs of all completed nodes here.

This “before” state tells you whether the inputs to the paused node are correct. If question is missing or malformed at this point, the bug is in an earlier node, not in Ask LLM.

Step 5: Resume and inspect the “after” state.

Run > Resume (F5). The flow runs Ask LLM and pauses again if there is another breakpoint, or continues to the end. After the node completes, the Shared Store shows the new values it wrote — in this case, answer.

Debug session — breakpoint cleared on the Classify Sentiment node,
positive path taken, Run Log shows execution trace

Debug session — breakpoint cleared on the Classify Sentiment node,
positive path taken, Run Log shows execution trace

Step 6: Use multiple breakpoints in a loop.

Open Tutorial 8 (Chat). Set breakpoints on **Get Input** and **Call LLM**. Run in debug mode. The flow pauses at Get Input (before asking for user input), then at Call LLM (before the LLM call). You can watch history accumulate in the Shared Store with each iteration through the loop.

Step 7: Stop a debug session.

Run > Stop. The flow terminates immediately regardless of where it was paused. Any shared store state accumulated so far is discarded.

Tips for effective debugging: - Set breakpoints on the node just *before* the suspected problem to verify its inputs. - Set breakpoints on the node just *after* the problem to verify its outputs. - Use the Run Log tab alongside the Shared Store tab: the log shows timing and errors; the store shows data. - Remove all breakpoints (Node > Remove All Breakpoints) before running for performance testing.

Tutorial 20: Subflow Composition

What you’ll learn: How to embed a reusable sub-graph inside a parent flow using the Subflow Node, when composition is the right design choice, and what happens to shared store state across the boundary.

Why subflow composition exists

As flows grow, they become hard to read and maintain. A 20-node flow is difficult to navigate; a parent flow with four Subflow Nodes each pointing to a 5-node sub-graph is much clearer. More importantly, sub-graphs are reusable: if three

different parent flows all need a summarization pipeline, you build it once and reference it from all three.

Subflow composition in Creator maps directly to PocketFlow's Flow nesting. When the runner reaches a Subflow Node, it executes the referenced graph inline and merges that graph's shared store changes back into the parent's shared store. The boundary is transparent at runtime.

Step-by-step

Step 1: Create the project.

New project: `tut_subflow`. This project will have two graphs.

Step 2: Build the reusable sub-graph.

In Project Explorer, right-click the `graphs/` folder and create a new graph: `summarizer.pfcgraph.yaml`. Click it to open the canvas for that graph.

Add: Start Node → Basic Node (Load Text) → LLM Prompt Node (Summarize) → Stop Node. Wire sequentially with default actions.

Write Summarize's code:

```
class Summarize(Node):
    def prep(self, shared):
        return shared.get("text_to_summarize", "")

    def exec(self, prep_res):
        if not prep_res:
            return "No text provided."
        return call_llm(f"Summarize in one paragraph:
\n{prep_res}")

    def post(self, shared, prep_res, exec_res):
        shared["summary"] = exec_res
        return "default"
```

Validate the summarizer graph. It should be clean.

Step 3: Build the parent flow.

In Project Explorer, click `main.pfcgraph.yaml` to return to the main canvas. Add: Start Node, Basic Node (Prepare Content), Subflow Node, Basic Node (Use Summary), Stop Node.

Step 4: Configure the Subflow Node.

Select the Subflow Node. In Inspector, the [Subflow] section appears at the bottom. Set `subflow_ref` to `graphs/summarizer.pfcgraph.yaml`. This is the relative path from the project root to the sub-graph file.

After setting `subflow_ref`, the Subflow Node's title updates to show the referenced graph name. The validator checks that this file exists and is a valid graph (PFCE2102 clears).

Step 5: Write the surrounding nodes.

```
class PrepareContent(Node):
    def prep(self, shared):
        return None

    def exec(self, prep_res):
        # In production, fetch from an API, database, or file.
        return (
            "PocketFlow is a 100-line Python framework for
building LLM pipelines. "
            "It uses a shared store to pass data between nodes.
Nodes follow a "
            "prep/exec/post lifecycle. The framework is designed
for clarity and "
            "minimal abstraction."
        )

    def post(self, shared, prep_res, exec_res):
        # Write to the key the summarizer reads from.
        shared["text_to_summarize"] = exec_res
        return "default"

class UseSummary(Node):
    def prep(self, shared):
        return shared.get("summary", "")

    def exec(self, prep_res):
        print("\nSummary result:", prep_res)
        return prep_res

    def post(self, shared, prep_res, exec_res):
        return "default"
```

Step 6: Wire the parent flow.

Start → Prepare Content → Subflow Node → Use Summary → Stop. All default actions.

Step 7: Run the parent flow.

Run > Run Active Flow from the main graph canvas. The runner executes Prepare Content, then enters the summarizer sub-graph (Load Text → Summarize), then returns to the parent and runs Use Summary. The Shared Store tab shows text_to_summarize (written by Prepare Content) and summary (written by the sub-graph's Summarize node) — both visible in the same store because the sub-graph merges its changes back.

When to use subflow composition: - When the same sequence of nodes appears in multiple flows. - When a flow is long enough that collapsing sections into sub-graphs improves readability. - When different team members own different sub-graphs and need to develop them independently.

Expected result: summary in the Shared Store contains a one-paragraph condensation of the prepared content, produced by the sub-graph's Summarize node.

Tutorial 21: Exporting and Running a Standalone Project

What you'll learn: How to export a Creator project as a self-contained Python package that runs without Creator installed, and how the export protects your custom code from being overwritten.

Why standalone export exists

Creator is a development tool. Your users, your CI/CD pipeline, and your production servers do not run Creator — they run Python. The export feature converts your Creator project into a standard Python package that anyone can install and run with `pip install pocketflow && python main.py`.

The export is designed around a critical safety rule: code you write by hand is never overwritten. The generated files (which Creator rebuilds from your graph each time) are always in `generated/`. The custom files (which you edit) are in `custom/`. Re-exporting updates `generated/` and leaves `custom/` untouched.

The project structure

When you export, Creator writes:

```
exports/<project_name>/
├── generated/
│   ├── main_nodes.py      ← auto-generated Node subclasses
│   └── main_flow.py       ← auto-generated flow wiring (from
(from your code editor tabs)
your graph edges)
```

```

├─ custom/
│   └─ main_custom.py      ← your code (never overwritten on re-
export)
├─ tests/
│   └─ test_main.py        ← basic test scaffold
└─ main.py                 ← entry point – imports and runs the
flow

```

generated/main_flow.py translates your graph's edges into PocketFlow's >> wiring syntax:

```

from pocketflow import Flow
from generated.main_nodes import AskLlm, Start, Stop

ask_llm = AskLlm()
flow = Flow(start=ask_llm)
ask_llm >> ask_llm  # default action

```

You never edit generated/. You do edit custom/main_custom.py to add helper functions, configuration, or business logic that the nodes call.

Step-by-step

Step 1: Open a complete project.

Open the Tutorial 7 project (Hello World) or any validated project. The project must pass validation before export — Creator checks this automatically.

Step 2: Export.

File > Export PocketFlow Project... (Ctrl+E). A file dialog appears. Choose a parent directory. Creator creates the exports/<project_name>/ folder structure there.

Watch the progress bar in the status bar. When it completes, a toast notification confirms the export path.

Step 3: Inspect the exported files.

In Project Explorer, navigate to the export directory (or open a terminal). Open generated/main_nodes.py — you will recognise your prep, exec, and post code from the Creator editor, with the proper imports added. Open generated/main_flow.py — you will see the >> wiring derived from your graph edges.

Open custom/main_custom.py. This file is where you put call_llm implementations, API keys, and any helper code the nodes reference.

Step 4: Run outside Creator.

```
cd exports/hello_world
pip install pocketflow
python main.py
```

If your nodes call `call_llm`, implement it in `custom/main_custom.py` before running. A minimal implementation:

```
# custom/main_custom.py
import os
from openai import OpenAI

_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def call_llm(prompt: str) -> str:
    resp = _client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
    )
    return resp.choices[0].message.content
```

Step 5: Re-export after making changes.

Return to Creator. Change a node's code or add an edge. Re-export (Ctrl+E). Open `generated/main_nodes.py` — your changes are there. Open `custom/main_custom.py` — your `call_llm` implementation is unchanged. The separation is guaranteed by the exporter.

Tips: - Add your export directory to `.gitignore` if you do not want generated files in version control. - Or commit the export and use it as the deployable artifact — `generated/` is deterministic, so diffs are clean. - The `tests/` directory contains a scaffold. Expand it to test your business logic.

Tutorial 22: Shared Store Designer

What you'll learn: How to use the Shared Store Designer to define, document, and validate the schema of data flowing through your flow, and why schema documentation pays dividends in multi-node pipelines.

Why the Shared Store Designer exists

The shared store is a Python dictionary — at runtime, any node can write any key with any value. This flexibility is powerful but makes large flows hard to reason about. Which node writes `extracted_data`? Is `items` a list of strings or a list of dicts? What is the default value of `model`?

The Shared Store Designer answers these questions visually. It does not enforce types at runtime (Python’s dynamic nature makes that impractical without invasive changes), but it documents the contract so every developer knows what to expect. It also feeds Creator’s autocomplete when you type `shared[ " in the code editor.`

Understanding the columns

Column	Purpose
Namespace	Groups related keys (e.g., user, llm, data). Cosmetic only — no runtime effect.
Key	The exact string used in <code>shared["key"]</code>
Type	JSON Schema type: string, integer, number, boolean, array, object
Default	The value pre-loaded into the store before the flow starts. Leave blank for “not set”.

Step-by-step

Step 1: Open the Shared Store Designer.

Open any project. In Project Explorer, double-click **Shared Store**. The designer opens as a table with four columns and a toolbar above it.

Step 2: Add keys for a typical LLM flow.

Click the + button (or press Insert) to add a row. Fill in:

Namespace	Key	Type	Default
user	question	string	What is PocketFlow?
llm	answer	string	
llm	model	string	gpt-4o-mini
app	max_tokens	integer	512
data	history	array	

The `question` key has a default, so the flow can run without pre-populating the store. The `answer` key has no default — it must be written by a node before it can be read.

Step 3: Validate the schema.

Click **Validate** in the toolbar. Creator checks that: - All Type values are valid JSON Schema type names. - Namespace and Key fields are non-empty. - No two rows share the same Namespace + Key combination.

Fix any errors reported. Common mistakes: typing `str` instead of `string`, or `int` instead of `integer`.

Step 4: Save the schema.

Click **OK**. The schema is saved to `project.pfcproj.yaml` under a `store_schema` key. The file is human-readable YAML — you can inspect or edit it directly if needed.

Step 5: Use the schema during a run.

Run > Run Active Flow. When the flow is running, click the **Shared Store** tab. The tab shows all keys — both those in the schema and any unscheduled keys nodes write dynamically. Schema keys with defaults appear immediately; others appear when first written.

Step 6: Use the schema in the code editor.

Open any node in the code editor. When you type `shared[`, Creator shows an autocomplete dropdown listing all keys from the schema. This prevents typos (`shared["anwser"]` instead of `shared["answer"]`) that cause silent `KeyError` bugs.

Tips: - Use namespaces consistently: all LLM inputs/outputs under `llm`, all user data under `user`, intermediate processing data under `data`. This makes the schema easy to scan. - Define defaults for any key that must be set for the flow to run without manual intervention. A flow with sensible defaults can be run immediately after export without extra setup. - The schema is living documentation. Update it when you add or rename keys — stale schemas are worse than no schema because they mislead readers.

Tutorial 25: Packaging for Distribution

What you'll learn: How to use Creator's PyInstaller integration to build a single self-contained executable, what is included in the bundle, and how to distribute it to users who do not have Python installed.

Why packaging matters

Distributing a Python application normally requires the recipient to install Python, create a virtual environment, and install dependencies. For a developer

tool like Creator, that is acceptable. For an application you build with Creator — a custom AI assistant, a document processor, an automated workflow — most users will not do this.

PyInstaller bundles the Python interpreter, all dependencies, and your application into a single executable file. The user double-clicks it (or runs it from a terminal) and it just works. No Python required on the target machine.

What is included in the bundle

When you package Creator itself (or an exported Creator project), the bundle includes:

- All Python source files and compiled .pyc bytecode
- The PySide6 Qt libraries (for Creator's own UI)
- All help documents and tutorial Markdown files
- Translation .qm files for all supported locales
- Jinja2 templates used for code generation
- Your exported project's generated/ and custom/ directories (if packaging an export)

The resulting executable is large (typically 80–200 MB) because Qt libraries alone are substantial. This is normal and expected for Qt applications.

Step-by-step

Step 1: Install PyInstaller.

PyInstaller is not installed by default. Install it in your virtual environment:

```
pip install pyinstaller
```

Verify the installation:

```
pyinstaller --version
```

Step 2: Package for your current platform.

Creator ships with packaging scripts in scripts/:

For Linux:

```
bash scripts/package.sh linux
```

For macOS:

```
bash scripts/package.sh macos
```


For Windows (run in a Windows terminal or PowerShell):

```
bash scripts/package.sh windows
```

The script runs PyInstaller with the appropriate spec file. Output goes to `dist/`.

Step 3: Locate the output.

Platform Output

Linux `dist/pocketflow-creator` (single executable)

macOS `dist/PocketFlow Creator.app` (application bundle)

Windows `dist/pocketflow-creator.exe` (single executable)

Step 4: Test the executable.

Run the output directly from `dist/` without Creator running in the background:

```
# Linux / macOS
./dist/pocketflow-creator

# Windows
dist\pocketflow-creator.exe
```

Verify that the UI opens, a new project can be created, and the help browser works (it reads from the bundled Markdown files, not from your source tree).

Step 5: Distribute.

Copy the output file to the target machine. No installation is required. On macOS, users may need to right-click → Open the first time to bypass Gatekeeper.

Packaging an exported project (not Creator itself):

If you want to distribute the flow you built — not Creator — export it first (Tutorial 21), then write a minimal PyInstaller spec for the exported `main.py`. A typical spec:

```
# my_flow.spec
a = Analysis(
    ["main.py"],
    pathex=["exports/my_flow"],
    datas=[("custom", "custom"), ("generated", "generated")],
    hiddenimports=["pocketflow"],
)
pyz = PYZ(a.pure, a.zipped_data)
exe = EXE(pyz, a.scripts, a.binaries, a.datas, name="my_flow")
```

Build with:

```
pyinstaller my_flow.spec
```

Troubleshooting common packaging issues:

- **Missing module at runtime:** Add it to `hiddenimports` in the spec file.
 - **Missing data file:** Add it to `datas` as a `(src, dest)` tuple.
 - **File too large:** Use `--exclude-module` to drop unused heavy dependencies (e.g., `tkinter`).
 - **Permission denied on Linux:** Run `chmod +x dist/pocketflow-creator` after packaging.
-

[→ Continue to Part 4 — Exercises](#)

Part 4 — Creator System Exercises

These exercises are open-ended projects. Unlike the tutorials in Parts 1–3, they do not give you step-by-step instructions for every action. Instead, each exercise gives you an objective, explains what you are building and why it is interesting, identifies the skills and patterns you will need, and offers stretch goals that push further.

Use Parts 1–3 as your reference. When you get stuck, re-read the tutorial for the relevant pattern rather than guessing. The goal is to build fluency — the ability to reach for the right pattern without being told which one to use.

[← Tutorials Index](#)

Exercise A: Build a Complete News Summariser

What you are building

A flow that fetches news headlines, summarises each one with an LLM, ranks them by importance, and assembles a daily digest report saved to a Markdown file.

This is a representative real-world pipeline: external data in, LLM processing in the middle, formatted output at the end. It combines batch processing, conditional routing, and file output in a single project.

Why this is worth building

News summarisation is a canonical LLM use case, but building it end-to-end teaches something more important: how to design a pipeline where the data contract between nodes is explicit and the output format is predictable. A pipeline that produces clean, consistent output is reusable; one that produces ad-hoc strings is not.

You will also practice the Shared Store Designer — defining your schema before writing code forces you to think about what each node needs and what it produces before you write a single line.

Skills exercised

- Multi-stage workflow (Tutorial 10)
- Batch Node for per-item LLM calls (Tutorial 14)
- Conditional routing with Router Node (Tutorial 11)
- Shared Store Designer for schema definition (Tutorial 22)
- Custom node type wizard for the RSS reader (Tutorial 5)
- Export and standalone run (Tutorial 21)

Objective

Build a flow with the following graph:

```
[Start] → [Fetch Headlines] → [Summarise Each (BatchNode)]
                        → [Rank Headlines (Router)] —important→ [Format
Digest]
                                                    ↳routine→ [Filter Out]
→ [Format Digest]
    → [Format Digest] → [Save Report] → [Stop]
```

The digest should be a Markdown file written to `output/digest.md` containing: - A header with today's date - A section for important headlines with their summaries - A section for routine headlines (optional: omit entirely if you filter them out)

Step-by-step guidance

1. Design the schema first.

Open the Shared Store Designer before writing any code. Define at minimum:

Key	Type	Notes
headlines	array	

Key	Type	Notes
		List of headline strings fetched by Fetch Headlines
summaries	array	One summary string per headline, same order
important	array	Filtered list of (headline, summary) pairs rated important
routine	array	Filtered list of routine (headline, summary) pairs
digest	string	Final formatted Markdown report

Deciding these keys upfront means every node has a clear contract before you write its prep and post methods.

2. Create a custom node type for the RSS reader.

Use Node > New Custom Node Type. Give it: - Type ID: `rss_reader_node` - Category: Data/IO - Properties: `feed_url` (string, default: `https://feeds.bbc.co.uk/news/rss.xml`) - Actions: `default`

The wizard creates the YAML and a Python skeleton. Implement `exec` using the `feedparser` library (install with `pip install feedparser`). If you prefer not to use a real feed, return a static list of 5–10 headline strings.

3. Use BatchNode for summarisation.

Summarise Each should extend `BatchNode`. Its `prep` returns `shared["headlines"]`. Its `exec` receives one headline string and returns a one-sentence summary. Its `post` receives the list of all summaries and writes `shared["summaries"]`.

Write the prompt carefully: “Summarise this headline in one sentence without adding information not in the headline.” This constraint prevents the LLM from hallucinating context.

4. Route by importance.

Rank Headlines is a Router Node with actions `important` and `routine`. In `exec`, call the LLM with a prompt that classifies each (headline, summary) pair. A simple approach: classify all pairs in one call by asking for a JSON list of indices rated important.

5. Format and save.

Format Digest assembles the final Markdown string. Save Report writes it to output/digest.md (create the output/ directory if it does not exist).

6. Validate, run, export.

Validate the project (Ctrl+Shift+V). Run with the Mock Provider first to verify the graph structure — mock LLM responses will be placeholder strings, but you can confirm nodes execute in the right order and the Shared Store populates correctly. Then switch to a real provider and run again to see real summaries.

Export (Ctrl+E) and run standalone:

```
cd exports/news_summariser
pip install pocketflow feedparser
python main.py
```

Stretch goals

- Replace the static headline list with a real RSS library fetch. Use aiohttp in an AsyncNode (Tutorial 26) to fetch the feed without blocking.
 - Schedule the flow to run daily using a cron job or a simple Python scheduler.
 - Add a second Router branch for a third category: breaking (for urgent stories) that goes to a separate section in the digest with a visual flag.
-

Exercise B: Coding Agent with Memory

What you are building

An agent that accepts a programming task in plain English, writes a Python solution, runs a test suite against it, reads the test results, and iterates until the tests pass — or until a maximum iteration count is reached.

Why this is worth building

The coding agent exercises every advanced pattern at once: agentic looping, human-in-the-loop review, subflow composition for the repair cycle, and debug-mode inspection of shared state across iterations. It also forces you to think carefully about what the shared store holds across multiple rounds — one of the trickier aspects of stateful agent design.

Skills exercised

- Agentic loop with Router Node (Tutorial 12)
- Human-in-the-Loop for test review (Tutorial 15)
- Subflow composition for the fix cycle (Tutorial 20)

- Debug Mode with breakpoints (Tutorial 19)
- AsyncNode for running tests as a subprocess (Tutorial 26)

Objective

Build a flow with this high-level structure:

```
[Start] → [Get Task] → [Plan] → [Write Code] → [Run Tests]
      —pass—→ [Human Review] —approved—→ [Save Output] →
[Stop]
      —fail—→ [Fix Errors] → [Write Code] (loop)
[Human Review] —rejected—→ [Fix Errors] → [Write Code]
(loop)
```

The agent should: - Accept a task description (e.g., “Write a function that returns the Fibonacci sequence up to n”) - Plan the implementation in one LLM call - Write the Python code in a second LLM call - Run `pytest` against the written code - If tests fail, read the output and revise the code - If tests pass, present to a human reviewer before saving

Step-by-step guidance

1. Define the shared store schema.

Key decisions: - `task` (string): the original task description - `plan` (string): the LLM-generated plan - `code` (string): the current Python implementation - `test_output` (string): the raw `pytest` output - `test_passed` (boolean): whether the last test run passed - `iteration` (integer): how many code revision rounds have occurred - `max_iterations` (integer, default: 5): safety limit - `feedback` (string): human reviewer feedback

2. Build the fix loop as a subflow.

Create `graphs/fix_loop.pfcgraph.yaml`. This sub-graph contains: Start → Read Test Output → Diagnose Failure → Revise Code → Stop.

The `Diagnose Failure` node calls the LLM with the code, the test output, and the error message to produce a diagnosis. `Revise Code` calls the LLM again with the diagnosis to produce a revised implementation.

In the main graph, the `Fix Errors` node is a Subflow Node pointing to `fix_loop.pfcgraph.yaml`.

3. Run tests with a subprocess.

`Run Tests` writes `shared["code"]` to a temporary file, then runs `pytest` on it. Use `subprocess.run` in `exec` to capture `stdout` and `stderr`. Write the combined

output to `shared["test_output"]` and set `shared["test_passed"]` based on the return code.

4. Add the human review gate.

Human Review is a Human Review Node (Tutorial 15). It prints the current code and test results, asks for approval, and returns approved or rejected. If rejected, ask for feedback and write it to `shared["feedback"]`.

5. Add a maximum iteration guard.

In the Router Node that reads `test_passed` and `iteration`, add logic: if `iteration >= max_iterations`, route to the human review regardless of test status. Without this, a persistent test failure loops forever.

6. Debug with breakpoints.

Set breakpoints on `Write Code` and `Run Tests`. Run in Debug Mode (Shift+F5). After each code generation, inspect the Shared Store to verify code contains valid Python. After each test run, inspect `test_output` and `test_passed`.

Stretch goals

- Replace the simple test file write with a proper temp directory and a generated test scaffold based on the task description.
- Add a second LLM call before `Write Code` that converts the task into a test first (“test-driven” generation): write the test, then write code to pass it.
- Track `iteration` in the Shared Store Designer with a default of 0 and confirm it increments correctly by watching it in Debug Mode.

Exercise C: Multi-Provider LLM Router

What you are building

A flow that classifies each incoming request by complexity, then routes it to a different LLM provider: fast/cheap for simple requests, powerful/expensive for complex ones, with a fallback path for errors.

Why this is worth building

In production, not all LLM requests need the same model. Routing by complexity reduces cost significantly — simple factual lookups do not need a frontier model. This exercise teaches you to design a custom node type with configurable properties, wire a three-way Router, and test each path independently using the debug tools.

Skills exercised

- Custom node type wizard with properties and multiple actions (Tutorial 5)
- Conditional routing with three-way Router Node (Tutorial 11)
- Inspector properties for runtime configuration (Tutorial 3)
- Validation and error badges (Tutorial 18)
- Debug Mode to test each path (Tutorial 19)

Objective

Create a custom node type `llm_router_node` with these properties:

Property	Type	Default	Description
<code>fast_model</code>	string	<code>ollama/mistral</code>	Model for low-complexity requests
<code>smart_model</code>	string	<code>gpt-4o</code>	Model for high-complexity requests
<code>threshold</code>	integer	5	Complexity score above which to use the smart model

The node classifies the input with a cheap LLM call (or a heuristic) and returns one of three actions: `fast`, `smart`, or `fallback` (for errors or ambiguous cases).

Build a flow:

```
[Start] → [Classify Complexity (llm_router_node)]
      |
      |—fast—→ [Fast LLM] → [Merge Output] → [Stop]
      |
      |—smart—→ [Smart LLM] → [Merge Output]
      |
      |—fallback—→ [Fallback] → [Merge Output]
```

Step-by-step guidance

1. Create the custom node type.

Use Node > New Custom Node Type. The wizard generates a YAML file in `node_types/llm_router_node.yaml` and a Python skeleton. Fill in the three properties and three actions. Save and confirm the node appears in the palette under a “Routing” or custom category.

2. Implement the classification logic.

In the node's exec, score the input by length or call a fast local model to estimate complexity (1–10 scale). Return the action string based on the score vs. threshold.

A simple heuristic that avoids a classification LLM call entirely:

```
def exec(self, prep_res):
    question = prep_res
    score = min(10, len(question.split()) // 5 + 1)
    if score <= self.threshold:
        return "fast"
    return "smart"
```

This is appropriate for an exercise; in production you would use an LLM or a trained classifier.

3. Configure models in Inspector.

Drag the `llm_router_node` from the palette. In Inspector, set `fast_model`, `smart_model`, and `threshold` for this instance. This is what configurable properties are for: the logic is in the code, the values are in the Inspector.

4. Wire the three-way Router.

Wire all three output actions. Validate the project — confirm there are no PFCE2101 errors (undeclared actions). A common mistake is declaring `fast`, `smart`, `fallback` in the node type but forgetting to wire the `fallback` edge, leaving it unresolved.

5. Test each path in Debug Mode.

Set `user_question` in the Shared Store Designer to a short question (should route fast). Run in Debug Mode. Verify the route taken in the Run Log. Change the question to a complex multi-part query. Re-run. Verify `smart` is chosen.

Set a breakpoint on the Router Node. Inspect `shared["route"]` after the classification to confirm the score and decision are correct.

6. Export and test each path.

Export the project. Edit `custom/main_custom.py` to implement `call_llm` for both the fast and smart models (the fast model points to Ollama, the smart model to the OpenAI API or similar). Run `python main.py` and exercise each path.

Stretch goals

- Add a `classify_model` property to the router node so the classification call itself can use a configurable model separate from the fast and smart models.

- Add a fourth action: `cached` — if the exact question has been asked before (check a dict in shared state), return the cached answer without calling any model.
 - Implement a cost tracker: each path writes its estimated token cost to `shared["cost"]`, and a final aggregator node logs the total after each run.
-

Exercise D: Full IDE Workout

What you are building

This is a comprehensive end-to-end exercise that verifies you have mastered every major feature of Creator. There is no specific project to build — you work through a checklist of actions, each of which exercises a different part of the IDE. At the end, you should have touched every major feature at least once.

Why this matters

Individual tutorials teach patterns in isolation. This exercise forces you to move between panels, switch modes, and use features in combination — which is how real project work actually happens. It also surfaces any gaps: if a step is unfamiliar, go back to the relevant tutorial before continuing.

The workout

Complete every item on the list. Tick each one off as you finish it.

Project and Canvas

☐

Create a new project from the **Simple LLM Flow** template (File > New From Template)

☐

Add 3 additional nodes from the palette — use at least one Batch Node and one Router Node

☐

Edit node titles and Actions fields in the Object Inspector

☐

Arrange nodes manually and use View > Zoom to Fit (Ctrl+0) to tidy the canvas

☐

Draw an edge, then delete it and re-draw it with a different action label

☐

Select multiple nodes with Shift+Click and move them together

Custom Node Type

- ☐ Create one custom node type via the wizard (Node > New Custom Node Type)
- ☐ Give it at least two properties and two actions
- ☐ Drag an instance of your custom type from the palette onto the canvas
- ☐ Set one of its properties in the Inspector to a non-default value
- ☐ Verify the type appears correctly in the validation output (Project > Validate Project)

Code Editor

- ☐ Implement all node code using double-click → Python editor
- ☐ Use the `# --- NODE_START` and `# --- NODE_END` markers as the boundary for your code
- ☐ Delete a node class body from the code editor and confirm the node disappears from the canvas
- ☐ Re-add it by dragging from the palette (confirm it gets fresh `# --- NODE_START` markers)

Shared Store

- ☐ Define a shared store schema with at least 4 keys in the Shared Store Designer
- ☐ Include at least one key with a default value, one of type array, and one of type integer
- ☐ Use the Shared Store tab during a run to confirm your nodes write the correct values

Debug Mode

- ☐ Set a breakpoint on a non-trivial node (F9)
- ☐

- ☐ Run > Debug Active Flow (Shift+F5)
- ☐ Inspect the Shared Store while paused — confirm values match what you expect
- ☐ Resume execution (F5) and confirm it continues to the next breakpoint or to the end
- ☐ Remove all breakpoints and run normally to confirm the flow completes cleanly

Export and Standalone Run

- ☐ Export the project (File > Export PocketFlow Project..., Ctrl+E)
- ☐ Open the export directory and verify custom/, generated/, and main.py exist
- ☐ Implement `call_llm` in custom/main_custom.py
- ☐ Run `python main.py` from the export directory and confirm it executes without Creator

Settings and Localisation

- ☐ Switch from Dark to System (or Light) theme via Tools > Options
- ☐ Switch the UI language to Español and restart Creator
- ☐ Verify at least three menu items appear in Spanish
- ☐ Switch back to English and restart

What success looks like

You have completed the workout when every checkbox is ticked and the exported project runs correctly from the command line without any import errors or missing-file errors. If any step fails, treat it as a diagnostic: identify which tutorial covers that feature and review it before attempting the step again.

Stretch goals

- Repeat the workout, but this time build a genuinely useful project rather than a demonstration one — a tool you would actually use.
- Time yourself. The first run might take 90 minutes. A fluent Creator user completes this workout in under 20 minutes.
- Record which steps took the longest. Those are your weak spots — go back to the relevant tutorials and re-read them until the steps feel automatic.

[← Back to Tutorials Index](#)

Part 3: Advanced Topics

Hardware I/O Integration

Tutorial: Hardware I/O Nodes — Serial, Audio, Video, and Webcam

What you’ll learn: Use Hardware I/O nodes to interact with USB serial devices, microphones, speakers, cameras, and webcams.

Prerequisites: Tutorial 2 (Your First Flow), Tutorial 22 (Standalone Scripts)

Hardware I/O Node Categories

PocketFlow Creator includes 7 Hardware I/O node types:

Node Type	Purpose	Requires
USB Serial Input	Read data from serial port (e.g., Arduino, sensor)	pyserial
USB Serial Output	Send data to serial port	pyserial
Audio Input	Record audio from microphone	sounddevice, soundfile
Audio Output	Play audio from speaker	sounddevice, soundfile
Video Input	Record video from camera	opencv-python
Video Output	Play video file	opencv-python

Node Type	Purpose	Requires
Webcam	Capture frame or stream from webcam	opencv-python

Example 1: Read from USB Serial (Arduino)

Scenario

You have an Arduino connected via USB that reports temperature readings every second.

Setup

1. **New Project** → `arduino_monitor`
2. **Create flow** → `read_temperature.pfcgraph.yaml`
3. **Add nodes:**
 - **Start** node
 - **USB Serial Input** node (center)
 - **Log** node (right)
 - **Stop** node (far right)
4. **Wire edges:**
 - Start → USB Serial Input
 - USB Serial Input → Log
 - Log → Stop
5. **Configure USB Serial Input** (Object Inspector):
 - port: `/dev/ttyUSB0` (or COM3 on Windows)
 - baudrate: 9600
 - output_key: `temperature_raw`
6. **Configure Log** node:
 - input_key: `temperature_raw`
 - log_level: `info`

Run

```
python exports/arduino_monitor/standalone/  
read_temperature_standalone.py
```

```
# Output:  
# INFO in Log Node: 23.5C
```

Example 2: Record Audio from Microphone

Scenario

Record a 5-second audio clip, save it to a file, then play it back.

Setup

1. **New Project** → voice_recorder
2. **Create flow** → record_and_play.pfcgraph.yaml
3. **Add nodes:**
 - **Start** node
 - **Audio Input** node (left)
 - **Audio Output** node (right)
 - **Stop** node (far right)
4. **Wire edges:**
 - Start → Audio Input
 - Audio Input → Audio Output
 - Audio Output → Stop
5. **Configure Audio Input** (Object Inspector):
 - duration: 5.0 (seconds)
 - sample_rate: 16000 (Hz)
 - output_file: recording.wav
 - output_key: audio_file
6. **Configure Audio Output:**
 - input_key: audio_file
 - output_key: status

Run

```
python exports/voice_recorder/standalone/  
record_and_play_standalone.py
```

```
# Records 5 seconds from microphone → saves to recording.wav  
# Then plays recording.wav back through speaker
```

Example 3: Capture Webcam and Send to LLM

Scenario

Take a screenshot from webcam, send it to Claude for image analysis.

Setup

1. **New Project** → webcam_analyzer
2. **Create flow** → analyze_webcam.pfcgraph.yaml
3. **Add nodes:**
 - **Start** node
 - **Webcam** node (left)
 - **Image Vision** node (center) — requires Anthropic provider
 - **Log** node (right)
 - **Stop** node (far right)
4. **Wire edges:**
 - Start → Webcam
 - Webcam → Image Vision
 - Image Vision → Log
 - Log → Stop
5. **Configure Webcam** (Object Inspector):
 - operation: capture (single frame)
 - output_file: frame.jpg
 - output_key: image_path
6. **Configure Image Vision:**
 - image_path_key: image_path
 - task: describe what you see
 - output_key: description
 - Set provider to Anthropic (Tools > Provider Manager)
7. **Configure Log:**
 - input_key: description

Run

```
export ANTHROPIC_API_KEY="sk-ant-..."
python exports/webcam_analyzer/standalone/
analyze_webcam_standalone.py

# Output:
# INFO in Log Node: "I can see a desk with a laptop, keyboard,
and monitor..."
```

Example 4: Video Recording with Duration Limit

Scenario

Record 10 seconds of video from the camera.

Setup

1. **New Project** → video_capture
2. **Create flow** → record_video.pfcgraph.yaml
3. **Add nodes:**
 - **Start** node
 - **Video Input** node (center)
 - **Stop** node (right)
4. **Wire:**
 - Start → Video Input → Stop
5. **Configure Video Input** (Object Inspector):
 - duration: 10.0 (seconds)
 - output_file: output.mp4
 - output_key: video_path

Run

```
python exports/video_capture/standalone/  
record_video_standalone.py
```

```
# Records 10 seconds of video from camera to output.mp4
```

Example 5: Serial Communication Loop (Arduino ↔ Creator)

Scenario

Continuously read sensor data from Arduino, process it, and send commands back.

Setup

1. **New Project** → bidirectional_serial
2. **Create flow** → arduino_control.pfcgraph.yaml
3. **Add nodes:**
 - **Start** node
 - **USB Serial Input** node (reads sensor)
 - **Condition** node (checks if temp > threshold)
 - **USB Serial Output** node (sends control command)
 - **Log** node
 - **Stop** node
4. **Wire edges:**
 - Start → USB Serial Input

- USB Serial Input → Condition (route: default)
- Condition → USB Serial Output (action: true)
- USB Serial Output → Log
- Log → Stop

5. **Configure USB Serial Input:**

- port: /dev/ttyUSB0
- baudrate: 9600
- output_key: sensor_reading

6. **Configure Condition:**

- input_key: sensor_reading
- condition: float(input) > 25.0 (route to true if temp > 25°C)

7. **Configure USB Serial Output:**

- input_key: command (value to send)
- output_key: status

Run

```
python exports/bidirectional_serial/standalone/
arduino_control_standalone.py
```

```
# Reads: "23.5"
# Condition: False (≤ 25)
#
# Reads: "26.2"
# Condition: True (> 25)
# Sends: "FAN_ON" to Arduino
```

Installation of Optional Dependencies

By default, Creator includes only stdlib imports. Hardware nodes require optional libraries:

For USB Serial

```
pip install pyserial
```

For Audio (Input/Output)

```
pip install sounddevice soundfile
```

For Video (Input/Output, Webcam)

```
pip install opencv-python
```

Once installed, Hardware I/O nodes work seamlessly in standalone scripts.

Properties Reference

USB Serial Input

Property	Type	Default	Description
port	string	/dev/ttyUSB0	Serial port path (e.g., /dev/ttyUSB0, COM3)
baudrate	integer	9600	Baud rate (9600, 115200, etc.)
output_key	string	data	Shared store key for received data

Actions: default (success), timeout (no data within 5 sec)

USB Serial Output

Property	Type	Default	Description
port	string	/dev/ttyUSB0	Serial port path
baudrate	integer	9600	Baud rate
input_key	string	data	Shared store key for data to send
output_key	string	status	Shared store key for status

Actions: default (success), error

Audio Input

Property	Type	Default	Description
duration	float	5.0	Recording duration (seconds)
sample_rate	integer	16000	Sample rate (Hz)
output_file	string	recording.wav	Output WAV file path
output_key	string	audio_file	Shared store key for file path

Actions: default (success), error

Audio Output

Property	Type	Default	Description
input_key	string	audio_file	Shared store key for audio file path
output_key	string	status	Shared store key for status

Actions: default (success), error

Video Input

Property	Type	Default	Description
duration	float	5.0	Recording duration (seconds)
output_file	string	recording.mp4	Output MP4 file path
output_key	string	video_file	Shared store key for file path

Actions: default (success), error

Video Output

Property	Type	Default	Description
input_key	string	video_file	Shared store key for video file path
output_key	string	status	Shared store key for status

Actions: default (success), error

Webcam

Property	Type	Default	Description
operation	string	capture	capture (single frame) or stream (multiple frames)
frame_count	integer	30	Frames to capture in stream mode
output_file	string	frame.jpg	Output file path for captured frame
output_key	string	image	Shared store key for output

Actions: default (success), error

Tips and Best Practices

Serial Communication

- Always check the correct port: `ls /dev/tty*` (Mac/Linux) or Device Manager (Windows)
- Verify baud rate matches your device (usually 9600 or 115200)
- Data is read as a string; parse with **String Ops** or **Regex** node

Audio

- Microphone input streams to WAV file (lossless)
- Specify duration carefully — recording will block until complete
- Test microphone permissions (sudo may be needed on Linux)

Video

- Video recording uses your primary camera device
- Recording duration ties up the camera — no concurrent use
- MP4 codec assumes OpenCV ffmpeg support

Webcam

- Capture mode returns a single frame (JPEG)
 - Stream mode returns N frames as a list (useful for motion detection)
 - Check camera permissions in System Preferences (macOS) or Privacy Settings (Windows)
-

Troubleshooting

Issue	Solution
<code>ModuleNotFoundError: No module named 'serial'</code>	<code>pip install pyserial</code>
Serial port not found	Check device connection; verify port path with <code>ls /dev/tty*</code>
No permission to access /dev/ttyUSB0	<code>sudo usermod -a -G dialout \$USER</code> (Linux) or run with <code>sudo</code>
Microphone permission denied	Check Privacy > Microphone (macOS) or Sound settings (Windows)

Issue	Solution
Camera busy or not found	Close other apps using camera; check device manager

Next Steps

- Combine Hardware I/O with **LLM** nodes for intelligent sensor processing
 - Use **API Call** node to upload captured images to cloud services
 - Create flows that bridge IoT devices and AI services
 - Deploy as standalone scripts to Raspberry Pi or other edge devices
-

Standalone Python Scripts

Tutorial: Exporting Standalone Python Scripts

What you'll learn: Export a PocketFlow graph as a self-contained Python script with no framework dependencies.

Prerequisites: Tutorial 1 (IDE Tour), Tutorial 2 (Your First Flow)

Why Standalone Scripts?

PocketFlow Creator normally exports a Python package with template stubs. But sometimes you need: - **A single .py file** that runs the flow independently - **No PocketFlow framework** required to run - **No pip install needed** beyond stdlib (optional external libs detected at runtime) - **Embedded provider implementations** (Ollama, OpenAI, Anthropic, Gemini, DeepSeek)

When you export a standalone script, you get exactly that.

Step 1: Create a Simple Flow

1. **New Project** → File > New Project...
 - Name: sentiment_analyzer
 - Create a graph called analyze_sentiment.pfcgraph.yaml
2. **Add nodes** to the canvas:
 - **Start Node** → position at left

- **LLM Prompt Node** → center
 - **Stop Node** → right
 - 3. **Wire edges**
 - Start → LLM Prompt (action: default)
 - LLM Prompt → Stop (action: default)
 - 4. **Configure LLM Prompt Node** (in Object Inspector):
 - prompt_file: Analyze this text and rate sentiment 1–10: {{input}}
 - output_key: result
 - input_key: input
 - 5. **Configure provider**
 - Tools > Provider Manager
 - Add an Ollama provider (or OpenAI, Anthropic, etc.)
 - Set as default or wire to the LLM node
-

Step 2: Export as Standalone Script

1. File > Export PocketFlow Project

- Location: any folder
- Click **Export**

2. Check the export folder

```
exports/sentiment_analyzer/  
├── generated/                # Framework-based Python package  
│   ├── analyze_sentiment_nodes.py  
│   ├── analyze_sentiment_flow.py  
│   └── __init__.py  
├── custom/  
├── tests/  
├── standalone/              # ← NEW: Self-contained scripts  
│   ├── analyze_sentiment_standalone.py  
│   └── __init__.py  
└── main.py
```

3. Open the standalone script

```
cat exports/sentiment_analyzer/standalone/  
analyze_sentiment_standalone.py
```

Step 3: What's Inside the Standalone Script

The generated script contains:

1. Imports (stdlib only)

```
import copy
import json
import os
import re
import sys
import time
import urllib.error
import urllib.parse
import urllib.request
from pathlib import Path
```

2. Embedded Provider Classes

— Provider Classes

```
class OllamaProvider:
    """Ollama-compatible LLM provider (full implementation
included)."""
    def __init__(self, base_url, default_model, timeout):
        ...

    def complete(self, prompt, model=None):
        ...

class OpenAIProvider:
    """OpenAI-compatible LLM provider (full implementation
included)."""
    ...
```

3. Provider Instances

— Provider Instances

```
_provider_local_ollama = OllamaProvider(
    base_url="http://localhost:11434",
    default_model="qwen2.5-coder:14b",
    timeout=120,
)

# Cloud provider keys loaded from environment:
# export OPENAI_API_KEY="sk-..."
_provider_openai = OpenAIProvider(
    api_key=os.environ.get("OPENAI_API_KEY", ""),
    base_url="https://api.openai.com/v1",
```



```

        default_model="gpt-4o-mini",
        timeout=120,
    )

```

4. Graph Definition

— Graph data

```

_START = "start-node-1"

_NODES = {
    "start-node-1": {
        "type": "start_node",
        "title": "Start",
        "props": {},
        "provider": None,
    },
    "llm-node-1": {
        "type": "llm_prompt_node",
        "title": "Analyze Sentiment",
        "props": {
            "prompt_file": "Analyze sentiment...",
            "output_key": "result",
        },
        "provider": _provider_local_ollama,
    },
    ...
}

_EDGES = [
    {"from": "start-node-1", "action": "default", "to": "llm-
node-1"},
    {"from": "llm-node-1", "action": "default", "to": "stop-
node-1"},
]

```

5. Node Dispatch (execution logic)

— Node Dispatch

```

def _run_node(node_id, node, shared, outgoing_actions):
    """Execute the specified node; return the action to take
    next."""
    node_type = node["type"]

    if node_type == "start_node":

```

```

        return "default"

    elif node_type == "llm_prompt_node":
        provider = node["provider"]
        prompt = node["props"]["prompt_file"]
        # Interpolate variables from shared store
        for key, val in shared.items():
            prompt = prompt.replace(f"{{{key}}}", str(val))
        result = provider.complete(prompt)
        shared[node["props"]["output_key"]] = result
        return "default"

    elif node_type == "stop_node":
        return ""

```

6. Flow Runner

— Flow Runner

```

def run_flow(shared=None):
    """Execute the flow and return the final shared store
    state."""
    shared = dict(shared or {})

    edge_map = {}
    for edge in _EDGES:
        if edge["from"] not in edge_map:
            edge_map[edge["from"]] = []
        edge_map[edge["from"]].append(edge)

    current_node_id = _START
    visited = 0

    while current_node_id and visited < 200:
        node = _NODES.get(current_node_id)
        if not node:
            break
        visited += 1

        outgoing = edge_map.get(current_node_id, [])
        chosen_action = _run_node(current_node_id, node, shared,
outgoing)

        next_edge = next(
            (e for e in outgoing if e["action"] ==
chosen_action),
            outgoing[0] if outgoing else None,

```

```

    )
    current_node_id = next_edge["to"] if next_edge else ""

    return shared

if __name__ == "__main__":
    result = run_flow({"input": "The food was terrible."})
    print(json.dumps(result, indent=2, default=str))

```

Step 4: Run the Standalone Script

Scenario A: Local Ollama

```

# With Ollama running locally on port 11434
python exports/sentiment_analyzer/standalone/
analyze_sentiment_standalone.py

# Output:
# {
#   "input": "The food was terrible.",
#   "result": "Sentiment: 2/10 (very negative)"
# }

```

Scenario B: Cloud Provider

```

# With OpenAI API key
export OPENAI_API_KEY="sk-..."
python exports/sentiment_analyzer/standalone/
analyze_sentiment_standalone.py

# Output:
# {
#   "input": "The food was terrible.",
#   "result": "Sentiment Rating: 1/10\nAnalysis: Strongly
negative..."
# }

```

Scenario C: Modify Shared Store at Runtime

```

# In Python
from sentiment_analyzer_standalone import run_flow

result = run_flow({
    "input": "This product exceeded all expectations!"
})

```

```
}  
print(result["result"]) # "Sentiment: 9/10 (very positive)"
```

Step 4b: Interactive Nodes (Human Input / Review)

If your flow includes **Human Input Node** or **Human Review Node**, the standalone script uses stdin/stdout/stderr:

Human Input Node Example

```
# Prompts on stdout, reads from stdin  
python script.py  
[Human Input Node] Enter your name:  
> Alice  
  
# Piped input  
echo "Alice" | python script.py  
  
# From file  
python script.py < names.txt > results.txt
```

Human Review Node Example

```
# Displays content on stdout, reads approval from stdin  
python script.py  
[Human Review Node] Review this:  
The quick brown fox jumps over the lazy dog  
Approve? [y/n]: y  
  
# Non-interactive (handles EOF gracefully)  
python script.py < /dev/null # No crash, sets action to  
'rejected'
```

I/O Redirection in CI/CD

```
# Jenkins/GitHub Actions/GitLab CI  
python script.py < input.txt > output.json 2> errors.log  
  
# Shell script  
#!/bin/bash  
cat << EOF | python script.py > results.json  
approval_input  
EOF  
  
# With timeout  
timeout 30 python script.py < input.txt
```

Key Points: - **stdin** for user input (Human Input Node, Human Review Node) - **stdout** for prompts and flow output - **stderr** for errors (automatically captured) - **EOF handling** — gracefully sets action to “cancelled” if input EOF reached - **Works in pipelines** — no GUI dialogs, pure text I/O

Step 5: Error Handling for Missing Dependencies

Some nodes require external libraries. The standalone script checks for them at runtime:

```
# If BeautifulSoup4 is missing:
$ python script.py
```

ERROR **in** Web Scrape Node: web_scrape_node requires BeautifulSoup4: pip install BeautifulSoup4

You can then install and re-run:

```
pip install BeautifulSoup4
python script.py
```

Supported Features in Standalone Scripts

Category	Nodes	Requires
Core Data	map, reduce, merge, transform, condition, loop, json_parse, list_ops, string_ops, log, timer, cache	stdlib only
	llm_prompt, json_llm, rag, chain_of_thought, majority_vote, debate, etc.	
LLM / Reasoning	web_search, web_scrape, api_call, pdf_extract, spreadsheet	LLM provider
External APIs		Optional: BeautifulSoup4, PyPDF2, openpyxl
Database		sqlite3 (built-in)

Category	Nodes	Requires
	db_schema, nl_to_sql, sql_execute	
Memory	registry, stack, queue, local_memory, secret	stdlib only
Hardware I/O	usb_serial, audio_input, audio_output, video_input, video_output, webcam	Optional: pyserial, sounddevice, opencv-python
Communication	socket, websocket, email, calendar, a2a	Optional: websockets, google- auth-oauthlib

Limitations

Standalone scripts run flows **line-by-line without the Creator UI**: - Node dispatch and execution - Shared store state management - LLM provider calls - File I/O, API calls, etc.

But they cannot: - Display the flow visually - Show real-time step-through debugging - Inspect shared store in Object Inspector - Modify the graph interactively

For those, use **Run Active Flow** or **Debug Active Flow** in Creator instead.

Next Steps

- Try exporting different flows as standalone scripts
- Modify the shared dict to test different inputs
- Deploy standalone scripts to CI/CD pipelines or serverless platforms
- See [Tutorial: Hardware I/O](#) for sensor and device integration

Part 4: Reference

All Node Types

Node Types Reference — Complete Catalog

76 built-in node types organized by category. Each entry shows what the node does, its properties, and example usage.

Quick Navigation

Category	Count
Flow Control	5
AI / LLM	6
AI / Reasoning	5
Web / Search	3
Data / Vector	4
Database / SQL	3
Voice / Audio	2
Hardware I/O	7
Document / Vision	3
Code / Execution	3
Data Processing	6
Calendar	2
MCP / Agent Protocol	3
Observability / Utility	4
Data Structures / Memory	6
Security	1
Human-in-the-Loop	2
Batch / Async	5
I/O	3
System / Shell	3
Networking	3
Text / Data Processing	5

Category	Count
Resilience	2
Messaging	3
TOTAL	76

Flow Control

Start Node

Purpose: Entry point of a flow. Always executes first. - **Actions:** default - **Properties:** None - **Shared Store:** None - **Example:** Start → [other nodes]

Stop Node

Purpose: Exit point of a flow. Flow terminates here. - **Actions:** None - **Properties:** None - **Shared Store:** None

Basic Node

Purpose: No-op passthrough node. Used for organization or as a placeholder. - **Actions:** default - **Properties:** None - **Shared Store:** None

Router Node

Purpose: Routes to different actions based on shared store value. - **Actions:** Configurable (comma-separated) - **Properties:** - routes: string — action names (e.g., "yes , no , maybe") - **Shared Store:** Read from shared store to determine which route

Subflow Node

Purpose: Execute a nested graph referenced by ID. - **Actions:** Configurable per subflow - **Properties:** - subflow_id: string — ID of the graph to execute - **Shared Store:** Passes through to subflow; propagates results back

AI / LLM

LLM Prompt Node

Purpose: Send a prompt to an LLM and receive text completion. - **Actions:** default - **Properties:** - prompt_file: string — prompt text or path to .md file - prompt_type: string — "string" or "path" - input_key: string — read input

from this shared store key - output_key: string — write result to this key - model: string — LLM model name (blank = project default) - temperature: float — 0–1 (lower = more deterministic) - max_tokens: integer — max response length - system_prompt: string — system context override - **Shared Store:** Reads input_key, writes output_key - **Example:** Sentiment analysis, summarization, translation

JSON LLM Node

Purpose: Prompt LLM with JSON schema; parse response as JSON. - **Actions:** default (valid JSON), invalid (parse error) - **Properties:** Similar to LLM Prompt Node - **Shared Store:** Writes parsed JSON object to output_key

Classifier Node

Purpose: Classify input into predefined categories using LLM. - **Actions:** Dynamic (one per category) - **Properties:** - prompt_file: string — classification prompt - categories: string — comma-separated category names - input_key: string — text to classify - output_key: string — winning category - **Shared Store:** Reads input, writes winning category - **Example:** Spam detection, sentiment, topic classification

Judge Node

Purpose: LLM evaluates text and routes based on criteria (e.g., “pass” vs. “fail”). - **Actions:** Configurable (typically pass, fail) - **Properties:** - prompt_file: string — evaluation prompt - input_key: string — content to judge - output_key: string — judgment result - **Shared Store:** Reads input, writes judgment

Agent Node

Purpose: Multi-turn LLM agent that can reason or take actions. - **Actions:** continue (agent wants more steps), done (agent finished) - **Properties:** - input_key: string — user message or task - output_key: string — final agent result - model: string — LLM model - max_iterations: integer — max reasoning steps (default 10) - **Shared Store:** Reads input, writes result

RAG Node

Purpose: Retrieval-Augmented Generation — fetch context from vector DB, pass to LLM. - **Actions:** default, no_context - **Properties:** - query_key: string — user question - vector_index_key: string — vector index reference - context_key: string — write retrieved context - output_key: string — write LLM response - **Shared Store:** Reads query, writes context and response

AI / Reasoning

Chain of Thought Node

Purpose: Multi-step reasoning with explicit step tracking. - **Actions:** default - **Properties:** - problem_key: string — input problem - thoughts_key: string — write accumulated thoughts - solution_key: string — write final answer - max_steps: integer — max reasoning steps - model: string — LLM model - **Shared Store:** Reads problem, writes thoughts and solution

Majority Vote Node

Purpose: Call LLM N times, pick most common answer. - **Actions:** default - **Properties:** - prompt_key: string — prompt text - num_votes: integer — number of independent calls - output_key: string — winning answer - model: string — LLM model - **Shared Store:** Reads prompt, writes majority result

Supervisor Node

Purpose: Evaluate output against criteria; route to approval or retry. - **Actions:** approved, retry, rejected - **Properties:** - input_key: string — work to review - criteria: string — evaluation criteria - output_key: string — feedback - model: string — LLM model (for evaluation) - **Shared Store:** Reads input, writes feedback

Debate Advocate Node

Purpose: Generate argument for or against a claim. - **Actions:** default - **Properties:** - topic_key: string — claim to argue about - position: string — "for" or "against" - output_key: string — generated argument - model: string — LLM model - **Shared Store:** Reads topic, writes argument

Debate Judge Node

Purpose: Compare two arguments and select the stronger one. - **Actions:** argument_a, argument_b, tie - **Properties:** - argument_a_key: string — first argument - argument_b_key: string — second argument - output_key: string — verdict - model: string — LLM model - **Shared Store:** Reads arguments, writes verdict

Web / Search

Web Search Node

Purpose: Query search engine (Brave, Google) and retrieve results. - **Actions:** default (success), error - **Properties:** - query_key: string — search query - output_key: string — result list - **Shared Store:** Reads query, writes results - **Requires:** SEARCH_API_KEY environment variable

Web Scrape Node

Purpose: Fetch URL and extract text content with BeautifulSoup. - **Actions:** default (success), error - **Properties:** - url_key: string — URL to scrape - output_key: string — extracted text - **Shared Store:** Reads URL, writes text (first 5000 chars) - **Requires:** beautifulsoup4 pip package

API Call Node

Purpose: Make HTTP request (GET/POST/etc.) and parse JSON response. - **Actions:** default (success), error - **Properties:** - endpoint: string — URL - method: string — "GET", "POST", etc. - input_key: string — payload (for POST) - output_key: string — response JSON - headers: dict — optional HTTP headers - **Shared Store:** Reads payload, writes response

Data / Vector

Text Chunk Node

Purpose: Split long text into overlapping chunks. - **Actions:** default - **Properties:** - input_key: string — text to chunk - output_key: string — list of chunks - chunk_size: integer — chars per chunk (default 1000) - overlap: integer — overlap between chunks - **Shared Store:** Reads text, writes chunk list

Embed Node

Purpose: Convert text to embeddings vector using LLM provider. - **Actions:** default - **Properties:** - input_key: string — text to embed - output_key: string — embedding vector - **Shared Store:** Reads text, writes vector - **Requires:** LLM provider with embedding support

Vector Index Node

Purpose: Build searchable index from vectors. - **Actions:** default - **Properties:** - vectors_key: string — input vectors - output_key: string — index reference - index_type: string — "simple", "faiss", etc. - **Shared Store:** Reads vectors, writes index

Vector Retrieve Node

Purpose: Search index for similar vectors (cosine similarity). - **Actions:** default (found), not_found - **Properties:** - index_key: string — vector index - query_key: string — query vector - output_key: string — top-K results - top_k: integer — max results (default 5) - **Shared Store:** Reads index and query, writes results

Database / SQL

DB Schema Node

Purpose: Extract schema from SQLite database. - **Actions:** default (success), error - **Properties:** - db_path: string — path to .db file - output_key: string — schema as dict - **Shared Store:** Writes schema - **Requires:** sqlite3 (built-in)

NL to SQL Node

Purpose: Convert natural language question to SQL using LLM. - **Actions:** default (success), error - **Properties:** - query_key: string — natural language question - schema_key: string — database schema - output_key: string — generated SQL - model: string — LLM model - **Shared Store:** Reads query and schema, writes SQL

SQL Execute Node

Purpose: Execute SQL query on SQLite database. - **Actions:** default (success), error - **Properties:** - db_path: string — database file - sql_key: string — SQL command - output_key: string — results (list of dicts) - **Shared Store:** Reads SQL, writes results - **Requires:** sqlite3 (built-in)

Voice / Audio

Speech to Text Node

Purpose: Convert audio file to text using Google Speech API. - **Actions:** default (success), error - **Properties:** - audio_file_key: string — path to WAV/MP3 - output_key: string — transcribed text - **Shared Store:** Reads file path, writes text - **Requires:** SpeechRecognition pip package

Text to Speech Node

Purpose: Generate audio file from text. - **Actions:** default (success), error - **Properties:** - input_key: string — text to speak - output_file: string — output MP3 path - output_key: string — file path written to shared store - **Shared Store:** Reads text, writes file path - **Requires:** pyttsx3 pip package

Hardware I/O

USB Serial Input

Purpose: Read data from serial port (Arduino, sensors, etc.). - **Actions:** default (success), timeout - **Properties:** - port: string — /dev/ttyUSB0 or COM3 - baudrate: integer — 9600, 115200, etc. - output_key: string — received data - **Shared Store:** Writes received data - **Requires:** pyserial pip package

USB Serial Output

Purpose: Write data to serial port. - **Actions:** default (success), error - **Properties:** - port: string — serial port path - baudrate: integer — baud rate - input_key: string — data to send - output_key: string — status - **Shared Store:** Reads data to send, writes status - **Requires:** pyserial pip package

Audio Input

Purpose: Record audio from microphone. - **Actions:** default (success), error - **Properties:** - duration: float — seconds to record - sample_rate: integer — Hz (default 16000) - output_file: string — output WAV path - output_key: string — file path - **Shared Store:** Writes file path - **Requires:** sounddevice, soundfile pip packages

Audio Output

Purpose: Play audio file through speaker. - **Actions:** default (success), error -

Properties: - input_key: string — audio file path - output_key: string — status -

Shared Store: Reads file path, writes status - **Requires:** sounddevice, soundfile
pip packages

Video Input

Purpose: Record video from camera. - **Actions:** default (success), error -

Properties: - duration: float — seconds to record - output_file: string —

output MP4 path - output_key: string — file path - **Shared Store:** Writes file path
- **Requires:** opencv-python pip package

Video Output

Purpose: Play video file. - **Actions:** default (success), error - **Properties:** -

input_key: string — video file path - output_key: string — status - **Shared**

Store: Reads file path, writes status - **Requires:** opencv-python pip package

Webcam

Purpose: Capture frame or stream from webcam. - **Actions:** default (success),

error - **Properties:** - operation: string — "capture" (single) or "stream"

(multiple) - frame_count: integer — frames to capture - output_file: string —

output file path - output_key: string — image/frames - **Shared Store:** Writes

frame or frame list - **Requires:** opencv-python pip package

Document / Vision

PDF Extract Node

Purpose: Extract text from PDF file. - **Actions:** default (success), error -

Properties: - file_path_key: string — PDF file path - output_key: string —

extracted text - **Shared Store:** Reads file path, writes text - **Requires:** PyPDF2 pip
package

Image Vision Node

Purpose: Analyze image with LLM vision capability. - **Actions:** default (success),

error - **Properties:** - image_path_key: string — image file path - task: string —

what to do (e.g., "describe") - output_key: string — analysis result - **Shared**

Store: Reads file path, writes analysis - **Requires:** LLM provider with vision support

Data Validate Node

Purpose: Check if data matches expected type or schema. - **Actions:** default (valid), invalid - **Properties:** - input_key: string — data to validate - output_key: string — boolean (valid/invalid) - validation_type: string — "type", "schema", etc. - expected_type: string — "str", "dict", "list", etc. - **Shared Store:** Reads data, writes validation result

Code / Execution

Code Gen Node

Purpose: Generate code using LLM. - **Actions:** default (success), error - **Properties:** - spec_key: string — code specification - output_key: string — generated code - language: string — "python", "javascript", etc. - model: string — LLM model - **Shared Store:** Reads spec, writes code

Code Exec Node

Purpose: Execute Python code dynamically. - **Actions:** default (success), error - **Properties:** - code: string — Python code to execute - output_key: string — execution result (stdout) - **Shared Store:** Available as shared variable in code; writes result

Test Gen Node

Purpose: Generate test cases for code using LLM. - **Actions:** default (success), error - **Properties:** - code_key: string — code to test - output_key: string — generated tests - test_framework: string — "pytest", "unittest", etc. - model: string — LLM model - **Shared Store:** Reads code, writes tests

Data Processing

Map Node

Purpose: Apply transformation to each item in a list. - **Actions:** default - **Properties:** - input_key: string — list to map - output_key: string — transformed list - operation: string — operation name - **Shared Store:** Reads list, writes result

Reduce Node

Purpose: Accumulate list to single value. - **Actions:** default - **Properties:** - input_key: string — list to reduce - output_key: string — final value - operation: string — reduce operation - initial_value: any — starting value - **Shared Store:** Reads list, writes result

Condition Node

Purpose: Branch flow based on boolean condition. - **Actions:** true, false - **Properties:** - input_key: string — value to test - condition: string — Python expression - **Shared Store:** Reads value, evaluates condition

Loop Counter Node

Purpose: Repeat a sub-flow N times with counter. - **Actions:** continue (next iteration), done - **Properties:** - max_iterations: integer — times to repeat - counter_key: string — write current counter - output_key: string — accumulated results - **Shared Store:** Writes counter and results

Transform Node

Purpose: Convert data structure (dict ↔ list, etc.). - **Actions:** default - **Properties:** - input_key: string — data to transform - output_key: string — transformed data - operation: string — transformation type - **Shared Store:** Reads input, writes output

Merge Node

Purpose: Combine multiple lists or dicts into one. - **Actions:** default - **Properties:** - input_keys: list — keys to merge - output_key: string — merged result - merge_type: string — "list" or "dict" - **Shared Store:** Reads inputs, writes merged result

Calendar

Calendar Read Node

Purpose: Fetch events from Google Calendar. - **Actions:** default (success), error - **Properties:** - calendar_id: string — Google Calendar ID - output_key: string — events list - max_events: integer — max to fetch (default 10) - **Shared Store:** Writes events - **Requires:** GOOGLE_CALENDAR_ID env var, Google Auth libs

Calendar Write Node

Purpose: Create event on Google Calendar. - **Actions:** default (success), error - **Properties:** - calendar_id: string — Google Calendar ID - event_key: string — event data (dict) - output_key: string — created event ID - **Shared Store:** Reads event, writes event ID - **Requires:** Google Auth libs

MCP / Agent Protocol

MCP Tool Node

Purpose: Call tool via Model Context Protocol (MCP) server. - **Actions:** default (success), error - **Properties:** - tool_name: string — tool to invoke - input_key: string — tool arguments - output_key: string — tool result - **Shared Store:** Reads args, writes result - **Requires:** MCP server running, MCP_SERVER_URL env var

A2A Send Node

Purpose: Send message to another agent. - **Actions:** default - **Properties:** - recipient_key: string — recipient ID - message_key: string — message to send - output_key: string — status - **Shared Store:** Reads recipient and message, writes status - **Internal:** Uses __a2a_messages__ shared store namespace

A2A Receive Node

Purpose: Receive message from another agent. - **Actions:** default (message received), empty - **Properties:** - sender_key: string — sender ID - output_key: string — received message - **Shared Store:** Reads sender, writes message - **Internal:** Uses __a2a_messages__ namespace

Observability / Utility

Log Node

Purpose: Write message to console and run log. - **Actions:** default - **Properties:** - input_key: string — message to log - output_key: string — message (passthrough) - log_level: string — "info", "debug", "warn", "error" - **Shared Store:** Reads message, writes same

Timer Node

Purpose: Measure execution time of sub-flow. - **Actions:** default - **Properties:** - input_key: string — operation or ID - output_key: string — elapsed time (seconds) - **Shared Store:** Reads input, writes elapsed time

Cache Node

Purpose: Cache result by key; return cached value on hit. - **Actions:** default (hit), miss - **Properties:** - key: string — cache key - input_key: string — value to cache - output_key: string — cached or new value - ttl: integer — cache lifetime (seconds) - **Shared Store:** Reads/writes cache via __cache__ namespace

Trace Node

Purpose: Add structured trace/span to execution. - **Actions:** default - **Properties:** - span_name: string — span name for tracing - input_key: string — data to trace - output_key: string — trace ID - **Shared Store:** Reads input, writes trace info

Data Structures / Memory

Registry Node

Purpose: Persistent key-value storage (persistent dict). - **Actions:** default - **Properties:** - operation: string — "set", "get", "delete" - key: string — registry key - input_key: string — value (for set) - output_key: string — retrieved value (for get) - **Shared Store:** Operates on __registry__ namespace

Stack Push Node

Purpose: Push value onto a stack. - **Actions:** default - **Properties:** - stack_name: string — named stack - input_key: string — value to push - **Shared Store:** Updates __stack_<name>__ array

Stack Pop Node

Purpose: Pop value from stack. - **Actions:** default (success), empty - **Properties:** - stack_name: string — named stack - output_key: string — popped value - **Shared Store:** Reads from __stack_<name>__ array, writes value

Queue Enqueue Node

Purpose: Add item to queue (FIFO). - **Actions:** default - **Properties:** - queue_name: string — named queue - input_key: string — item to add - **Shared Store:** Updates __queue_<name>__ array

Queue Dequeue Node

Purpose: Remove item from queue (FIFO). - **Actions:** default (success), empty - **Properties:** - queue_name: string — named queue - output_key: string — dequeued item - **Shared Store:** Reads from __queue_<name>__ array

Local Memory Node

Purpose: Temporary local storage (session-scoped dict). - **Actions:** default - **Properties:** - operation: string — "set", "get", "clear" - key: string — memory key - input_key: string — value (for set) - output_key: string — retrieved value (for get) - **Shared Store:** Operates on __local_memory__ namespace

Security

Secret Node

Purpose: Load secrets from environment, dotenv, or vault. - **Actions:** default (found), not_found - **Properties:** - key: string — environment variable name or secret path - source: string — "env", "dotenv", "aws_secrets", "vault" - output_key: string — loaded secret value - **Shared Store:** Writes secret - **Requires:** Appropriate env vars or libs (boto3 for AWS, hvac for Vault)

Human-in-the-Loop

Human Review Node

Purpose: Pause flow; user reviews content and decides pass/fail. - **Actions:** approved (user enters y), rejected (user enters n or cancels) - **Properties:** - input_key: string — content to review - output_key: string — review feedback - instructions: string — review instructions - **Shared Store:** Reads content, writes feedback - **In Creator UI:** Interactive dialog box - **In Standalone Scripts:** - Displays content to stdout - Prompts "Approve? [y/n]:" to stdout - Reads response from stdin - Handles EOF/Ctrl+C gracefully (routes to rejected) - Works with

pipelined input: `echo "y" | python script.py` - Works in CI/CD: `python script.py < approval.txt`

Human Input Node

Purpose: Pause flow; user enters text. - **Actions:** saved (if input provided), cancelled (if empty or EOF) - **Properties:** - `prompt`: string — prompt text to show user - `output_key`: string — user's input text - **Shared Store:** Writes user input to `output_key` - **In Creator UI:** Interactive input dialog - **In Standalone Scripts:** - Displays prompt to stdout - Reads input line from stdin - Handles EOF/Ctrl+C gracefully (routes to cancelled) - Works with pipelined input: `echo "Alice" | python script.py` - Works in shell scripts: `python script.py << EOF\nJohn\nEOF` - Non-interactive mode: `python script.py < /dev/null` (no crash, action = cancelled)

Batch / Async

Batch Node

Purpose: Process items in a batch. - **Actions:** default - **Properties:** - `items_key`: string — list of items - `output_key`: string — processed items - **Shared Store:** Reads items, writes results

Async Node

Purpose: Mark an operation as async (fire-and-forget). - **Actions:** default - **Properties:** - `input_key`: string — data to process async - `output_key`: string — async handle/ID - **Shared Store:** Reads input, writes handle

Async Batch Node

Purpose: Process batch items asynchronously. - **Actions:** default - **Properties:** - `items_key`: string — items to process - `output_key`: string — async handles - **Shared Store:** Reads items, writes handles

Async Parallel Batch Node

Purpose: Process batch items in parallel (max concurrency). - **Actions:** default - **Properties:** - `items_key`: string — items - `output_key`: string — results - `max_workers`: integer — max concurrent tasks - **Shared Store:** Reads items, writes results

Shell Command Node

Purpose: Execute shell command and capture output. - **Actions:** default (success), error - **Properties:** - command: string — shell command to run - output_key: string — stdout - shell_type: string — "bash", "sh", "zsh", "powershell", "cmd" - timeout: integer — max seconds to wait (default 30) - **Shared Store:** Writes stdout/stderr - **Requires:** Shell available (bash, PowerShell, etc.)

I/O

File Reader Node

Purpose: Read file contents. - **Actions:** default (success), error - **Properties:** - file_path_key: string — path to read - output_key: string — file contents - encoding: string — "utf-8" (default), "ascii", etc. - **Shared Store:** Reads path, writes contents

File Writer Node

Purpose: Write data to file. - **Actions:** default (success), error - **Properties:** - file_path_key: string — path to write - input_key: string — data to write - output_key: string — bytes written - mode: string — "w" (overwrite), "a" (append) - **Shared Store:** Reads path and data, writes byte count

Python Tool Node

Purpose: Load and execute Python module. - **Actions:** default (success), error - **Properties:** - tool_path: string — path to .py file - input_key: string — input to run() function - output_key: string — function result - **Shared Store:** Reads input, writes output - **Requires:** Module has def run(input) function

System / Shell

TTY Serial Node

Purpose: Read/write from serial port (see also USB Serial Input/Output). - **Actions:** read (success), timeout, written - **Properties:** - port: string — serial port path - baudrate: integer — baud rate - operation: string — "read" or "write" - input_key: string — data to write (for write) - output_key: string — data read (for read) - **Shared Store:** Reads/writes serial data

Spreadsheet Node

Purpose: Read/write CSV, TSV, or Excel files. - **Actions:** default (success), error - **Properties:** - file_path_key: string — file to read/write - operation: string — "read" or "write" - sheet_name: string — Excel sheet (if Excel) - output_key: string — data (for read) - input_key: string — data to write (for write) - **Shared Store:** Reads/writes tabular data - **Requires:** openpyxl for Excel

Networking

Socket Node

Purpose: TCP socket client/server communication. - **Actions:** default (success), error - **Properties:** - operation: string — "connect" (client) or "listen" (server) - host: string — hostname/IP - port: integer — port number - input_key: string — data to send (for connect) - output_key: string — received data - **Shared Store:** Reads/writes socket data

WebSocket Node

Purpose: WebSocket communication. - **Actions:** default (success), error - **Properties:** - operation: string — "send" or "listen" - url: string — WebSocket URL (e.g., ws://localhost:8000) - input_key: string — message to send - output_key: string — received message - **Shared Store:** Reads/writes messages - **Requires:** websockets pip package

Webhook Trigger Node

Purpose: Receive and process webhook payload. - **Actions:** default - **Properties:** - payload_key: string — payload from webhook - output_key: string — parsed payload - webhook_path: string — endpoint path (e.g., /webhook) - **Shared Store:** Writes parsed webhook data

Text / Data Processing

Regex Node

Purpose: Pattern matching and text replacement. - **Actions:** default (found/replaced), not_found - **Properties:** - input_key: string — text to search - output_key: string — matches or replaced text - pattern: string — regex pattern - operation: string — "findall", "sub", "match" - replacement: string — replacement text (for sub) - **Shared Store:** Reads text, writes results

Template Render Node

Purpose: Simple variable interpolation in text. - **Actions:** default - **Properties:** - `template_key`: string — template with `{{var}}` placeholders - `output_key`: string — rendered output - **Shared Store:** Reads template, writes rendered text

JSON Parse Node

Purpose: Parse JSON string to object. - **Actions:** default (valid), invalid - **Properties:** - `input_key`: string — JSON string - `output_key`: string — parsed object - **Shared Store:** Reads JSON, writes object

List Operations Node

Purpose: Transform lists (sort, reverse, unique, etc.). - **Actions:** default - **Properties:** - `input_key`: string — list to transform - `output_key`: string — result - `operation`: string — "sort", "reverse", "unique", "length" - **Shared Store:** Reads list, writes result

String Operations Node

Purpose: Transform strings (upper, lower, capitalize, etc.). - **Actions:** default - **Properties:** - `input_key`: string — string to transform - `output_key`: string — result - `operation`: string — "upper", "lower", "capitalize", "length", "trim" - **Shared Store:** Reads string, writes result

Resilience

Retry Node

Purpose: Retry failed operation with exponential backoff. - **Actions:** default (success), exhausted (max retries reached) - **Properties:** - `input_key`: string — operation/input - `output_key`: string — result - `max_retries`: integer — max attempt count (default 3) - `backoff_factor`: float — exponential backoff multiplier - **Shared Store:** Reads input, writes result

Rate Limiter Node

Purpose: Enforce request rate limits. - **Actions:** default (allowed), throttled - **Properties:** - `input_key`: string — request data - `output_key`: string — result - `requests_per_second`: float — rate limit - **Shared Store:** Reads input, writes result

Messaging

Email Send Node

Purpose: Send email via SMTP. - **Actions:** default (success), error - **Properties:** - recipient_key: string — to: email address - subject_key: string — email subject - body_key: string — email body (HTML or plain text) - output_key: string — status - **Shared Store:** Reads email data, writes status - **Requires:** EMAIL_ADDRESS, EMAIL_PASSWORD, SMTP_SERVER env vars

Email Read Node

Purpose: Fetch emails from IMAP mailbox. - **Actions:** default (success), error - **Properties:** - folder: string — mailbox folder (default "INBOX") - max_emails: integer — max to fetch - output_key: string — email list - **Shared Store:** Writes emails - **Requires:** EMAIL_ADDRESS, EMAIL_PASSWORD, IMAP_SERVER env vars

Notification Node

Purpose: Send notification to Slack, Discord, Teams, or Telegram. - **Actions:** default (sent), error - **Properties:** - channel: string — "slack", "discord", "teams", "telegram" - message_key: string — message text - output_key: string — status - **Shared Store:** Reads message, writes status - **Requires:** Webhook URL in appropriate env var (SLACK_WEBHOOK, etc.)

Context Compact Node

Purpose: Compress context using different strategies. - **Actions:** default - **Properties:** - input_key: string — context to compress - output_key: string — compressed context - strategy: string — "truncate", "summarize", "sliding_window" - max_length: integer — max output length - **Shared Store:** Reads context, writes compressed version

Conversation History Node

Purpose: Maintain multi-turn conversation state. - **Actions:** default - **Properties:** - input_key: string — new user message - output_key: string — conversation history - max_history: integer — max messages to keep - **Shared Store:** Reads message, writes history - **Internal:** Stores history in __conversation_history__ namespace

Summary

Total coverage: 76 node types across 23 categories. Each node is fully implemented in: - Visual palette with custom icons - Standalone Python script generation - Object Inspector property editor - Node dispatcher with type-based execution - Proper I/O handling in standalone scripts: - **stdin** for interactive nodes (Human Input, Human Review) - **stdout** for prompts and normal output - **stderr** for error messages - Works in pipes, CI/CD, and shell scripts - Graceful EOF handling for non-interactive execution

For tutorials and examples, see: - [Standalone Scripts Tutorial](#) — includes I/O and interactive examples - [Hardware I/O Tutorial](#) - [Your First Flow Tutorial](#)

UI Reference

Canvas

Graph Canvas

The canvas is the central panel where you visually design your PocketFlow workflow.

PocketFlow Creator canvas and panels
PocketFlow Creator canvas and panels

Basic Operations

Operation	How
Add a node	Drag a node type from the Component Palette and drop it on the canvas
Select a node	Click once — properties appear in the Object Inspector
Move a node	Click and hold, then drag to a new position
Delete a node	Select it; press the Delete key
Open code	Double-click a node to open its class in the Python editor
Connect nodes	Hover over the source node until action ports appear; drag from a port to a target node

Operation

Select an edge

Delete an edge

How

Click the line between two nodes

Select the edge; press the **Delete** key

Navigation

Action

Zoom in/out

Pan

Zoom to Fit

Zoom to selected node

Auto Arrange...

How

Ctrl+Scroll wheel, or Ctrl++ / Ctrl+-

Middle-drag or Space-drag

View > Zoom to Fit (Ctrl+0)

Ctrl+Shift+Z

View > Auto Arrange... (Ctrl+Shift+L) — opens settings dialog before running

Auto Arrange

View > Auto Arrange... (Ctrl+Shift+L) opens a settings dialog before rearranging the canvas.

Setting

Algorithm

Connector style

H Gap / V Gap

Max Cols

Options

Layered BFS, Grid, Force-directed (spring-embedder)

Straight, Curved (quadratic Bezier), Orthogonal (right-angle)

Horizontal and vertical spacing between nodes

Maximum columns (used by the Grid algorithm)

Settings are persisted per-project in the `.pfcproj.yaml` file under `auto_arrange:`. The operation is fully undoable with Ctrl+Z.

Undo / Redo

Action Shortcut

Undo Ctrl+Z

Redo Ctrl+Y

Undo covers: add node, delete node/edge, add edge, edit property, change edge action, move node, and Auto Arrange. The undo stack clears when a project is opened or created.

Node Anatomy

Each node shows: - **Title** — the display name (editable in Inspector) - **Type badge** — the node type (e.g., `llm_prompt_node`) - **Action ports** — right-side ellipses, one per declared action - **Input port** — left-side ellipse; receives incoming edges - **Red border** — validation error; check Problems tab

Breakpoints

- Select a node, then Node > Toggle Breakpoint (F9)
- A red dot appears in the node's corner
- Debug mode (Shift+F5) pauses when this node is about to execute

Code Sync

Every node has a corresponding class in `code/<graph_stem>.py`. When you delete a node from the canvas, its class block is removed from the code file.

[← Help Index](#) | [Inspector Help](#) | [Palette Help](#)

Component Palette

Component Palette

The Component Palette lists every node type available for dragging onto the canvas. It is divided into three sections, each separated by a labelled divider.

Section 1 — Built-in Nodes

The built-in nodes ship with PocketFlow Creator and are always available. They are grouped by category inside the palette.

Category	Node types
Flow Control	Start, Stop, Router, Subflow
Core / General	Basic Node, Python Tool
AI / LLM	LLM Prompt, JSON LLM, Classifier, Agent, RAG, Judge
AI / Reasoning	Chain of Thought, Majority Vote, Supervisor, Debate Advocate, Debate Judge

Category	Node types
AI / LLM Utilities	Context Compact, Conversation History
Human-in-the-Loop	Human Review, Human Input
Data / IO	File Reader, File Writer
Data Processing	JSON Parse, List Operations, String Operations, Regex, Template Render
Data / Vector	Vector Store, Embedding
Data Structures / Memory	Key-Value Store, Sliding Window Memory
Code / Execution	Code Executor, Python REPL
Processing / Async	Batch, Async, Async Batch, Async Parallel Batch
Web / Search	HTTP Request, Web Scrape, Web Search
Database / SQL	SQL Query
Voice / Audio	Speech-to-Text, Text-to-Speech
Document / Vision	PDF Reader, Image Analyser
Calendar	Calendar Event
MCP / Agent Protocol	MCP Tool Call
Observability / Utility	Logger, Timer, Counter
System / Shell	Shell Command, TTY Serial, Spreadsheet
Networking	Socket, WebSocket, Webhook Trigger
Resilience	Retry, Rate Limiter
Messaging	Email Send, Email Read, Notification
Security	Secret

Section 2 — Scientific & Engineering Add-on Nodes

These 34 domain-specific nodes ship with Creator in `addon_nodes/` and appear under the — **Scientific & Engineering** — divider. They are grouped by domain.

Domain	Node types
Geospatial	USGS Elevation Point, USGS 3DEP Elevation, National Map Download, Earthquake Catalog, Landsat Search & Download, ShakeMap Fetch, ShakeMap Scenario
Hydrology / Water	USGS Water Data, NWIS Query, StreamStats Basin, SWMM Run, EPANET

Domain

Weather / Atmosphere

Building Energy

Aerospace — CFD & Geometry

Aerospace — Propulsion & MDO

Wind Energy

Scientific Computing

Data Catalog

Node types

Run, MODFLOW 6 Run, FloPy Model, pyWatershed

NOAA Weather, WRF Model

EnergyPlus Run

Open VSP Geometry, VSPAERO Analysis, SU2 CFD, Cart3D Analysis, FUN3D Run

NASA CEA, RocketPy Flight, GMAT

Script, OpenMDAO Model, Optimization, NASA Trick Simulation

OpenFAST, KiteFAST

MATLAB Engine, Octave Script

USGS Data Catalog Search

For full property documentation see the [Node Reference](#). For hands-on tutorials see [GTKN Parts 20–25](#).

Section 3 — Custom Nodes

User-installed node packages appear under the — **Custom Nodes** — divider, grouped by category. Install packages via **Tools** → **Node Type Library** → **Install node package....**

See [Creating Custom Nodes](#) for how to write and install your own packages.

Using the Palette

1. Find the node type by scrolling or scanning the category groups.
2. Click and hold on a node type.
3. Drag it onto the **Graph Canvas**.
4. Release — the node appears at the drop position.
5. Click the new node to select it and edit its properties in the **Object Inspector**.

Tip: Node types also appear as icon buttons in the **Node toolbar** across the top of the window. Click any icon to drop that node at the centre of the current canvas view.

Node Lifecycle

Every node follows the `prep` → `exec` → `post` lifecycle:

`shared store` → `prep(shared)` → `exec(prepare_res)` → `post(shared, prepare_res, exec_res)` → `action string`

The `action string` returned by `post()` determines which edge to follow next.

[← Help Index](#) | [Canvas Help](#)

Object Inspector

Object Inspector

The Object Inspector (right panel) displays and edits properties of the selected node or edge.

Node Properties

Field	Editable	Description
ID	No	Unique internal identifier — auto-generated, never changes
Type	No	Node type from the palette (e.g., <code>llm_prompt_node</code>)
Title	Yes	Display name shown on the canvas
Position X / Y	No	Canvas coordinates — drag the node to reposition
Actions	Yes	Comma-separated output action names (e.g., <code>default</code> or <code>yes, no, maybe</code>)
Reads	Yes	Shared-store keys this node reads in <code>prep()</code> — documents the node's data contract
Writes	Yes	Shared-store keys this node writes in <code>post()</code> —

Field	Editable	Description
		documents the node's data contract

Custom type properties appear below the standard fields when a node type is selected. Properties with a fixed set of allowed values (e.g., `prompt_type`) are shown as a **drop-down selector**; all other properties use a text field.

Understanding Actions, Reads, and Writes

These three fields define the **data contract** of a node — what it needs, what it produces, and which path it takes next. Together they are the most important metadata you can give a node.

Actions — “Which way do I go next?”

When a node finishes, its `post()` method returns a **string**. That string is the **action**. PocketFlow looks up the outgoing edge whose label matches that string and follows it to the next node.

```
post() returns "approved"
  ↓
graph follows the "approved" edge
  ↓
next node runs
```

Actions in the Inspector is the comma-separated list of strings this node's `post()` might return. The validator checks that every outgoing edge label matches one of these declared actions — if you wire an edge labelled "success" but the node only declares "default", you get error PFCE2101.

Examples: - Simple linear node: default - Binary gate: approved, rejected - Classifier: positive, negative, neutral - Agent loop: done, continue

Tip: Declare every action before drawing edges. The canvas creates edge labels automatically from the output port you drag from, so your declared actions become clickable ports on the right side of the node tile.

Reads — “What do I take from the shared store?”

The **shared store** is a plain Python dict that every node in the flow can see. It is the only way data moves between nodes. When a node needs data produced by an earlier node, its `prep()` method reads it from the shared store:

```
def prep(self, shared: dict) -> str:
    return shared["user_input"]    # <-- reads "user_input" from
the store
```

Reads in the Inspector documents which keys this node's `prep()` pulls from the store. This is **documentation only** — **not enforced at runtime** — but it is invaluable:

- The **Data Flow Report** (Project > Data Flow Report) uses Reads declarations to show the full key lifecycle: who writes a key, who reads it, and whether any node reads a key that nobody writes.
- When you look at a graph cold, clicking a node and seeing reads: `user_input` tells you instantly what that node depends on — without opening the code editor.

Writes — “What do I put back into the shared store?”

After doing its work, a node's `post()` method writes results back into the shared store so downstream nodes can use them:

```
def post(self, shared: dict, prep_res, exec_res: str) -> str:
    shared["llm_response"] = exec_res    # <-- writes
"llm_response" into the store
    return "default"
```

Writes in the Inspector documents which keys this node's `post()` puts into the store. Like Reads, this is **documentation only**, but it feeds the Data Flow Report and makes the graph self-describing.

The Full Picture

INSPECTOR FIELD	MAPS TO	WHAT IT DOES
Reads store depends on)	<code>prep(shared)</code>	Pull inputs from the shared (documents what this node
Writes store produces)	<code>post(shared)</code>	Push outputs into the shared (documents what this node
Actions execute edge labels)	return value of <code>post()</code>	Choose the next node to (validated against outgoing

All three together make a node's contract explicit and visible on the canvas — no code reading required to understand what a node does, needs, or produces.

LLM Node Prompt Properties

Property	Values	Description
<code>prompt_type</code>	<code>string</code> / <code>path</code>	How <code>prompt_file</code> is interpreted
<code>prompt_file</code>	any text	Literal prompt text (when <code>prompt_type = string</code>) or relative file path (when <code>prompt_type = path</code>)

Setting `prompt_type = string` lets you type the prompt directly into the Inspector without creating a separate file. Setting `prompt_type = path` reads the prompt from a Markdown file at runtime — useful for long, reusable, or version-controlled prompts.

How to Edit a Field

1. Click the **blue value cell** in the Value column
2. The cell enters edit mode
3. Type the new value and press **Enter** — or click elsewhere to commit
4. Changes sync to the graph model immediately; validation re-runs automatically

Edge Properties

Click an edge (the line between two nodes) to see: - **Action** — the label on this transition; must match a declared action on the source node

Validation Feedback

When you change a field that affects validation (e.g., Actions), the canvas re-validates immediately. Nodes with errors gain a red border; the Problems tab shows details.

[← Help Index](#) | [Canvas Help](#)

Code Editor

Code Editor

The bottom **Python** tab is a syntax-highlighting editor for the graph's companion code file.

The Code File

Every graph has a companion file at `code/<graph_stem>.py`. This file: - Is created automatically when you first double-click a node - Contains one class stub per node, delimited by marker comments - Is your working implementation file during design time - Is merged into `custom/` on export (existing files are never overwritten)

Opening the Editor

- **Double-click a node** on the canvas — the file opens and scrolls to that node's class
- **Project Explorer > double-click a .py file** — opens the file in the Python tab

Marker Format

```
# --- NODE_START: <node_id> ---
class MyNode(Node):
    """<type_id>: <title>"""

    def prep(self, shared: dict) -> object:
        return None

    def exec(self, prep_res: object) -> object:
        return None

    def post(self, shared: dict, prep_res: object, exec_res:
object) -> str:
        return "default"

# --- NODE_END: <node_id> ---
```

Do not remove the `NODE_START` / `NODE_END` marker comments. They allow Creator to: - Locate the class for double-click navigation - Remove the class when you delete the node from the canvas

Syntax Highlighting

Python keywords, strings, comments, and decorators are highlighted. The editor does not run a language server — for full IDE features, open the file in your preferred external editor (the file is plain text).

Saving

- **Ctrl+S** — saves the currently active editor tab to disk
- **Ctrl+Shift+S** (Save All) — saves all open editor tabs

YAML and Markdown Editors

The **YAML** and **Markdown** tabs work the same way: - **YAML** — validates on every keystroke; parse errors appear in the status bar - **Markdown** — live preview updates in the right pane of the splitter

[← Help Index](#) | [Tutorial 4: The Code Editor](#)

Project Explorer

Project Explorer

The Project Explorer (left panel) shows the file and folder structure of the open project.

Tree Structure

MyProject/	
├─ graphs/	← double-click a .pfcgraph.yaml to open
it on canvas	
├─ prompts/	← double-click a .md file to open it in
Markdown editor	
├─ node_types/	← double-click a .yaml file to open it in
YAML editor	
├─ tools/	← double-click a .py file to open it in
Python editor	
├─ schemas/	← JSON Schema files for structured output
validation	
└─ Shared Store	← double-click to open the Shared Store
Designer	

Opening Files

File type	Action
<code>.pfcgraph.yaml</code>	Opens the graph on the canvas
<code>.md</code>	Opens in the Markdown editor tab
<code>.yaml</code>	Opens in the YAML editor tab
<code>.py</code>	Opens in the Python editor tab
Shared Store	Opens the Shared Store Designer dialog

After Opening a Graph

- The canvas shows the graph's nodes and edges
- The Component Palette becomes active
- The Object Inspector shows properties when a node is selected

Project Metadata

The project root contains `<name>.pfcproj.yaml` with project-level settings including: - Project name and description - Shared store schema - Default provider settings

[← Help Index](#) | [Canvas Help](#)

Shared Store

Shared Store Designer

The Shared Store Designer documents and validates the data contracts between nodes.

What Is the Shared Store?

The shared store is a plain Python dict passed to every node in the flow. Nodes read from it in `prep()` and write to it in `post()`. It is the only communication channel between nodes.

Opening the Designer

- **Tools > Shared Store Inspector...** — opens the designer for the current project

- **Project Explorer** > **double-click “Shared Store”** — same dialog

Columns

Column	Description
Namespace	Groups related keys (e.g., llm, user, data). Use any string.
Key	The dict key used in code: <code>shared["key"]</code> or <code>shared.get("key")</code>
Type	JSON Schema type: string, integer, number, boolean, array, object, null
Default	Optional initial value; displayed in the Run Log before any node sets the key

Adding, Editing, Removing Rows

- **Add:** Click an empty row and type; or use the add-row button
- **Edit:** Click any cell to edit in place
- **Remove:** Select a row and press Delete

Validate

Click **Validate** to check: - All type names are valid JSON Schema primitive types - No duplicate Namespace+Key combinations

How It Affects Runs

The shared store schema is documentation — it does not enforce types at runtime. During a run, the **Shared Store** tab shows live key/value pairs. Keys defined in the designer appear with their types annotated.

Saving

Click **OK** to save. The schema is stored in `project.pfcproj.yaml` under the `shared_store_schema` field.

[← Help Index](#) | [About PocketFlow](#)

Provider Manager

Provider Manager

Tools → Provider Manager opens the dialog where you create and manage named provider profiles. Each profile stores the configuration for an LLM service (Ollama, OpenAI, Claude, etc.).

Overview

The Provider Manager dialog is split into two panels:

Left panel (Profiles list): - Shows all your saved provider profiles - ★ indicates the default profile (used by flows that don't specify a provider) - Buttons: + **Add** (new), **Delete**, **Set Default** ★

Right panel (Profile editor): - Name, API type, base URL, default model, timeout - API Key field with options to enter directly or read from environment - **Test Connection** button to verify the settings work

Creating a New Profile

1. Click + **Add**
 2. Enter a profile name (e.g., "Local Ollama", "OpenAI Production")
 3. Select an **API type** from the dropdown
 4. The **Base URL** and **Default model** fields auto-populate with sensible defaults
 5. Customize if needed (e.g., change port for Ollama running on a non-standard port)
 6. If the provider requires an API key, enter it or select "Read from environment variable"
 7. Click **Test Connection** to verify everything works
 8. Click **OK** to save
-

Provider Types

Ollama (local)

Free, open-source LLM inference engine for local models.

Field	Default	Notes
Base URL	http://localhost:11434	Change if Ollama runs on a different port/host
Default model	qwen2.5-coder:14b	Pull models with ollama pull <model-name>
Timeout	120 s	Increase for large models or slow machines

Setup: Install Ollama from ollama.ai, run `ollama serve`, and pull a model.

LM Studio (local)

Desktop application for running local LLMs with a GUI.

Field	Default	Notes
Base URL	http://localhost:1234/v1	Change if LM Studio runs on a different port
Default model	meta-llama-3.1-8b	Set to the model you've loaded in LM Studio
Timeout	120 s	Increase for large models

Setup: Install LM Studio from lmstudio.ai and load a model.

OpenAI-compatible

OpenAI's API format, used by OpenAI, Azure OpenAI, Deepseek, Groq, and others.

Field	Notes
Base URL	e.g. <code>https://api.openai.com/v1</code> (OpenAI), <code>https://api.deepseek.com/v1</code> (Deepseek)
Default model	e.g. <code>gpt-4o</code> , <code>gpt-4o-mini</code> , <code>deepseek-chat</code>
API Key	Required; stored securely (see Security note below)
Timeout	120 s (usually sufficient for cloud APIs)

Anthropic (Claude)

Claude models from Anthropic.

Field	Default	Notes
Default model	claude-haiku-4-5	

Field	Default	Notes
		Other options: claude-sonnet-4-6, claude-opus-4-8
API Key	(required)	Get from console.anthropic.com
Timeout	120 s	Usually sufficient

Google Gemini

Google's generative AI models.

Field	Default	Notes
Default model	gemini-2.0-flash	Also: gemini-1.5-pro, gemini-1.5-flash
API Key	(required)	Get from aistudio.google.com
Timeout	120 s	Usually sufficient

Testing Connections

The **Test Connection** button sends a simple test prompt to verify your provider is accessible.

After clicking **Test Connection**: - The button shows "Testing..." while the test runs
 - After 5-10 seconds, you'll see one of: - ✓ **Connection successful** — provider is working - ✗ **[error message]** — there's a problem (see Troubleshooting below) - ✗ **Test timed out (30 s)** — provider didn't respond in time

API Key Security

By default, API keys are **not** stored in project files. They're stored securely in your operating system's application settings: - **Windows**: Registry - **macOS**: Keychain/Preferences - **Linux**: ~/.config/

Storing API keys in the environment

For production use, set environment variables before starting PocketFlow Creator:

```
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."
./scripts/run_app.sh
```


Then in the Provider Manager, select “Read from environment variable” and enter the variable name (e.g., OPENAI_API_KEY).

Including API keys in project files

Warning: Only do this for non-production projects or when keys are not sensitive.

Check **Include API keys in project file** to store keys in plain text in your project YAML. This allows the project to be fully portable but exposes keys in the file.

Using Providers in Workflows

Set the default provider

Profiles with a ★ next to their name are the default. Click **Set Default ★** to change it. Flows without an explicit provider setting will use the default.

Use a specific provider in a node

1. Select an LLM node in your workflow
 2. In the Object Inspector, find the **provider** dropdown
 3. Select a profile or leave blank to use the default
-

Troubleshooting

“Connection refused” or “Cannot connect”

For Ollama/LM Studio: - Verify the service is running - Check the port number in the Base URL - Test directly: `curl http://localhost:11434/api/tags` (Ollama) or visit `http://localhost:1234` (LM Studio)

For cloud providers: - Verify your internet connection - Check that your API key is correct and hasn’t expired

“Model not found”

For Ollama: - Run `ollama pull qwen2.5-coder:14b` to pull the model - Run `ollama list` to see available models - Update the **Default model** field to a model you’ve pulled

For other providers: - Verify the model name is correct and you have access to it
- Some models may not be available in your region or account tier

“Invalid API key”

- Verify you’ve copied the entire key correctly
 - Check the key hasn’t expired
 - Make sure you’re using a key for the right provider
-

More Information

For detailed setup instructions, custom port configuration, and advanced options, see: - [docs/13_provider_setup.md](#) — Comprehensive provider setup guide - [README.md](#) — Quick start section

[← Help Index](#)

Custom Nodes

Creating Custom Nodes

PocketFlow Creator gives you two ways to extend the Component Palette with your own node types. Choose the approach that fits your workflow:

Approach

GUI Wizard

Node Package (.py file)

Best for

Quickly registering a node that belongs to a specific project; low ceremony; definition stored inside the project folder

Reusable nodes you want available in every project; nodes you share with teammates; nodes developed in your own editor

Both approaches produce a `NodeTypeDefinition` and make the new node draggable from the palette immediately.

Approach 1 — Using the GUI Wizard

1.1 Open the wizard

Go to **Node** → **New Custom Node Type...**

The **Node Type Wizard** opens with three tabs: **Definition**, **Actions**, and **Properties**.

1.2 Definition tab

Field	What to enter
Type ID	A snake_case identifier unique within this project (e.g. weather_fetch_node)
Display Name	The label shown in the palette and on the canvas (e.g. Weather Fetch)
Category	The palette group this node appears under (e.g. Web / Search)
Base Class	Leave as Node unless you need async behaviour (AsyncNode, BatchNode, etc.)
Description	Optional one-line summary shown in the Node Type Library

1.3 Actions tab

Actions are the named outputs that appear as connector labels on the node. Click **Add** to insert a row and type the action name (e.g. default, error, retry). The first action in the list is the default route.

Every node should have at least one action. The canvas uses these names to label the connector handles on the right side of the node.

1.4 Properties tab

Properties appear in the **Object Inspector** when the node is selected on the canvas. Each property has:

Column	Meaning
Key	The Python attribute name (e.g. city_key)
Type	string, integer, number, bool, or choice
Default	The value pre-filled in the Inspector
Description	Tooltip text shown next to the field

For choice type, enter the allowed values as a comma-separated list in the **Default** column (e.g. `celsius, fahrenheit`).

1.5 Click OK

Creator writes a YAML definition file into `node_types/` inside your project folder and adds it to `project.yaml`. The node appears immediately in the palette under its category.

1.6 Write the implementation

Open the **Python editor** tab. A skeleton class matching your type ID is already present. Fill in `prep`, `exec`, and `post`:

```
from pocketflow import Node

class WeatherFetchNode(Node):
    CITY_KEY = "city"          # overridden per-instance by the
Inspector

    def prep(self, shared):
        return shared.get(self.CITY_KEY, "London")

    def exec(self, prep_res):
        # call your API here
        return {"city": prep_res, "temp_c": 18.5}

    def post(self, shared, prep_res, exec_res):
        shared["weather"] = exec_res
        return "default"
```

Tip: Inspector properties declared in the wizard are injected as uppercase class attributes before `prep` runs, so `self.CITY_KEY` reflects whatever the user typed in the Inspector.

Approach 2 — Writing a Node Package

A **node package** is either a single `.py` file, or a **multi-file folder**, that you write in any editor. Drop it into the user nodes directory and it loads automatically next time Creator starts, or immediately via the Node Type Library.

2.1 Where packages live

`~/.pocketflow_creator/nodes/`

On Windows this resolves to `C:\Users\<you>\.pocketflow_creator\nodes\`. Creator creates the folder on first launch; you can also open it from **Tools → Node Type Library → Open nodes folder**.

2.2 File naming

Single-file package — name the file after your node class in snake_case:

<code>weather_fetch_node.py</code>	✓	
<code>WeatherFetch.py</code>	✓	(works, but convention is snake_case)
<code>_helper.py</code>	x	(leading underscore – skipped by the loader)
<code>__init__.py</code>	x	(skipped)

Multi-file package — create a folder whose name matches the main entry-point file:

<code>weather_fetch/</code>		
<code>weather_fetch.py</code>	✓	entry point (folder name == file name)
<code>helpers.py</code>	✓	any helper modules you like
<code>models.py</code>	✓	
<code>_private.py</code>	✓	private helpers (not loaded as a package themselves)

The folder name and entry-point file must share the same name (underscores, no spaces).

A folder that does not follow this convention is reported in the  **Errors** tab and skipped.

2.3 The `__node_meta__` dict

Every package must contain a module-level `__node_meta__` dict. It carries all identity, package, and behaviour metadata in one place — no special syntax, just a plain Python dict:

```
__node_meta__ = {
    # — Identity
    "node": "Weather Fetch",          # display name shown in
the palette
    "category": "Web / Search",       # palette / toolbar group

    # — Package info (all optional)
    "version": "1.0.0",
    "author": "Your Name",
    "website": "https://yoursite.example.com",
    "repo": "https://github.com/you/weather-fetch-
```

```

node",
    "description": "Fetches current temperature for a
city.",
    "tags": ["weather", "api", "http"],
    "license": "MIT",
    "min_creator_version": "0.2.0",

# — Node behaviour


---


    "actions": ["default", "error"],
    "properties": {
        "city_key": {
            "type": "string",
            "default": "city",
            "description": "Shared-store key for the target city
name.",
        },
        "result_key": {
            "type": "string",
            "default": "weather",
            "description": "Shared-store key to write the result
dict into.",
        },
    },
},

# — Visual


---


    "color": "#0277bd",
# hex background colour for the palette icon
}

```

Required keys

Key	Type	Description
node	str	Display name
category	str	Palette group (can be an existing category or a new one)

Optional keys

Key	Type	Default	Description
version	str	"0.0.0"	Semantic version
author	str	" "	Author name
website	str	" "	

Key	Type	Default	Description
			Author or project website URL
repo	str	""	Source repository URL
description	str	""	One-line summary
tags	list[str] or comma-separated [] str		Search keywords
license	str	""	SPDX identifier (e.g. "MIT")
min_creator_version	str	""	Minimum Creator version required
actions	list[str]	["default"]	Named output connectors
properties	dict	{}	Inspector properties (same schema as the wizard)
color	str	"#555555"	Hex background colour for the auto-generated icon

2.4 The Node class

Put exactly one class in the file that either:

- **Extends `pocketflow.Node`** (or `AsyncNode`, `BatchNode`, `AsyncBatchNode`, `AsyncParallelBatchNode`), or
- **Has `prep`, `exec`, and `post` methods** (duck-typed — no import required if PocketFlow is not installed in your editor's environment)

The class name is converted to a `type_id` automatically:

Class name	Derived <code>type_id</code>
<code>WeatherFetchNode</code>	<code>weather_fetch_node</code>
<code>SQLRunner</code>	<code>sql_runner_node</code>
<code>MyNode</code>	<code>my_node</code>

Rule of thumb: if the class name already ends in `Node` or `_node` the suffix is not doubled.

2.5 Custom icon (optional)

Set `__node_icon__` to a callable with the signature
(p: QPainter, sz: float, bg: QColor) -> None. The function receives a prepared QPainter with antialiasing enabled and the background already filled. Draw white shapes on top.

```
def _draw_my_icon(p, sz, bg):
    from PySide6.QtCore import QPointF
    from PySide6.QtGui import QPolygonF, QColor, QBrush
    from PySide6.QtCore import Qt
    # Example: a filled diamond
    half = sz * 0.38
    cx, cy = sz / 2, sz / 2
    pts = QPolygonF([
        QPointF(cx, cy - half),
        QPointF(cx + half, cy),
        QPointF(cx, cy + half),
        QPointF(cx - half, cy),
    ])
    p.setPen(Qt.PenStyle.NoPen)
    p.setBrush(QBrush(QColor("white")))
    p.drawPolygon(pts)

__node_icon__ = _draw_my_icon
```

When `__node_icon__` is absent or None, Creator automatically generates a two-letter initials icon using the color from `__node_meta__`.

2.6 Complete example

Save the following as `~/.pocketflow_creator/nodes/weather_fetch_node.py`:

```
"""Fetches current conditions from the Open-Meteo API (no API key
needed)."""

__node_meta__ = {
    "node": "Weather Fetch",
    "category": "Web / Search",
    "version": "1.0.0",
    "author": "Your Name",
    "website": "https://open-meteo.com",
    "description": "Returns temperature and weather code for any
city.",
    "tags": ["weather", "api", "http"],
    "license": "MIT",
    "actions": ["default", "error"],
```



```

        "properties": {
            "city_key": {
                "type": "string",
                "default": "city",
                "description": "Shared-store key holding the target
city name.",
            },
            "result_key": {
                "type": "string",
                "default": "weather",
                "description": "Shared-store key to write the result
dict into.",
            },
        },
        "color": "#0277bd",
    }

```

```

__node_icon__ = None    # use auto-generated initials icon

```

```

class WeatherFetchNode:
    """Resolve a city name to coordinates, then fetch current
weather."""

    def prep(self, shared):
        return {
            "city": shared.get("city", "London"),
            "result_key": "weather",
        }

    def exec(self, prep_res):
        city = prep_res["city"]
        import json, urllib.request

        # Step 1: geocode
        geo_url = (
            "https://geocoding-api.open-meteo.com/v1/search"
            f"?name={city}&count=1&language=en&format=json"
        )
        with urllib.request.urlopen(geo_url, timeout=10) as r:
            geo = json.loads(r.read())
        if not geo.get("results"):
            return {"city": city, "error": f"City not found:
{city!r}"}

        loc = geo["results"][0]

        # Step 2: current weather
        wx_url = (

```

```

        "https://api.open-meteo.com/v1/forecast"
        f"?latitude={loc['latitude']}"
&longitude={loc['longitude']}"
        "&current=temperature_2m,weather_code"
    )
    with urllib.request.urlopen(wx_url, timeout=10) as r:
        wx = json.loads(r.read())
    c = wx["current"]
    return {
        "city": city,
        "latitude": loc["latitude"],
        "longitude": loc["longitude"],
        "temperature_c": c["temperature_2m"],
        "weather_code": c["weather_code"],
    }

def post(self, shared, prep_res, exec_res):
    shared[prep_res["result_key"]] = exec_res
    return "error" if "error" in exec_res else "default"

```

Approach 2b — Multi-File Node Packages

When a node needs helper modules (data models, API clients, constants, utilities), split it into a **folder package** instead of cramming everything into one file.

Convention

The folder name and entry-point filename must be identical:

```

~/.pocketflow_creator/nodes/
├── geocode_node/           ← folder name
│   ├── geocode_node.py    ← entry point (same name as folder)
│   ├── api_client.py      ← helper module
│   └── models.py          ← data models

```

Relative imports

Within the package, use **relative imports only**:

```

# geocode_node/geocode_node.py
from . import api_client    # relative – works correctly
from . import models        #

import api_client           # absolute – unreliable; do not
use

```

Creator loads each multi-file package in complete isolation using `importlib`'s `submodule_search_locations` — no `sys.path` mutation. Two packages that both contain a `helpers.py` will never interfere with each other.

Complete multi-file example

```
~/.pocketflow_creator/nodes/  
└─ geocode_node/  
   └─ geocode_node.py  
   └─ geo_client.py
```

geo_client.py:

```
"""Thin wrapper around the Open-Meteo geocoding endpoint."""  
import json, urllib.request  
  
def lookup(city: str) -> dict:  
    url = (  
        "https://geocoding-api.open-meteo.com/v1/search"  
        f"?name={city}&count=1&language=en&format=json"  
    )  
    with urllib.request.urlopen(url, timeout=10) as r:  
        data = json.loads(r.read())  
        results = data.get("results", [])  
        return results[0] if results else {}
```

geocode_node.py:

```
"""Resolves a city name to latitude/longitude via Open-Meteo."""  
from . import geo_client  
  
__node_meta__ = {  
    "node": "Geocode",  
    "category": "Geospatial",  
    "version": "1.0.0",  
    "description": "Resolves a city name to lat/lon.",  
    "actions": ["default", "not_found"],  
    "properties": {  
        "city_key": {"type": "string", "default": "city",  
"description": "Input key"},  
        "result_key": {"type": "string", "default": "location",  
"description": "Output key"},  
    },  
    "color": "#2e7d32",  
}  
  
__node_icon__ = None
```

```

class GeocodeNode:
    def prep(self, shared):
        return {"city": shared.get("city", ""), "result_key":
"location"}

    def exec(self, prep_res):
        return geo_client.lookup(prep_res["city"])

    def post(self, shared, prep_res, exec_res):
        if not exec_res:
            return "not_found"
        shared[prep_res["result_key"]] = exec_res
        return "default"

```

Installing a multi-file package via the GUI

1. Go to **Tools** → **Node Type Library** → **Installed Custom** tab.
2. Click **Install node package folder...**
3. Select the **folder** (e.g. `geocode_node/`).
4. Creator validates the `{name}/{name}.py` convention and copies the entire folder.

Testing a multi-file package outside Creator

```

import importlib.util, sys
from pathlib import Path

pkg_dir = Path.home() / ".pocketflow_creator/nodes/geocode_node"
entry    = pkg_dir / "geocode_node.py"
spec     = importlib.util.spec_from_file_location(
    "geocode_node", entry,
    submodule_search_locations=[str(pkg_dir)],
)
mod = importlib.util.module_from_spec(spec)
sys.modules["geocode_node"] = mod
spec.loader.exec_module(mod)

def test_geocode_london():
    node = mod.GeocodeNode()
    shared = {"city": "London"}
    prep    = node.prep(shared)
    result  = node.exec(prepare)
    assert "latitude" in result

```

Installing a Package via the GUI

If you received a node package from a colleague or downloaded one:

1. Go to **Tools → Node Type Library**.
2. Click the **Installed Custom** tab.
3. Click **Install node package (.py)...** for a single-file package, or **Install node package folder...** for a multi-file folder package.
4. Select the file or folder.
5. If a package with the same name is already installed, Creator asks whether to replace it.
6. After a successful install the node appears immediately in the **Component Palette** under its category. The toolbar shows the node after the next application restart.

To remove a package open the nodes folder (**Open nodes folder** button), delete the `.py` file or folder, and restart Creator.

The Node Type Library Dialog

Tools → Node Type Library has four tabs:

Tab	Contents
Built-in	All built-in nodes with ID, display name, and category
Scientific & Engineering	The 34 add-on scientific and engineering nodes that ship with Creator, grouped by domain (Aerospace, Hydrology, Geospatial, etc.)
Installed Custom	Every user-installed node package — version, author, license, source file/folder; click a row to see description, tags, website, and repo
 Errors	Packages (built-in add-ons or user packages) that failed to load, with the filename and error message. Fix the file and restart to retry

Load Order and the `type_id` Namespace

1. **Built-in nodes** are registered first.

2. **Add-on nodes** (the scientific & engineering packages that ship with Creator) are loaded next, in filename order.
3. **User packages** are loaded last, in alphabetical order — single-file .py files first, then multi-file folders.

Collision rules: - A user package whose `type_id` matches a built-in or add-on node is silently skipped; the existing definition wins and an error is recorded in the **Errors** tab. - Within user packages, a later entry silently overwrites an earlier entry with the same `type_id`. Rename one of them to resolve the conflict.

Sharing Your Node Package

A single-file package is a self-contained .py file — send it by email, post it in a GitHub Gist, or publish it on PyPI. Recipients install it through **Tools → Node Type Library → Install node package (.py)...** or by copying the file directly into `~/.pocketflow_creator/nodes/`.

A multi-file package is a folder — zip it or share the whole directory. Recipients extract it and install with **Install node package folder...** or by copying the folder into `~/.pocketflow_creator/nodes/`. Make sure the zip preserves the top-level folder so the `{name}/{name}.py` convention is intact after extraction.

Suggested `__node_meta__` fields to fill in before sharing:

- `version` — bump on every update so users know they have the latest
 - `author + website` — lets users reach you
 - `repo` — allows users to report issues and contribute improvements
 - `description + tags` — makes the node discoverable in the Library dialog
 - `license` — required for open-source distribution; "MIT" is the most permissive common choice
-

Developing a Package in an External Editor

Recommended workflow

1. Create the file in `~/.pocketflow_creator/nodes/` so it auto-loads on the next start.
2. Edit in your preferred editor (VS Code, PyCharm, etc.).
3. When you change the file, restart Creator (or use **Tools → Node Type Library → Install node package (.py)...** on the same file to hot-reload it).

Type hints and editor support

`pocketflow.Node` is available via `pip install pocketflow`. Adding it as a base class gives your editor full auto-complete on `prep`, `exec`, and `post`:

```
from pocketflow import Node

class WeatherFetchNode(Node):
    def prep(self, shared: dict) -> dict: ...
    def exec(self, prep_res: dict) -> dict: ...
    def post(self, shared: dict, prep_res: dict, exec_res: dict)
-> str: ...
```

If you do not want to install PocketFlow in your editor environment, duck-typing works fine — just define the three methods and the loader accepts the class.

Keeping secrets out of the package

Never hard-code API keys in the package. Use a **Secret Node** upstream to load credentials from environment variables or a `.env` file, and read them from the shared store in `prep`:

```
def prep(self, shared: dict) -> dict:
    return {
        "api_key": shared.get("openai_api_key", ""),    # loaded
by Secret Node
        "city":    shared.get("city", "London"),
    }
```

Testing outside Creator

Because the class has no Creator-specific dependencies, you can unit-test it with plain `pytest`:

```
# test_weather_fetch_node.py
import importlib.util, sys
from pathlib import Path

spec = importlib.util.spec_from_file_location(
    "weather_pkg",
    Path.home() / ".pocketflow_creator/nodes/
weather_fetch_node.py",
)
mod = importlib.util.module_from_spec(spec)
spec.loader.exec_module(mod)

def test_exec_returns_temperature():
    node = mod.WeatherFetchNode()
```

```
prep_res = {"city": "London", "result_key": "weather"}
result = node.exec(prepare_res)
assert "temperature_c" in result
assert result["city"] == "London"
```

Quick Reference

Minimal package skeleton

```
__node_meta__ = {
    "node": "My Node",
    "category": "Custom",
    "actions": ["default"],
}

class MyNode:
    def prep(self, shared): return None
    def exec(self, prep_res): return None
    def post(self, shared, prep_res, exec_res):
        return "default"
```

`__node_meta__` key reference

Key	Required	Type	Description
node	✓	str	Display name
category	✓	str	Palette group
version		str	Semantic version
author		str	Your name
website		str	Your website URL
repo		str	Source repository URL
description		str	One-line summary
tags		list[str]	Search keywords
license		str	SPDX identifier
min_creator_version		str	Minimum Creator version
actions		list[str]	Output connector names (default: ["default"])
properties		dict	Inspector property definitions
color		str	

Key	Required	Type	Description
			Hex icon background colour

Property definition schema

```

    "my_property": {
      "type":          "string",      # string | integer | number |
bool | choice
      "default":       "value",
      "description":   "Shown as a tooltip in the Inspector.",
      # For type "choice" only:
      "choices":       ["option_a", "option_b", "option_c"],
    }

```

[↑ Tutorials Index](#)

Appendix: Quick Reference

PocketFlow Node Reference

A concise description of every built-in node type available in the Component Palette. Each node generates a class that inherits from the named PocketFlow base class.

The Node Data Contract: Actions, Reads, and Writes

Every node has three Inspector fields that together describe its **data contract** — the formal agreement between the node and the rest of the flow.

Actions — the routing outputs

When a node finishes, its `post()` method returns a **string**. PocketFlow follows the outgoing edge whose label matches that string to reach the next node. The **Actions** field in the Inspector is the comma-separated list of strings `post()` might return.

`post()` returns "approved" → graph follows the "approved" edge
→ next node runs

Every outgoing edge label must match one of the node’s declared actions. The validator enforces this — undeclared actions produce error PFCE2101. Declaring all actions before drawing edges causes the canvas to create a labelled output port for each one, making the routing structure visible without opening any code.

Pattern	Actions declaration
Simple linear step	default
Binary gate	approved, rejected
Multi-way classifier	positive, negative, neutral
Self-looping agent	done, continue

Reads — what the node takes from the shared store

The **shared store** is the dict that flows through every node in the graph. It is the only channel through which nodes pass data to each other. A node’s `prep()` method pulls the inputs it needs:

```
def prep(self, shared: dict) -> str:
    return shared["user_input"]    # reads "user_input" from the
shared store
```

The **Reads** field documents which keys `prep()` consumes. It is not enforced at runtime, but it powers the **Data Flow Report** and makes dependencies visible on the canvas without reading code.

Writes — what the node puts back into the shared store

After doing its work, `post()` stores results for downstream nodes:

```
def post(self, shared: dict, prep_res, exec_res: str) -> str:
    shared["llm_response"] = exec_res    # writes "llm_response"
for downstream nodes
    return "default"
```

The **Writes** field documents which keys `post()` produces. Combined with Reads across all nodes, the **Data Flow Report** (Project > Data Flow Report) can show the full key lifecycle: who writes each key, who reads it, and whether any key is read before it is written.

Summary table

Inspector field	Maps to	Role
Reads	prep(shared)	Pull inputs from the shared store
Writes	post(shared)	Push outputs into the shared

store
Actions follow return value Choose which outgoing edge to
of post()

Reads and Writes are **documentation** — not runtime enforcement. Fill them in carefully and the Data Flow Report becomes a live audit of your graph's data flow.

Flow Control

Start Node

Base class: Node

Marks the entry point of a flow. Every graph must have exactly one start node. The `post()` method returns "default" to continue to the first real processing node. The Start Node itself does no work — it is a routing anchor only.

Stop Node

Base class: Node

Marks a terminal point in the flow. A graph may have multiple Stop Nodes (one per exit path). When the flow reaches a Stop Node it halts execution. Use separate Stop Nodes for distinct exit conditions (e.g. success vs error) to make the graph self-documenting.

Router Node

Base class: Node

Routes execution to one of several branches based on a decision made in `exec()`. The `post()` method returns the chosen action string; each action must be wired to a downstream node. Declare all possible actions in the Inspector **Actions** field. Use this for conditional logic, guardrails, and state-machine transitions.

Subflow Node

Base class: Node

Embeds a reusable sub-graph inside the current flow. Set the `subflow_ref` property to a path relative to the project root (e.g. `graphs/summarizer.pfcgraph.yaml`). When the runner reaches this node it executes

the referenced graph inline, merging its shared store state back into the parent flow before continuing.

LLM / AI

LLM Prompt Node

Base class: Node

The standard node for making a single call to a language model.

Property	Default	Description
prompt_type	string	string — treat prompt_file as literal text; path — read from a file
prompt_file	(empty)	The prompt text or file path, depending on prompt_type
model	(project default)	Override the model for this node only
temperature	0.7	Sampling temperature 0–1
max_tokens	1024	Max tokens in the response
output_key	output	Shared store key where the response is written

Choose `prompt_type = string` to type the prompt directly into the Inspector. Choose `prompt_type = path` to load a Markdown file at runtime — useful for long or version-controlled prompts. The Prompt Preview tab shows the resolved prompt either way.

JSON LLM Node

Base class: Node

Like LLM Prompt Node but instructs the model to respond with structured JSON. Supports the same `prompt_type / prompt_file` duality — use `string` for short inline instructions or `path` for a Markdown schema-description file. The parsed JSON object is written to the shared store under `output_key`. Use this for data extraction, classification with typed output, and any workflow that feeds LLM output into downstream Python code.

Classifier Node

Base class: Node

Sends a classification prompt to the LLM and returns one of a fixed set of labels. The labels are declared as **Actions** in the Inspector so the graph can route on the result without a separate Router Node. Simpler than a Router Node for pure text-in → label-out patterns (sentiment, intent detection, topic routing).

Agent Node

Base class: Node

Implements the decide-act loop of an autonomous LLM agent. On each iteration the LLM chooses an action from the `tools` list; the loop continues until the agent returns "answer" or `max_iterations` is reached. Wire `tool_call` edges to individual tool nodes that each loop back to the Agent Node. Wire answer to the final output node.

RAG Node

Base class: Node

Encapsulates the embedding and retrieval step of a Retrieval-Augmented Generation pipeline. Given a query, it embeds the query vector and retrieves the `top_k` most similar chunks from the index stored at `index_key` in the shared store. The retrieved context is written to `shared["context"]` for a downstream LLM Prompt Node to consume.

Judge Node

Base class: Node

Evaluates LLM-generated output against a `criteria` string using a second LLM call. Returns "pass" if the output meets the bar or "fail" to trigger a refinement loop. A `max_iterations` guard in `post()` forces "pass" after N attempts so the flow always terminates. Wire fail to a Refine node that improves the output and loops back to the generator.

Data / I/O

File Reader Node

Base class: Node

Reads a file from disk and writes its contents into the shared store. Set `file_path` in the Inspector to a path relative to the project root. The node is intentionally minimal — decoding, parsing, and chunking belong in a downstream Basic Node so you control the format.

File Writer Node

Base class: Node

Writes data from the shared store to a file on disk. The node stub is intentionally left for you to implement because the format (text, JSON, CSV, binary), path strategy, and encoding are application-specific. Fill in `exec()` with the serialisation logic your flow requires.

Python Tool Node

Base class: Node

Runs an arbitrary Python function or shell command. Use this for calculations, external API calls, data transformations, or any work that does not involve an LLM. `exec()` is generated as a plain Python method body — write whatever code you need. The node is the escape hatch for anything that does not fit another node type.

Processing

Basic Node

Base class: Node

The general-purpose building block. Use a Basic Node for any step that does not fit a more specific type: data preparation, state updates, printing output, calling a library, or any single-step operation with a "default" exit. When in doubt, start with a Basic Node and refactor to a more specific type once the pattern becomes clear.

Batch Node

Base class: BatchNode

Processes a list of items by calling `exec()` once per item. `prep()` returns the list; `exec(item)` handles one item; `post()` receives the full results list. Use this for map-style processing: scoring a batch of resumes, translating a list of sentences, or embedding a list of text chunks. The batch runs synchronously and sequentially.

Async Node

Base class: AsyncNode

A single-step node whose `exec_async()` method is an `async def` coroutine. Use this whenever the work is I/O-bound (HTTP requests, database queries, file reads) and you want to yield the event loop rather than block. Methods are named `prep_async`, `exec_async`, and `post_async`. The runner wraps execution in `asyncio.run()`.

Async Batch Node

Base class: AsyncBatchNode

Processes a list of items by awaiting `exec_async(item)` for each item in turn. Like Batch Node but non-blocking: each item's I/O yields the event loop while waiting. Use when each item requires an async operation (e.g. fetching a URL, querying an async database driver) but you do not need parallel execution.

Async Parallel Batch Node

Base class: AsyncParallelBatchNode

Processes a list of items by launching all `exec_async(item)` calls concurrently via `asyncio.gather`. Results are collected in the original order. Set `max_concurrency` in the Inspector to limit how many coroutines run simultaneously. Use this when items are independent and latency dominates — fetching many URLs, calling an API in parallel, or embedding many chunks concurrently.

Human in the Loop

Human Review Node

Base class: Node

Pauses the flow and presents content to a human reviewer. `exec()` prints or displays the content and waits for input. `post()` routes to "approved" or "rejected" (or any custom actions you declare). Use this for content moderation, data labelling, quality gates, or any step that requires a human decision before the flow continues.

Human Input Node

Base class: Node

Reads a string from the user at runtime (via `input()` or a UI prompt) and writes it to the shared store under `output_key`. Use this as the interactive entry point for flows that require user input each time they run — chatbots, question-answer tools, or any pipeline where the query varies per invocation.

AI / Reasoning

Chain of Thought Node

Base class: Node

Prompts an LLM to reason step-by-step before delivering an answer. Each reasoning step is a separate LLM call and the intermediate steps are stored so they can be inspected or logged. Use this when a single-shot answer is unreliable and you want the model to “show its work” across steps incremental reasoning passes.

Property	Default	Description
<code>model</code>	<code>gpt-4o</code>	LLM model name
<code>steps</code>	<code>3</code>	Number of reasoning steps
<code>output_key</code>	<code>cot_result</code>	Shared-store key for the final result

Majority Vote Node

Base class: Node

Generates `samples` independent answers to the same prompt and returns the most frequent response. Self-consistency via majority vote improves accuracy on tasks where the model occasionally makes mistakes. Works best with deterministic-ish questions (math, code, multi-choice reasoning) rather than open-ended generation.

Property	Default	Description
<code>model</code>	<code>gpt-4o</code>	LLM model name
<code>samples</code>	<code>5</code>	Number of independent samples
<code>output_key</code>	<code>voted_result</code>	Shared-store key for the winning answer

Supervisor Node

Base class: Node

Orchestrates one or more sub-agent nodes by deciding at each iteration whether to continue (hand off to the next agent step), declare done, or signal error. Wire continue to the sub-agent node and back to the Supervisor Node to build the supervision loop. Set `max_iterations` to prevent runaway loops.

Property	Default	Description
<code>model</code>	<code>gpt-4o</code>	LLM model name
<code>max_iterations</code>	<code>10</code>	Maximum supervision iterations
<code>output_key</code>	<code>supervisor_result</code>	Shared-store key for the final result

Debate Advocate Node

Base class: Node

Takes one side of a debate (pro or con) and generates the strongest possible argument for that position. Pair two Debate Advocate Nodes (one per position) and feed their outputs to a Debate Judge Node to implement an adversarial evaluation pattern that tends to surface counterarguments missed by single-shot critique.

Property	Default	Description
<code>model</code>	<code>gpt-4o</code>	LLM model name
<code>position</code>	<code>pro</code>	Debate position: pro or con
<code>output_key</code>	<code>advocate_argument</code>	Shared-store key for the argument text

Debate Judge Node

Base class: Node

Reads arguments from both sides of a debate (produced by Debate Advocate Nodes) and decides the winner. Returns `pro_wins`, `con_wins`, or `tie`. Wire each action to a different downstream node to take position-specific actions based on the outcome.

Property	Default	Description
<code>model</code>	<code>gpt-4o</code>	LLM model name

Property	Default	Description
output_key	debate_verdict	Shared-store key for the verdict string

Web / Search

Web Search Node

Base class: Node

Runs a web search query and returns a list of result dicts. Reads the query from query_key in the shared store, runs it through the configured search engine, and writes up to num_results results to output_key. Routes no_results when the search returns nothing, allowing the flow to fall back gracefully.

Property	Default	Description
engine	duckduckgo	Search engine: duckduckgo, google, bing
num_results	5	Maximum results to return
query_key	search_query	Shared-store key for the query string
output_key	search_results	Shared-store key for the results list

Web Scrape Node

Base class: Node

Fetches the HTML at the URL stored in url_key and extracts clean plain text. Strips navigation, ads, and boilerplate via a readability-style pass. Routes error on network failure or timeout. Use downstream with a Text Chunk Node for RAG ingestion or with an LLM Prompt Node for single-page summarisation.

Property	Default	Description
url_key	url	Shared-store key containing the target URL
output_key	scraped_text	Shared-store key for extracted text
timeout	10	HTTP timeout in seconds

API Call Node

Base class: Node

Makes an HTTP request to any REST endpoint. Supports GET, POST, PUT, and DELETE. The URL supports `{{key}}` interpolation from the shared store. Request headers and body are read from separate shared-store keys so they can be constructed dynamically by upstream nodes. Routes error on non-2xx responses.

Property	Default	Description
url	<i>(empty)</i>	Endpoint URL (supports <code>{{key}}</code> interpolation)
method	GET	HTTP method
headers_key	<i>(empty)</i>	Shared-store key for extra headers dict
body_key	<i>(empty)</i>	Shared-store key for request body
output_key	api_response	Shared-store key for the response

Data / Vector

Text Chunk Node

Base class: Node

Splits a long text into overlapping chunks of `chunk_size` tokens (or characters). Writes a list of chunk strings to `output_key`. Use as the first step of a RAG ingestion pipeline before an Embed Node and Vector Index Node.

Property	Default	Description
input_key	text	Shared-store key for the source text
chunk_size	512	Chunk size in tokens/characters
overlap	64	Overlap between consecutive chunks
output_key	chunks	Shared-store key for the chunks list

Embed Node

Base class: Node

Generates embedding vectors for a string or list of strings using the configured embedding model. Single string $\rightarrow$ single vector; list $\rightarrow$ list of vectors. Pair with Vector Index Node to build an index or with Vector Retrieve Node to embed a query before retrieval.

Property	Default	Description
model	text-embedding-3-small	Embedding model name
input_key	chunks	Shared-store key for text or text list
output_key	embeddings	Shared-store key for the embedding vector(s)

Vector Index Node

Base class: Node

Builds a vector store index from a list of embedding vectors. Supports FAISS, Chroma, and Pinecone backends. Writes the built index object to `index_key` for use by a downstream Vector Retrieve Node. Use this once during ingestion; cache the result if the document set rarely changes.

Property	Default	Description
store_type	faiss	Vector store backend: faiss, chroma, pinecone
embeddings_key	embeddings	Shared-store key for the vectors to index
index_key	vector_index	Shared-store key for the built index

Vector Retrieve Node

Base class: Node

Queries a vector index for the `top_k` chunks most similar to the query. Reads the index from `index_key` and the query from `query_key` (text string or pre-computed vector). Routes `no_results` when the index is empty or the similarity threshold is not met.

Property	Default	Description
index_key	vector_index	Shared-store key for the vector index
query_key	query	Shared-store key for the query text or vector
top_k	5	Number of nearest neighbours to return
output_key	retrieved_docs	Shared-store key for the results list

Database / SQL

DB Schema Node

Base class: Node

Inspects a database and generates a human-readable schema description. Reads the connection string (DSN or SQLAlchemy URL) from `connection_key` and writes a compact table/column description to `output_key`. Feed this into an NL to SQL Node as context so the LLM knows the available tables and columns.

Property	Default	Description
<code>connection_key</code>	<code>db_conn</code>	Shared-store key for the DB connection string
<code>output_key</code>	<code>db_schema</code>	Shared-store key for the schema description

NL to SQL Node

Base class: Node

Translates a natural-language question into a SQL query using an LLM. Reads the schema from `schema_key` and the question from `question_key`; writes the generated SQL to `output_key`. Wire to a SQL Execute Node to run the query and get results. Supply a prompt file to control the system-level instructions and few-shot examples.

Property	Default	Description
<code>model</code>	<code>gpt-4o</code>	LLM model name
<code>schema_key</code>	<code>db_schema</code>	Shared-store key for schema context
<code>question_key</code>	<code>nl_question</code>	Shared-store key for the natural-language question
<code>output_key</code>	<code>sql_query</code>	Shared-store key for the generated SQL

SQL Execute Node

Base class: Node

Executes a SQL statement against a database and writes the result rows to the shared store. Reads the connection from `connection_key` and the query from

sql_key. Routes error on syntax errors or connection failures. The result is a list of dicts, one per row.

Property	Default	Description
connection_key	db_conn	Shared-store key for the DB connection string
sql_key	sql_query	Shared-store key for the SQL to execute
output_key	query_results	Shared-store key for the result rows

Voice / Audio

Speech to Text Node

Base class: Node

Transcribes an audio file to text using an ASR model (default: Whisper). Reads the audio file path from audio_key and writes the transcript to output_key. Set language to a BCP-47 language code (e.g. en, es) for better accuracy; leave empty for auto-detection. Routes error on file-not-found or API failure.

Property	Default	Description
model	whisper-1	ASR model name
audio_key	audio_path	Shared-store key for the audio file path
output_key	transcript	Shared-store key for the transcript
language	(empty)	ISO language code (empty = auto-detect)

Text to Speech Node

Base class: Node

Converts text to speech using a TTS model. Reads the input text from text_key, synthesises audio with the chosen voice, and writes the output audio file path to output_key. Pair with a Speech to Text Node for voice-in / voice-out pipelines.

Property	Default	Description
model	tts-1	TTS model name
voice	alloy	Voice identifier

Property	Default	Description
text_key	tts_text	Shared-store key for the input text
output_key	audio_path	Shared-store key for the output audio file path

Document / Vision

PDF Extract Node

Base class: Node

Extracts plain text from a PDF file. Reads the file path from pdf_key. Set pages to a range like "1 - 5" to extract only specific pages; leave empty to extract all pages. Writes the combined extracted text to output_key. Pair with a Text Chunk Node and Vector Index Node for PDF-based RAG.

Property	Default	Description
pdf_key	pdf_path	Shared-store key for the PDF file path
output_key	pdf_text	Shared-store key for the extracted text
pages	(empty)	Page range, e.g. 1 - 5 (empty = all pages)

Image Vision Node

Base class: Node

Sends an image to a vision-capable LLM and returns a description or analysis. Reads the image path or URL from image_key. Supply a prompt file to ask specific questions about the image (e.g. "List all text visible in this image" or "Describe the chart"). Writes the model response to output_key.

Property	Default	Description
model	gpt-4o	Vision-capable LLM model name
image_key	image_path	Shared-store key for image path or URL
output_key	vision_result	Shared-store key for the description/analysis

Data Validate Node

Base class: Node

Validates data in the shared store against a JSON Schema. Reads the data from `input_key` and the schema from `schema_key`. Routes `valid` if validation passes, `invalid` otherwise, and writes the list of validation error messages to `errors_key` for downstream error-handling or human review.

Property	Default	Description
<code>input_key</code>	<code>data</code>	Shared-store key for the data to validate
<code>schema_key</code>	<code>validation_schema</code>	Shared-store key for the JSON Schema
<code>errors_key</code>	<code>validation_errors</code>	Shared-store key for the error list

Code / Execution

Code Gen Node

Base class: Node

Generates source code from a specification using an LLM. Reads the spec from `spec_key`, writes the generated code to `output_key`. Set `language` to control the target (Python, JavaScript, SQL, etc.). Supply a prompt file to give the model style guidelines, import conventions, or a partial code template to fill in.

Property	Default	Description
<code>model</code>	<code>gpt-4o</code>	LLM model name
<code>language</code>	<code>python</code>	Target programming language
<code>spec_key</code>	<code>code_spec</code>	Shared-store key for the code specification
<code>output_key</code>	<code>generated_code</code>	Shared-store key for the generated code

Code Exec Node

Base class: Node

Executes Python code from the shared store in a subprocess. Reads code from `code_key`, runs it with a `timeout` second limit, and writes `stdout/result` to `output_key`. Enable `sandbox` to run in a restricted subprocess with limited imports. Routes error on non-zero exit code or timeout. Use after Code Gen Node for a `generate` → `execute` → `verify` pattern.

Property	Default	Description
code_key	generated_code	Shared-store key for code to execute
sandbox	true	Run in restricted subprocess
timeout	30	Execution timeout in seconds
output_key	exec_output	Shared-store key for stdout/result

Test Gen Node

Base class: Node

Generates test code for existing source code using an LLM. Reads source from code_key and writes test code to output_key. Pair with Code Exec Node to run the generated tests immediately and Judge Node to score their quality. Supports pytest and unittest framework conventions.

Property	Default	Description
model	gpt-4o	LLM model name
code_key	generated_code	Shared-store key for the source code
output_key	test_code	Shared-store key for generated test code
framework	pytest	Test framework: pytest or unittest

Data Processing

Map Node

Base class: Node

Applies a Python expression to every item in a list. Reads the list from input_key, evaluates transform with the current item bound to the name item, and writes the transformed list to output_key. Use for lightweight per-item transformations that do not require LLM calls. For LLM-per-item work use Batch Node or Async Parallel Batch Node instead.

Property	Default	Description
input_key	items	Shared-store key for the input list
output_key	mapped_items	Shared-store key for the output list
transform	(empty)	

Property	Default	Description
		Python expression applied to each item (item variable)

Reduce Node

Base class: Node

Folds a list down to a single value. Reads the list from `input_key`, applies the reducer function, and writes the result to `output_key`. Built-in reducers: `sum`, `concat` (join strings), `max`, `min`. Or supply a Python expression using `acc` (accumulator) and `item` variables. Use after a Map Node or Batch Node to aggregate results.

Property	Default	Description
<code>input_key</code>	<code>mapped_items</code>	Shared-store key for the input list
<code>output_key</code>	<code>reduced_result</code>	Shared-store key for the reduced value
<code>reducer</code>	<code>sum</code>	Built-in name or Python expression

Condition Node

Base class: Node

Evaluates a Python expression and routes to `true` or `false`. The expression has access to the shared store via the name `store`. Use this for simple boolean gates (e.g. `"store['score'] > 0.8"` or `"len(store['results']) == 0"`). For complex multi-way routing use Router Node instead.

Property	Default	Description
<code>expression</code>	<i>(empty)</i>	Python expression (accesses shared store as <code>store</code>)

Loop Counter Node

Base class: Node

Implements a fixed-count loop by tracking an iteration counter in the shared store. Routes continue until `max_iterations` is reached, then routes done. Wire

continue to the loop body and back to this node; wire done to the exit node.
Declare `counter_key` to give the iteration number a named slot in the store.

Property	Default	Description
<code>max_iterations</code>	10	Maximum loop iterations
<code>counter_key</code>	<code>loop_count</code>	Shared-store key for the iteration counter

Transform Node

Base class: Node

Reshapes or reformats a single value using a Jinja2 template or Python expression. Reads from `input_key`, applies the template, and writes to `output_key`. Use for struct reshaping (dict → list), format conversions (ISO date → human-readable), or any mapping that does not require a loop or LLM call.

Property	Default	Description
<code>input_key</code>	<code>data</code>	Shared-store key for input data
<code>output_key</code>	<code>transformed_data</code>	Shared-store key for output data
<code>template</code>	<i>(empty)</i>	Jinja2 template or Python expression

Merge Node

Base class: Node

Combines values from several shared-store keys into a single value. Reads the keys listed in `input_keys` (comma-separated) and writes a merged result to `output_key`. Choose a strategy: `dict_update` merges dicts left-to-right, `list_concat` concatenates lists, `string_join` joins strings with a newline. Use at the end of a parallel fan-out to recombine results.

Property	Default	Description
<code>input_keys</code>	<i>(empty)</i>	Comma-separated list of shared-store keys to merge
<code>output_key</code>	<code>merged_data</code>	Shared-store key for the merged result
<code>strategy</code>	<code>dict_update</code>	Merge strategy: <code>dict_update</code> , <code>list_concat</code> , <code>string_join</code>

Calendar

Calendar Read Node

Base class: Node

Reads events from a Google Calendar (or compatible CalDAV calendar). Reads a time range dict ({`"start": ...`, `"end": ...`}) from `time_range_key` and writes the list of event dicts to `output_key`. Requires OAuth credentials to be set up in the runtime environment. Routes error on authentication or API failure.

Property	Default	Description
<code>calendar_id</code>	<code>primary</code>	Calendar ID or primary
<code>time_range_key</code>	<code>time_range</code>	Shared-store key for the time range dict
<code>output_key</code>	<code>calendar_events</code>	Shared-store key for the events list

Calendar Write Node

Base class: Node

Creates an event in a Google Calendar. Reads event data (title, start, end, description, attendees) from the dict at `event_key` and writes the new event's ID to `output_key`. Pair with Calendar Read Node to implement scheduling workflows that check for conflicts before booking.

Property	Default	Description
<code>calendar_id</code>	<code>primary</code>	Calendar ID or primary
<code>event_key</code>	<code>new_event</code>	Shared-store key for the event data dict
<code>output_key</code>	<code>created_event_id</code>	Shared-store key for the created event ID

MCP / Agent Protocol

MCP Tool Node

Base class: Node

Calls a tool on an MCP (Model Context Protocol) server. Reads the tool arguments from `args_key` and writes the tool's return value to `output_key`. Set `server_url` and `tool_name` in the Inspector. Use to integrate any MCP-compatible tool server

(file systems, databases, browser automation, custom tools) into a flow without writing HTTP boilerplate.

Property	Default	Description
server_url	<i>(empty)</i>	MCP server URL
tool_name	<i>(empty)</i>	Tool name to invoke on the server
args_key	tool_args	Shared-store key for the tool arguments dict
output_key	tool_result	Shared-store key for the tool result

A2A Send Node

Base class: Node

Sends a message to another agent via the Agent-to-Agent (A2A) protocol. Reads the outgoing message from `message_key` and posts it to the target agent's A2A endpoint. Writes the server's acknowledgement to `output_key`. Use to build multi-agent systems where specialised agents hand off tasks to each other.

Property	Default	Description
target_agent_url	<i>(empty)</i>	Target agent's A2A endpoint URL
message_key	outgoing_message	Shared-store key for the message to send
output_key	send_response	Shared-store key for the acknowledgement

A2A Receive Node

Base class: Node

Waits for an incoming A2A message and writes it to the shared store. Polls `listen_key` (where the runtime places incoming messages) until a message arrives or `timeout` seconds elapse. Routes `timeout` if no message is received; routes `default` when a message arrives and is written to `output_key`.

Property	Default	Description
listen_key	a2a_inbox	Shared-store key where incoming messages are placed
timeout	30	Wait timeout in seconds

Property	Default	Description
output_key	received_message	Shared-store key for the received message

Observability / Utility

Log Node

Base class: Node

Emits a structured log entry at the configured log level. `message_template` supports `{{key}}` interpolation from the shared store. Optionally lists specific shared-store key names in `keys_to_log` to append their values to the entry. Always routes default. Use for debugging, audit trails, and pipeline telemetry.

Property	Default	Description
message_template	<i>(empty)</i>	Log message template (<code>{{key}}</code> interpolation)
level	INFO	Log level: DEBUG, INFO, WARNING, ERROR
keys_to_log	<i>(empty)</i>	Comma-separated shared-store keys to include

Timer Node

Base class: Node

Measures elapsed wall-clock time between two points in a flow. Set `mode = start` at the beginning of a section to record a timestamp, and `mode = stop` at the end to write elapsed milliseconds to `output_key`. Use `label` to distinguish multiple concurrent timers. Useful for performance profiling and SLA monitoring.

Property	Default	Description
label	timer	Label for the timing measurement
mode	start	Operation: start or stop
output_key	elapsed_ms	Shared-store key for elapsed milliseconds (stop mode)

Cache Node

Base class: Node

Provides a simple key-value cache backed by the shared store. On first access (miss) the flow continues to compute the value; on a repeat access (hit) the cached value is returned without re-running the expensive downstream nodes. Build the cache key from shared-store values using `{{key}}` interpolation in `cache_key_template`. Set `ttl = 0` for no expiry.

Property	Default	Description
<code>cache_key_template</code>	<i>(empty)</i>	Template for the cache lookup key
<code>value_key</code>	<code>cache_value</code>	Shared-store key for the cached/to-cache value
<code>ttl</code>	<code>300</code>	Cache TTL in seconds (0 = no expiry)

Trace Node

Base class: Node

Emits an OpenTelemetry span around the current flow step. Set `span_name` to label the span. List shared-store keys in `keys_to_trace` to attach their values as span attributes. Wrap a sub-section of a flow with matching Trace Nodes (start/end) to get fine-grained distributed tracing without modifying application code. Always routes default.

Property	Default	Description
<code>span_name</code>	<i>(empty)</i>	OpenTelemetry span name
<code>keys_to_trace</code>	<i>(empty)</i>	Comma-separated shared-store keys to attach as attributes

Data Structures / Memory

Registry Node

Base class: Node

Provides named-entry storage backed by a dict in the shared store. Supports `get`, `set`, `delete`, and `list` operations on `registry_key`. Routes `found/not_found` on a `get` that finds or misses the entry. Use for configuration look-ups, object registries, and any pattern where you need to name-and-retrieve arbitrary values.

Property	Default	Description
operation	get	Operation: get, set, delete, list
registry_key	registry	Shared-store key for the registry dict
entry_name	<i>(empty)</i>	Name of the entry to get/set/delete
value_key	registry_value	Shared-store key for the value to set or retrieved

Stack Push Node

Base class: Node

Pushes a value onto a list in the shared store, treating it as a LIFO stack. Reads the value from `value_key` and appends it to the end of the list at `stack_key` (creating the list if absent). Pair with Stack Pop Node to implement depth-first traversal, undo stacks, or backtracking patterns.

Property	Default	Description
stack_key	stack	Shared-store key for the stack list
value_key	push_value	Shared-store key for the value to push

Stack Pop Node

Base class: Node

Pops the top item from a LIFO stack list in the shared store. Reads the list at `stack_key`, removes and returns the last element, and writes it to `output_key`. Routes empty if the list is empty or absent, allowing the flow to detect stack-underflow without crashing.

Property	Default	Description
stack_key	stack	Shared-store key for the stack list
output_key	popped_value	Shared-store key for the popped value

Queue Enqueue Node

Base class: Node

Appends a value to the tail of a FIFO queue list in the shared store. Reads the value from `value_key` and appends it to the end of the list at `queue_key`. Pair with Queue Dequeue Node to implement first-in-first-out work queues, breadth-first traversal, or any ordered producer/consumer pattern.

Property	Default	Description
<code>queue_key</code>	<code>queue</code>	Shared-store key for the queue list
<code>value_key</code>	<code>enqueue_value</code>	Shared-store key for the value to enqueue

Queue Dequeue Node

Base class: Node

Removes and returns the head item from a FIFO queue list in the shared store. Reads the list at `queue_key`, removes the first element, and writes it to `output_key`. Routes empty if the queue is empty or absent. Use with Queue Enqueue Node for producer/consumer patterns and breadth-first graph traversal.

Property	Default	Description
<code>queue_key</code>	<code>queue</code>	Shared-store key for the queue list
<code>output_key</code>	<code>dequeued_value</code>	Shared-store key for the dequeued value

Local Memory Node

Base class: Node

Provides persistent slot-based memory backed by a dict in the shared store. Supports `store` (write to a named slot), `recall` (read from a slot), and `clear` (delete a slot or all slots) operations. Use for conversational memory, per-session state, or any pattern that needs to remember a value across multiple flow runs within a session.

Property	Default	Description
<code>operation</code>	<code>store</code>	Operation: <code>store</code> , <code>recall</code> , <code>clear</code>
<code>memory_key</code>	<code>local_memory</code>	Shared-store key for the memory dict
<code>slot</code>	<i>(empty)</i>	Memory slot name
<code>value_key</code>	<code>memory_value</code>	

Property	Default	Description
		Shared-store key for the value to store or retrieved

Reference

Node Type	Base Class	Primary Use
Start Node	Node	Flow entry point
Stop Node	Node	Flow exit point
Basic Node	Node	General-purpose single step
Router Node	Node	Conditional branching
Subflow Node	Node	Embedded sub-graph
LLM Prompt Node	Node	LLM call — inline string or file prompt
JSON LLM Node	Node	LLM call — structured JSON output
Classifier Node	Node	LLM-based label classification
Agent Node	Node	Autonomous tool-calling loop
RAG Node	Node	Embedding and retrieval
Judge Node	Node	LLM output evaluation
File Reader Node	Node	Read file from disk
File Writer Node	Node	Write file to disk
Python Tool Node	Node	Arbitrary Python / shell
Batch Node	BatchNode	Sequential batch processing
Async Node	AsyncNode	Single async I/O step
Async Batch Node	AsyncBatchNode	Sequential async batch
Async Parallel Batch Node	AsyncParallelBatchNode	Concurrent async batch
Human Review Node	Node	Human approval gate
Human Input Node	Node	Interactive user input
Chain of Thought Node	Node	Step-by-step LLM reasoning
Majority Vote Node	Node	Self-consistency via sampling
Supervisor Node	Node	Multi-agent orchestration

Node Type	Base Class	Primary Use
Debate Advocate Node	Node	Adversarial argument generation
Debate Judge Node	Node	Adversarial debate decision
Web Search Node	Node	Live web search
Web Scrape Node	Node	HTML → plain text extraction
API Call Node	Node	HTTP REST call
Text Chunk Node	Node	Long text → chunk list
Embed Node	Node	Text → embedding vector
Vector Index Node	Node	Build vector store index
Vector Retrieve Node	Node	Query vector store
DB Schema Node	Node	Inspect DB schema
NL to SQL Node	Node	Natural language → SQL
SQL Execute Node	Node	Run SQL query
Speech to Text Node	Node	Audio → transcript
Text to Speech Node	Node	Text → audio file
PDF Extract Node	Node	PDF → plain text
Image Vision Node	Node	Image → LLM description
Data Validate Node	Node	JSON Schema validation
Code Gen Node	Node	LLM code generation
Code Exec Node	Node	Execute code in subprocess
Test Gen Node	Node	LLM test generation
Map Node	Node	Apply expression to each item
Reduce Node	Node	Fold list to single value
Condition Node	Node	Boolean expression gate
Loop Counter Node	Node	Fixed-count loop
Transform Node	Node	Reshape / reformat data
Merge Node	Node	Combine multiple store keys
Calendar Read Node	Node	Read calendar events
Calendar Write Node	Node	Create calendar event
MCP Tool Node	Node	MCP server tool call
A2A Send Node	Node	Agent-to-agent message send

Node Type	Base Class	Primary Use
A2A Receive Node	Node	Agent-to-agent message receive
Log Node	Node	Structured log emission
Timer Node	Node	Elapsed time measurement
Cache Node	Node	Key-value cache with TTL
Trace Node	Node	OpenTelemetry span emission
Registry Node	Node	Named-entry object store
Stack Push Node	Node	LIFO stack push
Stack Pop Node	Node	LIFO stack pop
Queue Enqueue Node	Node	FIFO queue enqueue
Queue Dequeue Node	Node	FIFO queue dequeue
Local Memory Node	Node	Slot-based session memory
Shell Command Node	Node	Execute bash/sh/zsh/powershell/cmd
TTY Serial Node	Node	Serial port / Arduino / MCU I/O
Spreadsheet Node	Node	Read/write CSV, TSV, Excel
Socket Node	Node	TCP/UDP socket I/O
WebSocket Node	AsyncNode	Async WebSocket client
Webhook Trigger Node	Node	Wait for incoming HTTP POST
Context Compact Node	Node	Compact LLM context (strategy pattern)
Conversation History Node	Node	Manage multi-turn message list
Regex Node	Node	Pattern match / extract / replace
Template Render Node	Node	Jinja2 template rendering
JSON Parse Node	Node	Parse / serialize JSON
List Operations Node	Node	Filter / sort / slice / unique list
String Operations Node	Node	Split / join / strip / replace string
Retry Node	Node	Retry with exponential backoff

Node Type	Base Class	Primary Use
Rate Limiter Node	Node	Throttle call rate
Email Send Node	Node	Send email (SMTP / SendGrid)
Email Read Node	Node	Fetch email from IMAP
Notification Node	Node	Slack / Discord / Teams / Telegram
Secret Node	Node	Read secret from env / vault

System / Shell / Hardware

Shell Command Node

Base class: Node

Executes a shell command string in a subprocess. The `shell` property selects the interpreter: auto detects the platform (bash on Linux, zsh on macOS, PowerShell on Windows), or you can pin to bash, sh, zsh, powershell, or cmd. Reads the command from `command_key`, writes stdout and stderr to separate keys, and routes error on non-zero exit code.

Property	Default	Description
<code>shell</code>	auto	Shell interpreter: auto, bash, sh, zsh, powershell, cmd
<code>command_key</code>	command	Shared-store key for the command string
<code>timeout</code>	30	Execution timeout in seconds
<code>stdout_key</code>	stdout	Shared-store key for captured stdout
<code>stderr_key</code>	stderr	Shared-store key for captured stderr
<code>env_key</code>	(empty)	Shared-store key for extra environment variables dict

TTY Serial Node

Base class: Node

Reads or writes data over a serial (TTY/COM) port. Use for Arduino, Raspberry Pi, microcontrollers, instruments, and any device that communicates over a serial connection. The port path lives in the shared store so it can be configured at runtime. `operation`: `open` and `close` manage the connection; `readline` reads until a newline (most MCU sketches send newline-terminated sensor data); `read` reads up to a buffer; `write` sends data. Routes `timeout` on read timeout, error on port failure.

Property	Default	Description
<code>operation</code>	<code>readline</code>	Operation: <code>open</code> , <code>close</code> , <code>read</code> , <code>readline</code> , <code>write</code>
<code>port_key</code>	<code>serial_port</code>	Shared-store key for port path (<code>/dev/ttyUSB0</code> , <code>COM3</code> , <code>/dev/tty.usbmodem1</code>)
<code>baud_rate</code>	<code>9600</code>	Serial baud rate
<code>timeout</code>	<code>1.0</code>	Read timeout in seconds (0 = non-blocking)
<code>encoding</code>	<code>utf-8</code>	Decode encoding: <code>utf-8</code> , <code>ascii</code> , <code>bytes</code> (raw bytearray)
<code>data_key</code>	<code>serial_data</code>	Shared-store key for data to write (<code>write</code> operation)
<code>output_key</code>	<code>serial_read</code>	Shared-store key for received data (<code>read</code> / <code>readline</code>)

Spreadsheet Node

Base class: Node

Reads or writes tabular data in CSV, TSV, or Excel (`.xlsx/.xls`) format. `format`: `auto` detects the format from the file extension. Supports configurable delimiters and four quoting styles that map directly to Python's `csv.QUOTE_*` constants. For Excel, `sheet_name` selects the worksheet. Reads produce a list of dicts (header mode) or a list of lists (no-header mode), which feeds naturally into Map Node, Reduce Node, or any LLM Batch Node.

Property	Default	Description
<code>operation</code>	<code>read</code>	Operation: <code>read</code> , <code>write</code> , <code>append</code>
<code>file_key</code>	<code>file_path</code>	Shared-store key for the file path

Property	Default	Description
format	auto	Format: auto (from extension), csv, tsv, excel
delimiter	,	Field delimiter (CSV/TSV only)
quoting	minimal	Quoting: minimal, all, non_numeric, none
sheet_name	Sheet1	Excel sheet name (Excel only)
has_header	true	Treat first row as column headers
encoding	utf-8	File encoding (CSV/TSV only)
data_key	table_data	Shared-store key for data to write
output_key	table_data	Shared-store key for read data

Networking / Sockets

Socket Node

Base class: Node

Low-level TCP/UDP socket operations. The socket object is created on connect and stored in the shared store under `socket_key` so subsequent Send/Receive/Close calls in the same flow can share it. Use for custom protocols, legacy system integration, instrument control, and any scenario where HTTP is not the right transport.

Property	Default	Description
operation	connect	Operation: connect, send, receive, close
host_key	socket_host	Shared-store key for hostname or IP
port_key	socket_port	Shared-store key for port number
proto	tcp	Protocol: tcp or udp
socket_key	socket	

Property	Default	Description
		Shared-store key holding the socket object between operations
data_key	socket_data	Shared-store key for data to send
output_key	socket_recv	Shared-store key for received data
timeout	5.0	Operation timeout in seconds
buffer_size	4096	Receive buffer size in bytes

WebSocket Node

Base class: AsyncNode

Async WebSocket client for `ws://` and `wss://` connections. Like Socket Node but built on `asyncio` for non-blocking I/O — use this when you need streaming LLM responses, live data feeds, or real-time agent-to-agent communication over WebSocket. The connection object is stored in the shared store between operations.

Property	Default	Description
operation	connect	Operation: connect, send, receive, close
url_key	ws_url	Shared-store key for the WebSocket URL (<code>ws://</code> or <code>wss://</code>)
ws_key	ws_conn	Shared-store key holding the WebSocket connection object
data_key	ws_send	Shared-store key for message to send
output_key	ws_recv	Shared-store key for received message
timeout	10.0	Receive timeout in seconds

Webhook Trigger Node

Base class: Node

Starts a lightweight HTTP server on the configured port and path and blocks until a single incoming POST request arrives. Writes the request body to `output_key` and headers to `headers_key`, then routes `triggered`. Routes timeout if no request arrives within the configured window. Use as the entry node for event-driven flows — CI/CD hooks, payment notifications, IoT push events, or any external system that calls back into a flow.

Property	Default	Description
port	8080	HTTP port to listen on
path	/webhook	URL path to listen on
timeout	60	Maximum seconds to wait for a POST request
output_key	webhook_payload	Shared-store key for the received request body
headers_key	webhook_headers	Shared-store key for the received request headers

AI / LLM Utilities

Context Compact Node

Base class: Node

Reduces the size of a message list or text block before sending to an LLM, preventing context window overflow. Uses a strategy pattern — swap the algorithm by changing the `strategy` property without rewiring the graph.

Strategy	Description
truncate	Keep the first or last N tokens; fast and deterministic
sliding_window	Keep the most recent N messages; preserves recency
summarize	LLM call that distills older content into a compact summary
extractive	Key-sentence extraction via TF-IDF / KeyBERT; no LLM needed
semantic_dedup	Embed all chunks and drop near-duplicates above a cosine threshold

Property	Default	Description
strategy	sliding_window	Compaction algorithm (see table above)

Property	Default	Description
input_key	messages	Shared-store key for the message list or text to compact
output_key	messages	Shared-store key to write the compacted result
max_tokens	2000	Target maximum tokens after compaction
model	(empty)	LLM model for summarize strategy (empty = project default)
similarity_threshold	0.92	Cosine similarity cutoff for semantic_dedup

Conversation History Node

Base class: Node

Manages a messages list (role + content dicts) in the shared store for multi-turn chat flows. Supports four operations: `append` adds a new message with the specified role; `trim` enforces a maximum message count; `clear` resets the list; `format` renders the list to a plain string for LLM APIs that take a single prompt rather than a message list.

Property	Default	Description
operation	append	Operation: append, trim, clear, format
history_key	messages	Shared-store key for the message list
role	user	Role for append: user, assistant, system
content_key	content	Shared-store key for the message content to append
max_messages	20	Maximum messages to keep on trim
output_key	chat_str	Shared-store key for formatted string output (format operation)

Text / Data Processing

Regex Node

Base class: Node

Applies a regular expression to a string in the shared store. `findall` returns a list of all matches. `match` / `search` return the first match object and route matched or `no_match`. `replace` returns the substituted string. `split` returns a list of segments. All results are written to `output_key`.

Property	Default	Description
operation	<code>findall</code>	Operation: <code>match</code> , <code>search</code> , <code>findall</code> , <code>replace</code> , <code>split</code>
pattern	<i>(empty)</i>	Regular expression pattern string
flags	<i>(empty)</i>	Regex flags: <code>i</code> (ignore case), <code>m</code> (multiline), <code>s</code> (dot-all)
input_key	<code>text</code>	Shared-store key for the input string
replacement	<i>(empty)</i>	Replacement string for <code>replace</code> (supports <code>\1</code> groups)
output_key	<code>regex_result</code>	Shared-store key for the result

Template Render Node

Base class: Node

Renders a Jinja2 template using shared store values as context. Set `template_type = string` for an inline template; set `template_type = path` and provide a path to a `.j2` or `.md` file for larger templates. Every key in the shared store is available as a template variable. Use for generating prompts, reports, emails, and any text that varies by run-time data.

Property	Default	Description
template_type	<code>string</code>	Template source: <code>string</code> (inline) or <code>path</code> (file)
template	<i>(empty)</i>	Inline Jinja2 template or path to <code>.j2</code> / <code>.md</code> file
output_key	<code>rendered</code>	

Property	Default	Description
		Shared-store key to write the rendered string

JSON Parse Node

Base class: Node

Parses a JSON string to a Python dict/list, or serializes a dict/list back to a JSON string. Routes error on malformed input. Use after API Call Node (response body is a JSON string) or before a node that needs a structured object. `indent > 0` produces pretty-printed output; `indent = 0` produces compact output.

Property	Default	Description
operation	parse	Operation: parse (string→dict) or serialize (dict→string)
input_key	json_str	Shared-store key for JSON string (parse) or object (serialize)
output_key	json_obj	Shared-store key for the result
indent	0	JSON indentation for serialize (0 = compact)

List Operations Node

Base class: Node

Performs collection operations on a list in the shared store. `filter` and `sort` accept a Python expression with an `item` variable. `slice` uses `start/stop` indices. `unique` removes duplicates (preserves order). `flatten` unpacks nested lists one level deep. `reverse` reverses in place. `count` writes the list length to `output_key`. Routes empty when the result is an empty list.

Property	Default	Description
operation	filter	Operation: filter, sort, slice, unique, flatten, reverse, count
input_key	items	Shared-store key for the input list
output_key	items	Shared-store key for the result

Property	Default	Description
expression	<i>(empty)</i>	Python expression for filter/sort (item variable)
start	0	Start index for slice
stop	-1	Stop index for slice (-1 = end)

String Operations Node

Base class: Node

Applies string transformations to a value in the shared store. All operations read from `input_key` and write to `output_key`. `split` produces a list; `join` expects a list as input and produces a string. `format` uses `{key}` placeholders resolved from the shared store. `truncate` appends ... if the string exceeds `max_length`.

Property	Default	Description
operation	strip	Operation: split, join, strip, upper, lower, replace, format, truncate
input_key	text	Shared-store key for the input string (or list for join)
output_key	text	Shared-store key for the result
separator		Separator for split and join
find	<i>(empty)</i>	Find string for replace
replacement	<i>(empty)</i>	Replacement string for replace
max_length	200	Maximum length for truncate
template	<i>(empty)</i>	Format template string (<code>{key}</code> placeholders) for format

Resilience / Flow Utilities

Retry Node

Base class: Node

Implements retry-with-exponential-backoff for a section of the flow. Place this node before the operation that might fail; wire its `retry` action back to the start of the fallible section and its `done` action to the success path. The node reads a status key to decide whether to retry or declare success; `status_key = "ok"` signals done. Routes `give_up` when `max_attempts` is exhausted.

Property	Default	Description
<code>max_attempts</code>	3	Maximum retry attempts before routing <code>give_up</code>
<code>backoff_base</code>	1.0	Base delay in seconds (doubles each retry)
<code>jitter</code>	true	Add random jitter to backoff delay
<code>attempt_key</code>	<code>retry_attempt</code>	Shared-store key for the current attempt counter
<code>status_key</code>	<code>retry_status</code>	Shared-store key read to decide retry vs done (<code>ok</code> = done)

Rate Limiter Node

Base class: Node

Enforces a per-minute rate limit by sleeping between calls when the last call was too recent. Reads a timestamp from `timestamp_key`, computes the required sleep duration, and writes the updated timestamp before routing `default`. Use `label` to maintain independent rate limiters for different API endpoints in the same flow.

Property	Default	Description
<code>calls_per_min</code>	60	Maximum calls allowed per minute
<code>timestamp_key</code>	<code>last_call_time</code>	Shared-store key for the last-call Unix timestamp
<code>label</code>	<code>default</code>	Rate limiter label (allows multiple independent limiters)

Messaging / Notifications

Email Send Node

Base class: Node

Sends an email via SMTP or a transactional email API (SendGrid, Mailgun). Reads subject, body, and recipients from the shared store. Set `html = true` to send HTML email. The `output_key` receives the server's message ID or delivery receipt on sent, or an error description on error.

Property	Default	Description
provider	smtp	Email provider: smtp, sendgrid, mailgun
to_key	email_to	Shared-store key for recipient address(es) (string or list)
subject_key	email_subject	Shared-store key for the email subject
body_key	email_body	Shared-store key for the email body (plain text or HTML)
html	false	Treat body as HTML
output_key	email_result	Shared-store key for send result / message ID

Email Read Node

Base class: Node

Fetches emails from an IMAP mailbox (or Gmail via Gmail API) and writes a list of message dicts to the shared store. Each dict contains `subject`, `from`, `body`, `date`, and `message_id` fields. Routes `no_mail` when the inbox is empty. Use to build flows that process inbound email — automated replies, ticket routing, data extraction from email reports.

Property	Default	Description
provider	imap	Mail provider: imap, gmail
folder	INBOX	Mailbox folder to check
max_messages	10	Maximum messages to fetch

Property	Default	Description
unread_only	true	Fetch only unread messages
output_key	emails	Shared-store key for the list of message dicts

Notification Node

Base class: Node

Sends a notification to a messaging platform via webhook. Supports Slack (Incoming Webhooks), Discord (Webhook URLs), Microsoft Teams (Connector cards), and Telegram (Bot API). Reads the platform webhook URL from the shared store. Set `title` for platforms that support rich message formatting (Slack blocks, Teams cards).

Property	Default	Description
channel	slack	Platform: slack, discord, teams, telegram
webhook_url_key	webhook_url	Shared-store key for the platform webhook URL
message_key	notification	Shared-store key for the message text
title	<i>(empty)</i>	Optional title / header for rich message formats

Security / Configuration

Secret Node

Base class: Node

Reads a secret or configuration value from an environment variable, `.env` file, AWS Secrets Manager, or HashiCorp Vault, and writes it to the shared store. Use this at the start of a flow to load credentials that should not be hard-coded in the Inspector or exported code. Routes `not_found` when `required = true` and the secret is missing; writes `None` when `required = false`.

Property	Default	Description
source	env	Secret source: env, dotenv, aws_secrets, vault

Property	Default	Description
secret_name	(empty)	Environment variable name or secret path/name
output_key	secret	Shared-store key to write the retrieved secret value
required	true	Route not_found when missing; write None when false

Addon Nodes — Geospatial

Addon nodes ship in `addon_nodes/` and appear under the **Scientific & Engineering** divider in the palette. Install them via **Tools → Node Type Library** if they are not already visible.

USGS Elevation Point Node

Base class: Node · **Actions:** default, error

Returns ground elevation (metres or feet) for a single latitude/longitude point via the USGS Elevation Point Query Service (EPQS). No API key required.

Property	Default	Description
lat_key	lat	Shared-store key holding latitude (decimal degrees)
lon_key	lon	Shared-store key holding longitude (decimal degrees)
units	Meters	Elevation units: Meters or Feet
result_key	elevation_result	Key to write the result dict (lat, lon, elevation, units)

USGS 3DEP Elevation Node

Base class: Node · **Actions:** default, error

Downloads a 3DEP Digital Elevation Model raster for a bounding box via the USGS WCS service and saves it as a GeoTIFF. Resolutions: 1/3, 1, or 2 arc-second.

Property	Default	Description
bbox_key	bbox	

Property	Default	Description
		Key holding [west, south, east, north] in decimal degrees
resolution	1/3	DEM resolution: 1/3, 1, or 2 arc-second
output_dir_key	dem_output_dir	Key holding the output directory path
result_key	dem_result	Key for result dict (output_path, crs, bbox)

National Map Download Node

Base class: Node · **Actions:** default, error

Searches The National Map (TNM) API for available USGS datasets within a bounding box. Can optionally download the discovered files. Supports NED, NHD, NAIP, Topo, and more.

Property	Default	Description
bbox_key	bbox	Key holding [west, south, east, north]
dataset	National Elevation Dataset (NED) 1/3 arc-second	TNM dataset product type
max_items	10	Maximum dataset records to return
download	false	If true, download files to output_dir_key
output_dir_key	tnm_output_dir	Directory for downloaded files
result_key	tnm_result	Key for result dict (items list, each with title, downloadURL, sizeInBytes)

Earthquake Catalog Node

Base class: Node · **Actions:** default, error

Fetches earthquake events from the USGS FDSN/ComCat API filtered by bounding box, time range, and minimum magnitude. Returns a list of event dicts. No API key required.

Property	Default	Description
bbox_key	bbox	Key holding [west, south, east, north]
start_time	2024-01-01	Start date (YYYY-MM-DD)
end_time	<i>(empty = now)</i>	End date; leave blank for the current time
min_mag	2.5	Minimum earthquake magnitude
max_results	100	Maximum events to return
result_key	eq_events	Key for list of event dicts (magnitude, depth_km, time, place, id)

Landsat Search & Download Node

Base class: Node · **Actions:** default, error

Searches and optionally downloads Landsat Collection 2 Level-2 scenes from the USGS M2M API. Requires a free USGS EarthExplorer account.

Property	Default	Description
username_key	usgs_username	Key holding USGS EarthExplorer username
token_key	usgs_token	Key holding USGS API token
bbox_key	bbox	Key holding [west, south, east, north]
dataset	landsat_ot_c2_l2	Landsat dataset identifier
start_date	<i>(empty)</i>	Search start date (YYYY-MM-DD)
max_cloud_pct	20	Maximum cloud cover percentage
max_results	10	Maximum scenes to return
download	false	If true, download scene data
output_dir_key	landsat_output_dir	Directory for downloaded scenes
result_key	landsat_scenes	Key for list of scene dicts

ShakeMap Fetch Node

Base class: Node · **Actions:** default, error

Downloads ShakeMap ground-motion products for a USGS earthquake event ID from `earthquake.usgs.gov`. No API key required.

Property	Default	Description
<code>event_id_key</code>	<code>eq_event_id</code>	Key holding the USGS event ID (e.g. <code>us7000n7n5</code>)
<code>product_type</code>	<code>download/grid.xml</code>	ShakeMap product to fetch
<code>output_dir_key</code>	<code>shakemap_output_dir</code>	Directory for downloaded files
<code>result_key</code>	<code>shakemap_fetch_result</code>	Key for result dict (<code>event_id</code> , <code>status</code> , <code>downloaded_files</code>)

ShakeMap Scenario Node

Base class: Node · **Actions:** default, error

Runs USGS ShakeMap v4 locally to generate a ground-motion scenario for a user-supplied `event.xml` fault-rupture definition. Requires a local ShakeMap v4 installation.

Property	Default	Description
<code>event_dir_key</code>	<code>shakemap_event_dir</code>	Key holding path to the event directory (contains <code>event.xml</code>)
<code>commands</code>	<code>assemble, model, contour</code>	Comma-separated ShakeMap pipeline steps
<code>result_key</code>	<code>shakemap_scenario_result</code>	Key for result dict (<code>status</code> , <code>grid_path</code> , <code>commands_run</code>)

Addon Nodes — Hydrology / Water

USGS Water Data Node

Base class: Node · **Actions:** default, error

Fetches instantaneous or daily time-series data from the USGS National Water Information System (NWIS) REST API for a specified gauge site and parameter. No API key required.

Property	Default	Description
site_key	usgs_site	Key holding the USGS site number (e.g. 01638500)
param_cd	00060	Parameter code: 00060 = discharge (cfs), 00065 = gage height (ft)
period	P7D	ISO 8601 period string (e.g. P7D, P1Y)
stat_cd	00003	Statistic code: 00003 = daily mean; blank = instantaneous
result_key	water_data	Key for result dict (site_name, param_name, values list)

NWIS Query Node

Base class: Node · **Actions:** default, error

Queries USGS NWIS in one of three modes: site (station metadata), peak (annual peak flow record), or stat (long-term monthly statistics).

Property	Default	Description
query_type	site	Query mode: site, peak, or stat
site_key	usgs_site	Key holding USGS site number
state_cd	(empty)	Two-letter state code for site mode searches
result_key	nwis_result	Key for result dict (contents depend on query_type)

StreamStats Basin Node

Base class: Node · **Actions:** default, error

Delineates a watershed and computes basin characteristics (drainage area, mean elevation, mean annual precipitation, etc.) via the USGS StreamStats REST API.

Property	Default	Description
lat_key	lat	Key holding pour-point latitude
lon_key	lon	Key holding pour-point longitude
state_cd	VA	Two-letter US state code (upper-case)
result_key	basin_result	Key for result dict (basin_characteristics dict, workspace_id)

SWMM Run Node

Base class: Node · **Actions:** default, error

Runs an EPA SWMM 5 stormwater simulation from a .inp file. Returns peak conduit flows, junction flooding volumes, and subcatchment runoff. Requires swmm5 on the PATH.

Property	Default	Description
inp_path_key	swmm_inp_path	Key holding path to the SWMM .inp file
report_key	swmm_rpt_path	Key holding path for the output .rpt report
result_key	swmm_result	Key for result dict (conduit_peak_flows, junction_flooding, status)

EPANET Run Node

Base class: Node · **Actions:** default, error

Runs an EPA EPANET 2 water distribution network simulation from a .inp file. Returns nodal pressures (psi), pipe flows, and velocities. Requires EPANET installed.

Property	Default	Description
inp_path_key	epanet_inp_path	Key holding path to the EPANET .inp file
result_key	epanet_result	Key for result dict (node_pressures,

Property	Default	Description
		pipe_flows, pipe_velocities)

MODFLOW 6 Run Node

Base class: Node · **Actions:** default, error

Runs a USGS MODFLOW 6 groundwater simulation from a pre-configured simulation directory containing `mf sim.nam`. Returns convergence status and head statistics. Requires `mf6`.

Property	Default	Description
<code>sim_dir_key</code>	<code>mf6_sim_dir</code>	Key holding path to the MODFLOW 6 simulation directory
<code>result_key</code>	<code>mf6_result</code>	Key for result dict (status, converged, iterations, head_stats)

FloPy Model Node

Base class: Node · **Actions:** default, error

Runs a MODFLOW simulation managed by a FloPy MFSimulation (or Modflow) object that you construct in Python and pass via the shared store. Returns head statistics.

Property	Default	Description
<code>model_key</code>	<code>flopy_model</code>	Key holding a configured FloPy model/simulation instance
<code>exe_name</code>	<code>mf6</code>	MODFLOW executable name (mf6, mf2005, mfnwt, etc.)
<code>result_key</code>	<code>flopy_result</code>	Key for result dict (success, head_stats, head file path)

pyWatershed Node

Base class: Node · **Actions:** default, error

Runs a USGS National Hydrologic Model (NHM) simulation via the pywatershed Python package. Returns streamflow and storage statistics for the simulated period.

Property	Default	Description
domain_dir_key	pws_domain_dir	Key holding path to the NHM domain directory
control_file_key	pws_control_file	Key holding the control file path (blank = control.yml in domain dir)
result_key	pws_result	Key for result dict (status, n_hru, n_days, streamflow_stats)

Addon Nodes — Weather / Atmosphere

NOAA Weather Node

Base class: Node · **Actions:** default, error

Fetches current NWS surface observations and the 7-day forecast for any US latitude/ longitude point via `api.weather.gov`. No API key required. US locations only.

Property	Default	Description
lat_key	lat	Key holding latitude (decimal degrees)
lon_key	lon	Key holding longitude (decimal degrees)
result_key	noaa_weather	Key for result dict (current_observation, forecast_periods list)

WRF Model Node

Base class: Node · **Actions:** default, error

Runs WRF-ARW (`real.exe` + `wrf.exe`) in a pre-configured WRF run directory. Returns wrfout file paths and run status. Requires a compiled WRF installation with MPI.

Property	Default	Description
run_dir_key	wrf_run_dir	Key holding absolute path to the WRF run directory
nprocs	4	Number of MPI processes
skip_real	false	Skip real.exe if wrfinput_d01 already exists
result_key	wrf_result	Key for result dict (status, real_status, wrf_status, elapsed_seconds, wrfout_files)

Addon Nodes — Building Energy

EnergyPlus Run Node

Base class: Node · **Actions:** default, error

Runs a DOE EnergyPlus building energy simulation from an IDF (or epJSON) input file and EPW weather file. Returns annual energy use and end-use breakdown. Requires energyplus.

Property	Default	Description
idf_path_key	eplus_idf_path	Key holding path to the .idf or .epJSON input file
weather_path_key	eplus_weather_path	Key holding path to the .epw weather file
output_dir_key	eplus_output_dir	Key holding the output directory path
result_key	eplus_result	Key for result dict (total_site_energy_kwh, peak_electricity_kw, end_uses, idf_name)

Addon Nodes — Aerospace

Open VSP Geometry Node

Base class: Node · **Actions:** default, error

Loads a .vsp3 OpenVSP parametric geometry model, optionally applies design variable overrides, and exports the geometry in a selected format. Uses the openvsp Python API.

Property	Default	Description
vsp3_path_key	vsp3_path	Key holding path to the .vsp3 model file
export_format	stl	Export format: stl, degen_geom, stp, iges, obj
design_vars_key	vsp_design_vars	Key holding design variable override dict (nested: container→group→param→value)
output_path_key	vsp_output_path	Key written with the exported file path

VSPAERO Analysis Node

Base class: Node · **Actions:** default, error

Runs the VSPAERO vortex-lattice / panel method aerodynamic solver on a DegenGeom CSV file (exported by the Open VSP Geometry Node) and returns CL, CD, CMy, and span efficiency.

Property	Default	Description
degen_geom_key	vsp_output_path	Key holding path to the DegenGeom CSV file
alpha	0.0	Angle of attack in degrees
mach	0.1	Freestream Mach number
result_key	vspaero_result	Key for result dict (CL, CD, CMy, e, alpha, mach)

SU2 CFD Node

Base class: Node · **Actions:** default, error

Runs SU2_CFD (the Stanford open-source CFD solver) with a user-supplied .cfg configuration file. Supports Euler, RANS, and other PDE systems.

Property	Default	Description
config_path_key	su2_config_path	Key holding path to the SU2 .cfg file
nprocs	1	Number of MPI processes (1 = serial)

Property	Default	Description
result_key	su2_result	Key for result dict (CL, CD, CMy, iterations, converged, final_residual)

Cart3D Analysis Node

Base class: Node · **Actions:** default, error

Runs a NASA Cart3D inviscid Euler CFD analysis. Cart3D auto-generates a Cartesian mesh from a closed triangulated surface — no manual meshing required.

Property	Default	Description
case_dir_key	cart3d_case_dir	Key holding path to the Cart3D case directory
aoa	0.0	Angle of attack in degrees
mach	0.5	Freestream Mach number
result_key	cart3d_result	Key for result dict (CL, CD, CM, alpha)

FUN3D Run Node

Base class: Node · **Actions:** default, error

Runs NASA FUN3D (nodet_mpi) for high-fidelity RANS or LES CFD in a pre-configured case directory containing fun3d.nml and an unstructured grid.

Property	Default	Description
case_dir_key	fun3d_case_dir	Key holding path to the FUN3D case directory
nprocs	4	Number of MPI processes
result_key	fun3d_result	Key for result dict (CL, CD, CM, iterations, converged, residual_history)

NASA CEA Node

Base class: Node · **Actions:** default, error

Computes rocket combustion thermochemical properties (Isp, Tc, γ , MW, C*) using the NASA CEA code via the rocketcea Python wrapper. No separate CEA installation needed.

Property	Default	Description
oxid	L0X	Oxidiser name (e.g. L0X, N2O4, IRFNA)
fuel	LH2	Fuel name (e.g. LH2, RP1, MMH, Methane)
pc_psia	1000	Chamber pressure in psia
eps	40	Nozzle expansion ratio (exit/throat area)
of_ratio	6.0	Oxidiser-to-fuel mass ratio
result_key	cea_result	Key for result dict (Isp_vac, Isp_del, T_chamber_K, gamma, MW, cstar)

RocketPy Flight Node

Base class: Node · **Actions:** default, error

Runs a RocketPy 6-DOF rocket flight simulation. You construct the rocketpy.Flight object in a preceding Basic Node and pass it via the shared store.

Property	Default	Description
flight_key	rocketpy_flight	Key holding a configured rocketpy.Flight instance
result_key	rocketpy_result	Key for result dict (apogee_m, max_speed_ms, max_mach, max_accel_ms2, flight_time_s, apogee_time_s)

GMAT Script Node

Base class: Node · **Actions:** default, error

Runs a GMAT orbital mechanics script in batch (headless) mode and optionally parses a GMAT report file to return numerical results. Requires a GMAT installation.

Property	Default	Description
script_path_key	gmat_script_path	Key holding path to the .script file
report_path_key	gmat_report_path	Key holding path to the expected report file
result_key	gmat_result	Key for result dict (status, script_path, report_data list of row dicts)

OpenMDAO Model Node

Base class: Node · **Actions:** default, error

Executes an OpenMDAO multidisciplinary analysis or optimisation problem. Accepts a configured `openmdao.api.Problem` object built in a preceding Basic Node.

Property	Default	Description
problem_key	openmdao_problem	Key holding a configured <code>openmdao.api.Problem</code> instance
driver	run_model	<code>run_model = MDA</code> analysis only; <code>run_driver = full</code> optimisation
result_key	openmdao_result	Key for result dict (design_variables, objectives, constraints, converged)

Optimization Node

Base class: Node · **Actions:** default, error

Minimises a scalar objective function using `scipy.optimize.minimize`. You supply a Python callable and an initial guess vector via the shared store.

Property	Default	Description
objective_key	objective_fn	Key holding a callable $f(x) \rightarrow \text{float}$
x0_key	x0	Key holding the initial guess (list or ndarray)
method	SLSQP	SciPy optimisation method: SLSQP, Nelder-Mead, BFGS, L-BFGS-B, etc.
bounds_key	(empty)	Key holding a list of (min, max) tuples
constraints_key	(empty)	Key holding a list of SciPy constraint dicts
result_key	opt_result	Key for result dict (x_optimal, f_optimal, success, nfev, message)

NASA Trick Simulation Node

Base class: Node · **Actions:** default, error

Builds (optionally) and runs a NASA Trick simulation. Reads Trick log files after the run and returns extracted variable time-series arrays.

Property	Default	Description
sim_dir_key	trick_sim_dir	Key holding path to the Trick simulation directory
input_file_key	trick_input_file	Key holding the Python input file path (relative to sim dir)
build	true	If true, run trick-CP to compile before executing
log_vars_key	trick_log_vars	Key holding a list of Trick variable names to extract from log files
result_key	trick_result	Key for result dict (build_status, run_status, sim_time_s, log_data)

Addon Nodes — Wind Energy

OpenFAST Node

Base class: Node · **Actions:** default, error

Runs an NREL OpenFAST aero-elastic wind turbine simulation from a .fst primary input file and returns rotor performance summary statistics. Requires openfast on the PATH.

Property	Default	Description
fst_path_key	openfast_fst_path	Key holding path to the primary .fst input file
result_key	openfast_result	Key for result dict (status, sim_time_s, elapsed_wall_s, performance_summary)

The performance_summary dict includes GenPwr_mean_kW, RtAeroFxb_mean_kN, RtTSR_mean, and RotSpeed_mean_rpm.

KiteFAST Node

Base class: Node · **Actions:** default, error

Runs an NREL KiteFAST airborne-wind-energy (AWE) simulation and returns tether tension, electrical power, and flight cycle statistics. Requires the kitefast executable.

Property	Default	Description
input_path_key	kitefast_input_path	Key holding path to the KiteFAST primary input file
result_key	kitefast_result	Key for result dict (status, sim_time_s, summary)

The summary dict includes mean_tether_tension_kN, mean_electrical_power_kW, flight_cycles, and mean_altitude_m.

Addon Nodes — Scientific Computing

MATLAB Engine Node

Base class: Node · **Actions:** default, error

Calls a MATLAB function or script via the `matlab.engine` Python interface and returns the first output argument. Requires a licensed MATLAB installation with the Python engine.

Property	Default	Description
<code>script_key</code>	<code>matlab_script</code>	Key holding the MATLAB function/script name to call
<code>args_key</code>	<code>matlab_args</code>	Key holding a list of positional arguments
<code>result_key</code>	<code>matlab_result</code>	Key for result dict (output, <code>matlab_version</code> , <code>status</code>)

Octave Script Node

Base class: Node · **Actions:** default, error

Executes a GNU Octave script file or inline Octave expression in batch mode and returns a named workspace variable. Only requires `octave` on the PATH — no Python interface.

Property	Default	Description
<code>script_path_key</code>	<code>octave_script_path</code>	Key holding path to a <code>.m</code> script file (blank = use inline code)
<code>inline_code_key</code>	<code>octave_code</code>	Key holding an inline Octave expression or script body
<code>result_var</code>	<code>result</code>	Name of the Octave workspace variable to extract and return
<code>result_key</code>	<code>octave_result</code>	Key for result dict (<code>stdout</code> , <code>status</code> , <code>result_var_value</code>)

Addon Nodes — Data Catalog

USGS Data Catalog Search Node

Base class: Node · **Actions:** default, error

Searches the USGS ScienceBase Catalog by keyword and returns a list of dataset records with titles, summaries, DOIs, and download links. No API key required.

Property	Default	Description
query_key	sciencebase_query	Key holding the keyword search string
max_results	20	Maximum catalog items to return
fields	id,title,summary,link	Comma-separated ScienceBase metadata fields to include
result_key	sb_results	Key for result dict (items list, total_count)

- [Getting to Know Nodes — Series Index](#)
- [Tutorials](#)
- [About PocketFlow](#)
- [Help Home](#)